O'REILLY®

2nd Edition

Early Release

RAW & UNEDITED

R Graphics Cookbook

PRACTICAL RECIPES FOR VISUALIZING DATA

Winston Chang

R Graphics Cookbook

Second Edition

Practical Recipes for Visualizing Data

Winston Chang

R Graphics Cookbook

by Winston Chang

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com/safari). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Editor: Marie Beaugureau
- Production Editor: Kristen Brown
- Copyeditor: FILL IN
- Proofreader: FILL IN
- Indexer: FILL IN
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Rebecca Demarest

- April 2018: Second Edition

Revision History for the Early Release

- 2017-11-09: First release

See http://oreilly.com/catalog/errata.csp?isbn=9781491978603 for release details.

978-1-491-97860-3

[LSI]

Chapter 1. R Basics

This chapter covers the basics: installing and using packages and loading data.

If you want to get started quickly, most of the recipes in this book require the ggplot2 and gcookbook packages to be installed on your computer. To do this, run:

```
install.packages(c("ggplot2", "gcookbook"))
```

Then, in each R session, before running the examples in this book, you can load them with:

```
library(ggplot2)
library(gcookbook)
```

Note

Chapter \@ref(CHAPTER-GGPLOT2) provides an introduction to the ggplot2 graphing package, for readers who are not already familiar with its use.

Packages in R are collections of functions and/or data that are bundled up for easy distribution, and installing a package will extend the functionality of R on your computer. If an R user creates a package and thinks that it might be useful for others, that user can distribute it through a package repository. The primary repository for distributing R packages is called CRAN (the Comprehensive R Archive Network), but there are others, such as Bioconductor and Omegahat.

1.1 Installing a Package

Problem

You want to install a package from CRAN.

Solution

Use install.packages() and give it the name of the package you want to install. To install ggplot2, run:

```
install.packages("ggplot2")
```

At this point you may be prompted to select a download mirror. It’s usually best to use the first choice, https://cloud.r-project.org/, as it is a cloud-based mirror with endpoints all over the world.

Discussion

When you tell R to install a package, it will automatically install any other packages that the first package depends on.

CRAN (the Comprehensive R Archive Network) is a repository of packages for R, and it is mirrored on many servers around the world. It is the default repository system used by R. There are other package repositories; Bioconductor, for example, is a repository of packages related to analyzing genomic data.

1.2 Loading a Package

Problem

You want to load an installed package.

Solution

Use `library()` and give it the name of the package you want to install. To load ggplot2, run:

```
library(ggplot2)
```

The package must already be installed on the computer.

Discussion

Most of the recipes in this book require loading a package before running the code, either for the graphing capabilities (as in the ggplot2 package) or for example data sets (as in the MASS and gcookbook packages).

One of R's quirks is the package/library terminology. Although you use the `library()` function to load a package, a package is not a library, and some longtime R users will get irate if you call it that.

A *library* is a directory that contains a set of packages. You might, for example, have a system-wide library as well as a library for each user.

1.3 Loading a Delimited Text Data File

Problem

You want to load data from a delimited text file.

Solution

The most common way to read in a file is to use comma-separated values (CSV) data:

```
data <- read.csv("datafile.csv")
```

Discussion

Since data files have many different formats, there are many options for loading them. For example, if the data file does *not* have headers in the first row:

```
data <- read.csv("datafile.csv", header = FALSE)
```

The resulting data frame will have columns named `V1`, `V2`, and so on, and you will probably want to rename them manually:

```
# Manually assign the header names
names(data) <- c("Column1", "Column2", "Column3")
```

You can set the delimiter with sep. If it is space-delimited, use `sep = " "`. If it is tab-delimited, use `\t`, as in:

```
data <- read.csv("datafile.csv", sep = "\t")
```

By default, strings in the data are treated as factors. Suppose this is your data file, and you read it in using `read.csv()`:

```
"First","Last","Sex","Number"
"Currer","Bell","F",2
"Dr.","Seuss","M",49
"","Student",NA,21
```

The resulting data frame will store `First` and `Last` as *factors*, though it makes more sense in this case to treat them as strings (or *character vectors* in R terminology). To differentiate this, use `stringsAsFactors=FALSE`. If there are any columns that should be treated as factors, you can then convert them individually:

```
data <- read.csv("datafile.csv", stringsAsFactors = FALSE)
# Convert to factor
data$Sex <- factor(data$Sex)
str(data)
#> 'data.frame': 3 obs. of 4 variables:
#> $ First : chr "Currer" "Dr." ""
#> $ Last : chr "Bell" "Seuss" "Student"
#> $ Sex : Factor w/ 2 levels "F","M": 1 2 NA
#> $ Number: int 2 49 21
```

TODO: Fix text output formatting

Alternatively, you could load the file with strings as factors, and then convert individual columns from factors to characters.

See Also

`read.csv()` is a convenience wrapper function around `read.table()`. If you need more control over the input, see `?read.table`.

1.4 Loading Data from an Excel File

Problem

You want to load data from an Excel file.

Solution

The readxl package has the function `read_excel()` for reading .xls and .xlsx files from Excel. This will read the first sheet of an Excel spreadsheet:

```
# Only need to install once
install.packages("readxl")

library(readxl)
data <- read_excel("datafile.xlsx", 1)
```

Discussion

With `read_excel()`, you can load from other sheets by specifying a number for sheetIndex or a name for sheetName:

```
data <- read_excel("datafile.xls", sheet = 2)

data <- read_excel("datafile.xls", sheet = "Revenues")
```

`read_excel()` uses the first row of the spreadsheet for column names. If you don't want to use that row for column names, use `col_names = FALSE`. The columns will instead be named `X1`, `X2`, and so on.

By default, `read_excel()` will infer the type of each column, but if you want to specify the type of each column, you can use the `col_types` argument. You can also drop columns if you specify the type as `"blank"`.

```
# Drop the first column, and specify the types of the next three columns
data <- read_excel("datafile.xls", col_types = c("blank", "text", "date", "numeric"))
```

See Also

See `?read_excel` for more options controlling the reading of these files.

There are other packages for reading Excel files. The gdata package has a function `read.xls()` for reading in .xls files, and the xlsx package has a function `read.xlsx()` for reading in .xlsx files. They require external software to be installed on your computer: `read.xls()` requires Java, and `read.xlsx()` requires Perl.

1.5 Loading Data from SPSS/SAS/Stata Files

Problem

You want to load data from a SPSS file, or from other programs like SAS or Stata.

Solution

The haven package has the function `read_sav()` for reading SPSS files. To load data from an SPSS file:

```
# Only need to install the first time
install.packages("haven")

library(foreign)
data <- read_sav("datafile.sav")
```

Discussion

The haven package also includes functions to read from other formats:

- `read_sas()`: SAS
- `read_dta()`: Stata

An alternative to haven is the foreign package. It also supports SPSS and Stata files, but it is not as up-to-date as the functions from haven. For example, it only supports Stata files up to version 12, while haven supports up to version 14 (the current version as of this writing).

The foreign package does support some other formats, including:

- `read.octave()`: Octave and MATLAB
- `read.systat()`:SYSTAT
- `read.xport()`: SAS XPORT
- `read.dta()`: Stata
- `read.spss()`: SPSS

See Also

Run `ls("package:foreign")` for a full list of functions in the foreign package.

Chapter 2. Scatter Plots

Scatter plots are used to display the relationship between two continuous variables. In a scatter plot, each observation in a data set is represented by a point. Often, a scatter plot will also have a line showing the predicted values based on some statistical model. This is easy to do with R and ggplot2, and can help to make sense of data when the trends aren't immediately obvious just by looking at it.

With large data sets, it can be problematic to plot every single observation because the points will be overplotted, obscuring one another. When this happens, you'll probably want to summarize the data before displaying it. We'll also see how to do that in this chapter.

2.1 Making a Basic Scatter Plot

Problem

You want to make a scatter plot.

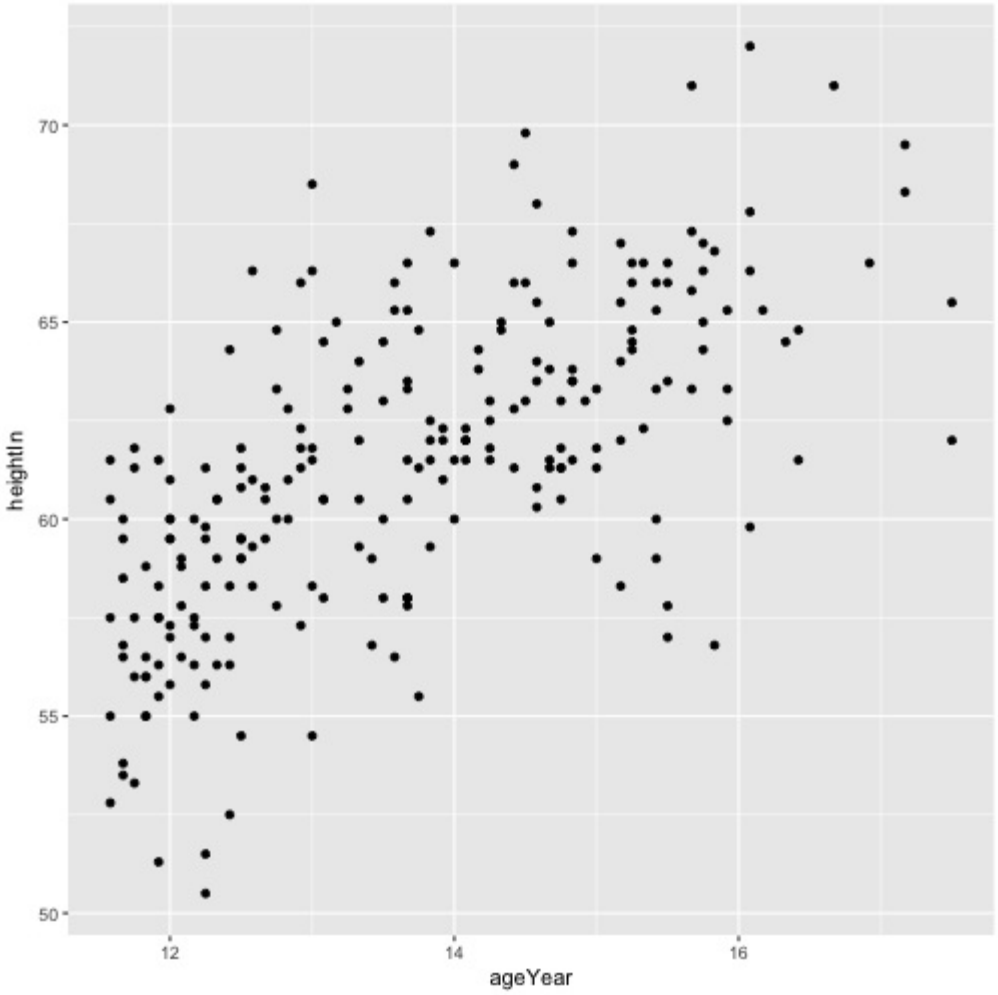
Solution

Use `geom_point()`, and map one variable to `x` and one to `y`.

In the `heightweight` data set, there are a number of columns, but we'll only use two in this example (Figure \@ref(fig:FIG-SCATTER-BASIC)):

```
library(gcookbook) # For the data set
# List the two columns we'll use
heightweight[, c("ageYear", "heightIn")]
#>   ageYear heightIn
#> 1   11.92     56.3
#> 2   12.92     62.3
#> 3   12.75     63.3
#> 4   13.42     59.0
#> 5   15.92     62.5
#> 6   14.25     62.5
#> 7   15.42     59.0
#> 8   11.83     56.5
#> # ... with 228 more rows
```

```
ggplot(heightweight, aes(x = ageYear, y = heightIn)) + geom_point()
```



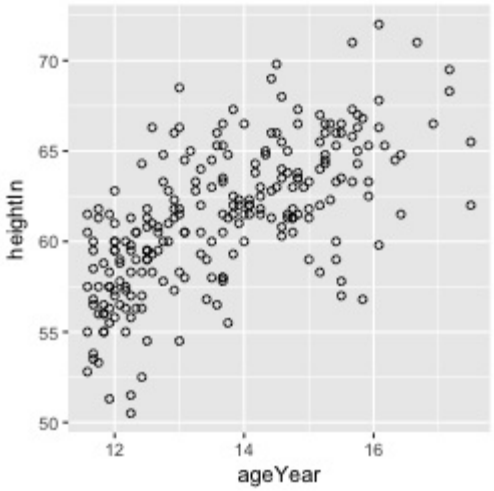
Discussion

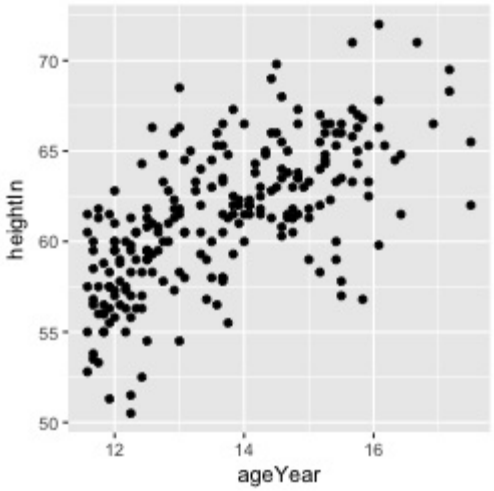
To use different shapes in a scatter plot, set the `shape` aesthetic. A common alternative to the default solid circles (shape #19) is hollow ones (#21), as seen in Figure \@ref(fig:FIG-SCATTER-BASIC-SHAPE-SIZE) (left):

```
ggplot(heightweight, aes(x = ageYear, y = heightIn)) + geom_point(shape = 21)
```

The size of the points can be controlled with the `size` aesthetic. The default value of size is 2. The following will set `size=1.5`, for smaller points (Figure \@ref(fig:FIG-SCATTER-BASIC-SHAPE-SIZE), right):

```
ggplot(heightweight, aes(x = ageYear, y = heightIn)) + geom_point(size = 1.5)
```





2.2 Grouping Data Points by a Variable Using Shape or Color

Problem

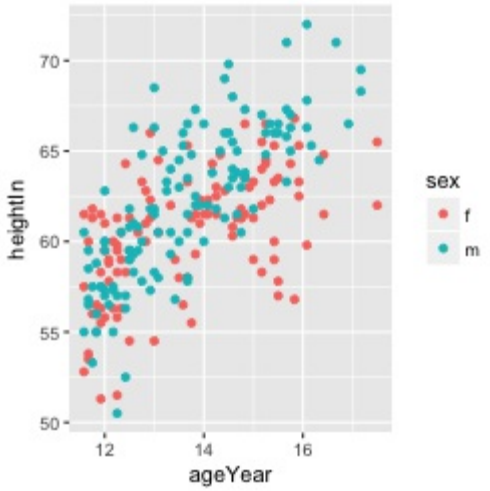
You want to visually group points by some variable, using shape or color.

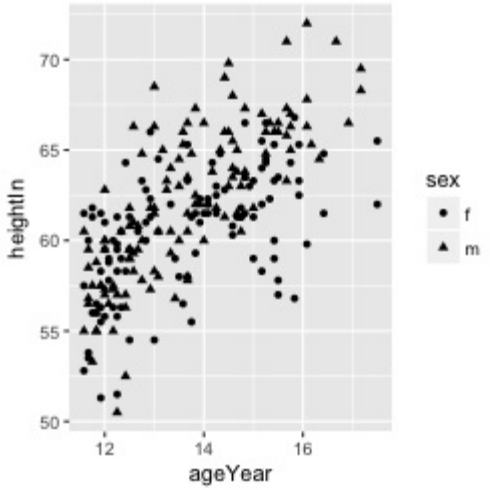
Solution

Map the grouping variable to `shape` or `colour`. In the `heightweight` data set, there are many columns, but we'll only use three of them in this example:

```
library(gcookbook) # For the data set
# Show the three columns we'll use
heightweight[, c("sex", "ageYear", "heightIn")]
#>   sex ageYear heightIn
#> 1   f   11.92     56.3
#> 2   f   12.92     62.3
#> 3   f   12.75     63.3
#> 4   f   13.42     59.0
#> 5   f   15.92     62.5
#> 6   f   14.25     62.5
#> 7   f   15.42     59.0
#> 8   f   11.83     56.5
#> # ... with 228 more rows
```

We can group points on the variable sex, by mapping sex to one of the aesthetics colour or shape (Figure \@ref(fig:FIG-SCATTER-SHAPE-COLOR)):





Discussion

The grouping variable must be categorical--in other words, a factor or character vector. If it is stored as a vector of numeric values, it should be converted to a factor before it is used as a grouping variable.

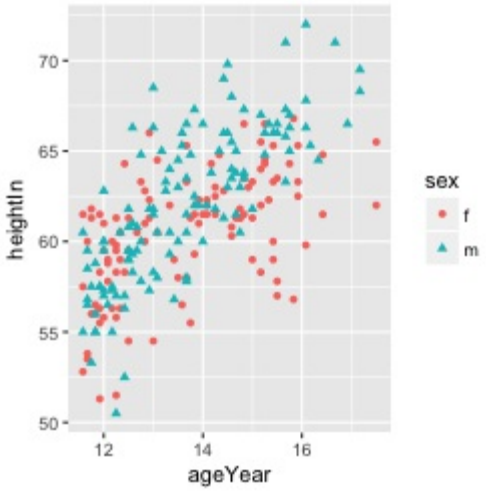
It is possible to map a variable to both `shape` and `colour`, or, if you have multiple grouping variables, to map different variables to them. Here, we'll map `sex` to `shape` and `colour` (Figure \@ref(fig:FIG-SCATTER-SHAPE-COLOR-BOTH), left):

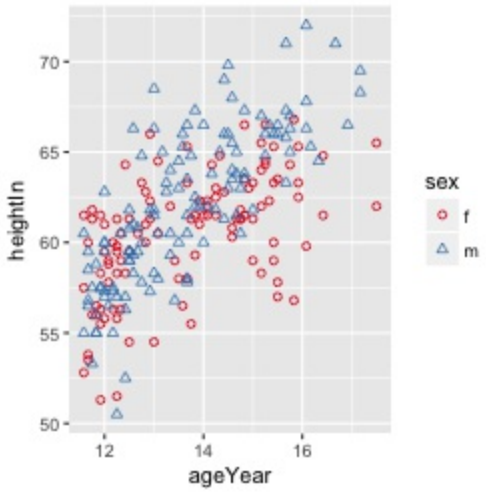
```
ggplot(heightweight, aes(x = ageYear, y = heightIn, shape = sex, colour = sex)) +
    geom_point()
```

The default shapes and colors may not be very appealing. Other shapes can be used with `scale_shape_manual()`, and other colors can be used with `scale_colour_brewer()` or `scale_colour_manual()`.

This will set different shapes and colors for the grouping variables (Figure \@ref(fig:FIG-SCATTER-SHAPE-COLOR-BOTH), right):

```
ggplot(heightweight, aes(x = ageYear, y = heightIn, shape = sex, colour = sex)) +
    geom_point() +
    scale_shape_manual(values = c(1,2)) +
    scale_colour_brewer(palette = "Set1")
```




70
65
60
55
50
heightIn
12
14
16
ageYear
sex
f
m

See Also

To use different shapes, see Recipe \@ref(RECIPE-SCATTER-SHAPES).

For more on using different colors, see Chapter \@ref(CHAPTER-COLORS).

2.3 Using Different Point Shapes

Problem

You want to use point shapes that are different from the defaults.

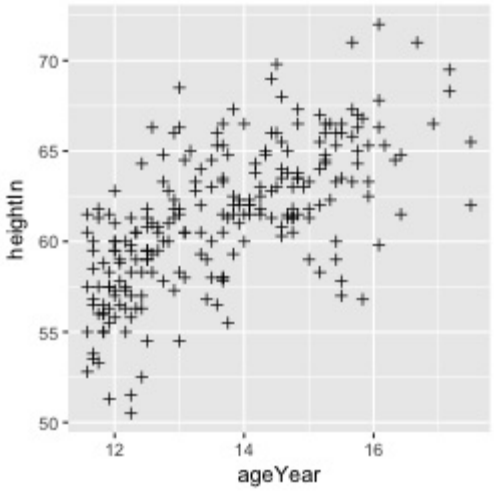
Solution

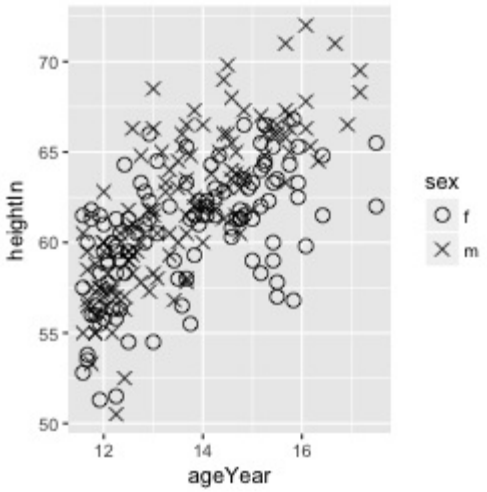
If you want to set the shape of all the points (Figure \@ref(fig:FIG-SCATTER-SHAPES)), specify the shape in `geom_point()`:

```
library(gcookbook) # For the data set

ggplot(heightweight, aes(x = ageYear, y = heightIn)) + geom_point(shape = 3)
```

If you have mapped a variable to shape, use `scale_shape_manual()` to change the shapes:

```
# Use slightly larger points and use a shape scale with custom values
ggplot(heightweight, aes(x = ageYear, y = heightIn, shape = sex)) +
    geom_point(size = 3) +
    scale_shape_manual(values = c(1, 4))
```

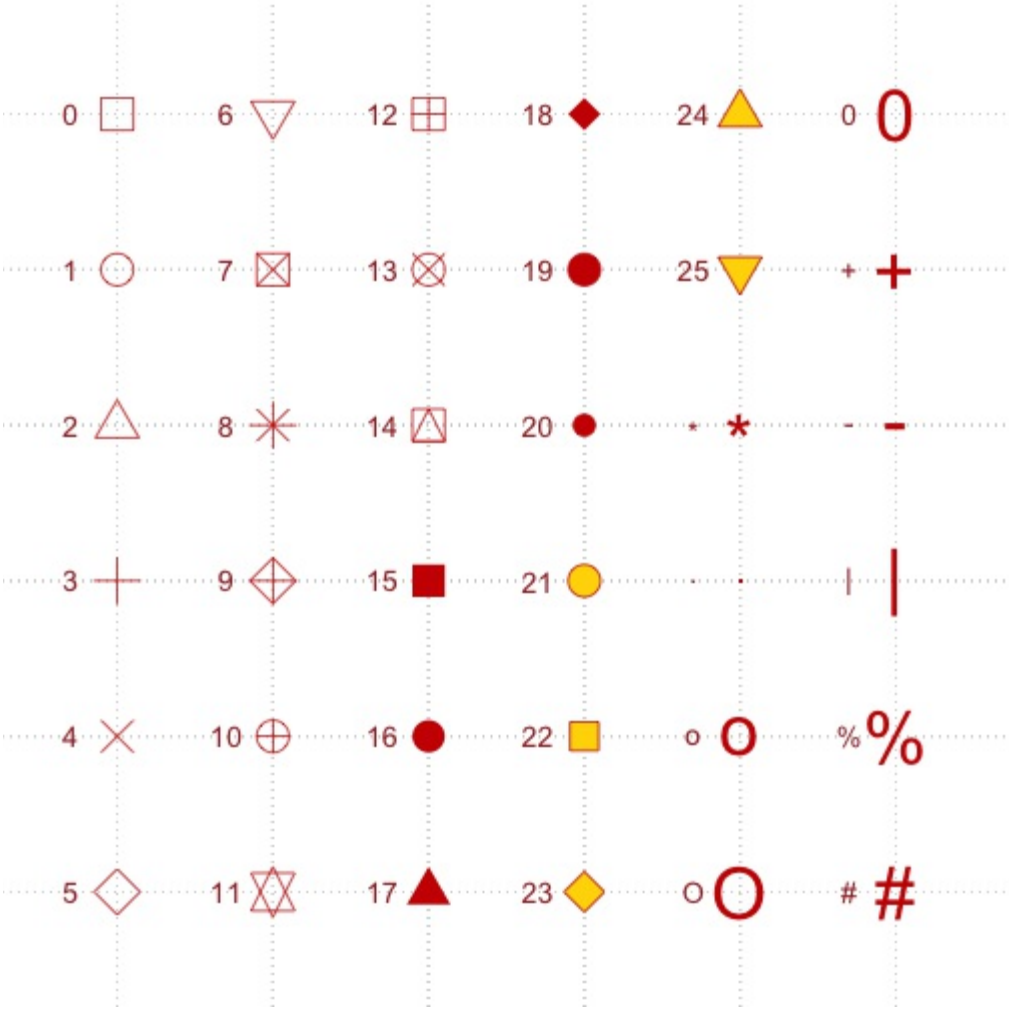




Discussion

Figure \@ref(fig:FIG-SCATTER-SHAPES-CHART) shows the shapes that are available in R graphics. Some of the point shapes (1–14) have just an outline, some (15–20) have just a solid fill, and some (21–25) have an outline and fill that can be controlled separately. (You can also use characters for points.)

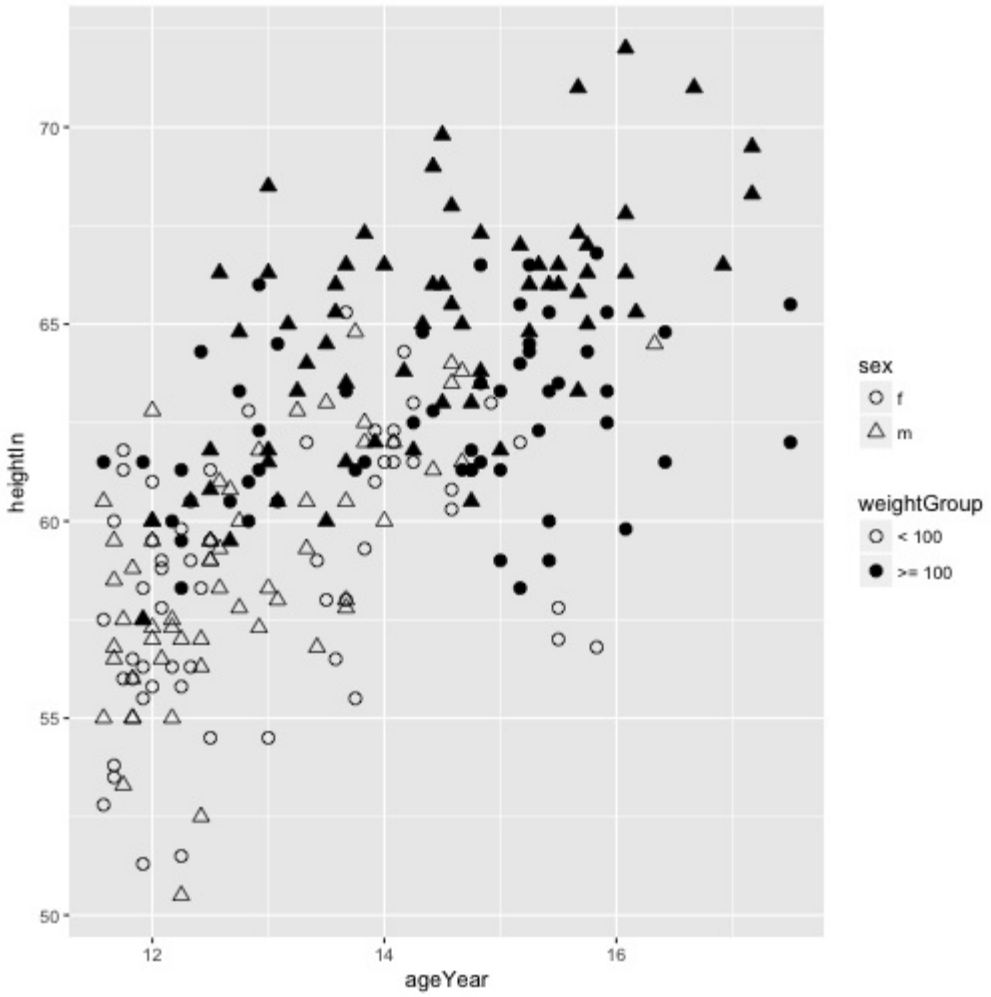
For shapes 1–20, the color of the entire point -- even the points that are solid -- is controlled by the `colour` aesthetic. For shapes 21–25, the outline is controlled by `colour` and the fill is controlled by `fill`.



It’s possible to have the shape represent one variable and the fill (empty or solid) represent another variable. This is done a little indirectly, by choosing shapes that have both colour and fill, and a color palette that includes `NA` and another color (the `NA` will result in a hollow shape). For example, we’ll take the `heightweight` data set and add another column that indicates whether the child weighed 100 pounds or more (Figure \@ref(fig:FIG-SCATTER-SHAPES-FILL)):

```
# Make a copy of the data
hw <- heightweight
# Categorize into <100 and >=100 groups
hw$weightGroup <- cut(hw$weightLb, breaks = c(-Inf, 100, Inf),
                      labels = c("< 100", ">= 100"))
```

```
# Use shapes with fill and color, and use colors that are empty (NA) and
# filled
ggplot(hw, aes(x = ageYear, y = heightIn, shape = sex, fill = weightGroup)) +
    geom_point(size = 2.5) +
    scale_shape_manual(values = c(21, 24)) +
    scale_fill_manual(values = c(NA, "black"),
      guide = guide_legend(override.aes = list(shape = 21)))
```



See Also

For more on using different colors, see Chapter \@ref(CHAPTER-COLORS).

For more information about recoding a continuous variable to a categorical one, see Recipe \@ref(RECIPE-DATAPREP-RECODE-CONTINUOUS).

2.4 Mapping a Continuous Variable to Color or Size

Problem

You want to represent a third continuous variable using color or size.

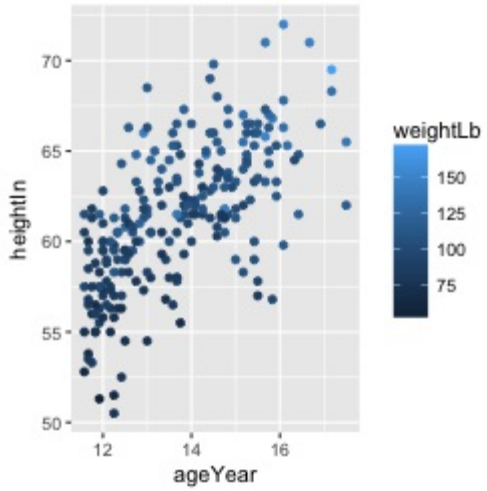
Solution

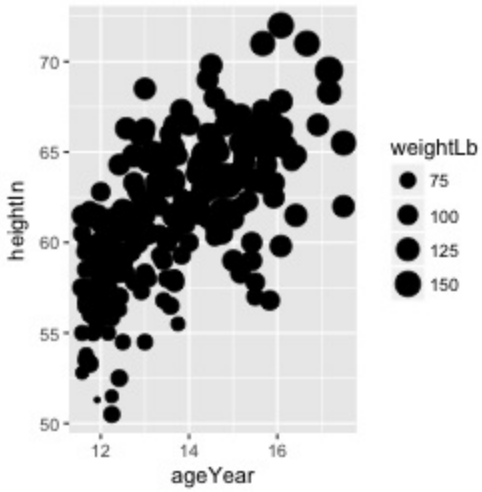
Map the continuous variable to `size` or `colour`. In the `heightweight` data set, there are many columns, but we'll only use four of them in this example:

```
library(gcookbook) # For the data set
# List the four columns we'll use
heightweight[, c("sex", "ageYear", "heightIn", "weightLb")]
#>   sex ageYear heightIn weightLb
#> 1   f   11.92     56.3     85.0
#> 2   f   12.92     62.3    105.0
#> 3   f   12.75     63.3    108.0
#> 4   f   13.42     59.0     92.0
#> 5   f   15.92     62.5    112.5
#> 6   f   14.25     62.5    112.0
#> 7   f   15.42     59.0    104.0
#> 8   f   11.83     56.5     69.0
#> # ... with 228 more rows
```

The basic scatter plot in Recipe \@ref(RECIPE-SCATTER-BASIC-SCATTER) shows the relationship between the continuous variables `ageYear` and `heightIn`. To represent a third continuous variable, `weightLb`, we must map it to another aesthetic property. We can map it to `colour` or `size`, as shown in Figure \@ref(fig:FIG-SCATTER-CONTINUOUS-COLOR-SIZE):

```
ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = weightLb)) +
    geom_point()

ggplot(heightweight, aes(x = ageYear, y = heightIn, size = weightLb)) +
    geom_point()
```




70
65
60
55
50
heightIn
12
14
16
ageYear
weightLb
75
100
125
150

Discussion

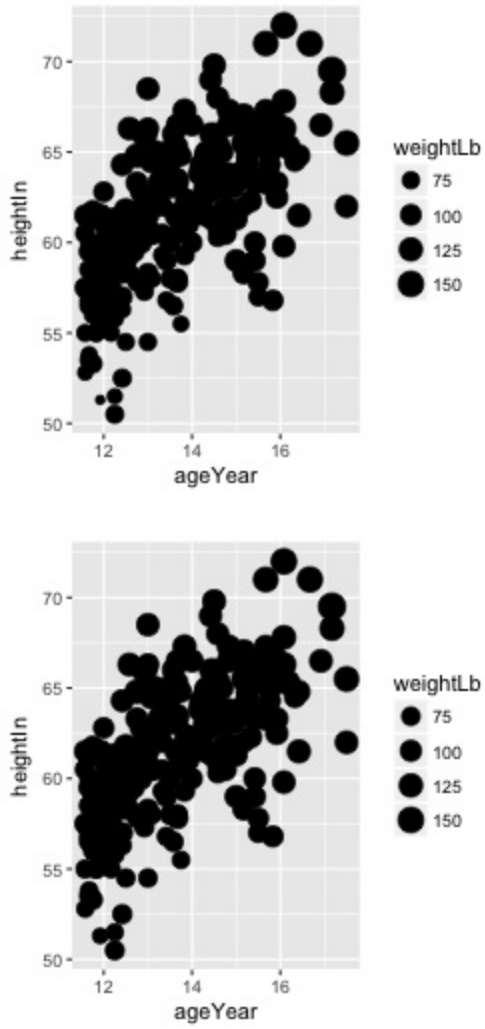
A basic scatter plot shows the relationship between two continuous variables: one mapped to the x-axis, and one to the y-axis. When there are more than two continuous variables, they must be mapped to other aesthetics, like size and color.

We can easily perceive small differences in spatial position, so we can interpret the variables mapped to *x* and *y* coordinates with high precision. We aren't very good at perceiving small differences in size and color, though, so we will interpret variables mapped to these aesthetic attributes with a much lower precision. When you map a variable to one of these properties, it should be one where precision is not very important for interpretation.

When a variable is mapped to `size`, the results can be perceptually misleading. The largest dots in Figure \@ref(fig:FIG-SCATTER-CONTINUOUS-COLOR-SIZE) have about 36 times the area of the smallest ones, but they represent only about 3.5 times the weight. This is because the by default the diameter of points goes from 1 to 6mm. If the data values go from 0 to 10, then the a value of 0 is represented with a 1mm point, and a value of 10 is represented with a 6mm point. Similarly, if the values go from 100 to 110, the 100 is represented with a 1mm point, and the 110 is represented with a 6mm point. And in all cases, the largest point will have 6 times the diameter of the smallest, and 36 times the area.

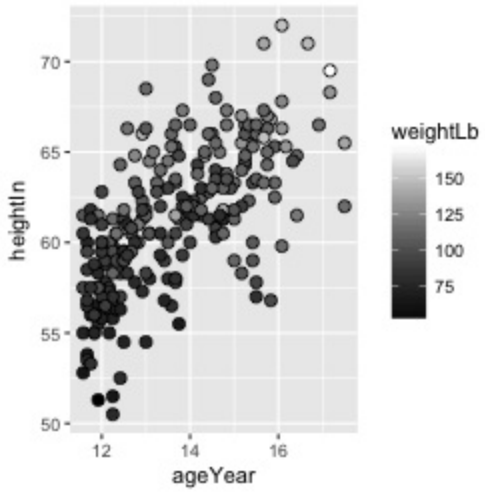
If it is important for the sizes to proportionally represent the quantities, you should first decide if you want the diameter of the points to represent the value, or if you want to area of the points to represent the value. Figure \@ref(fig:FIG-SCATTER-SIZE-AREA) shows the difference between these representations.

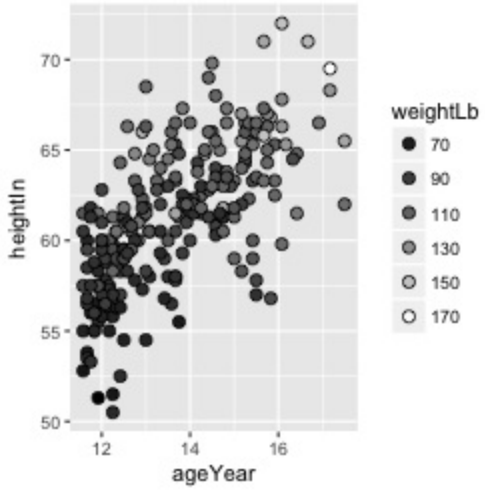
```
range(heightweight$weightLb)
#> [1]  50.5 171.5
size_range <- range(heightweight$weightLb) / max(heightweight$weightLb) *  6
size_range
#> [1] 1.766764 6.000000

ggplot(heightweight, aes(x = ageYear, y = heightIn, size = weightLb)) +
    geom_point() +
    scale_size_continuous(range = size_range)

ggplot(heightweight, aes(x = ageYear, y = heightIn, size = weightLb)) +
    geom_point() +
    scale_size_area()
```



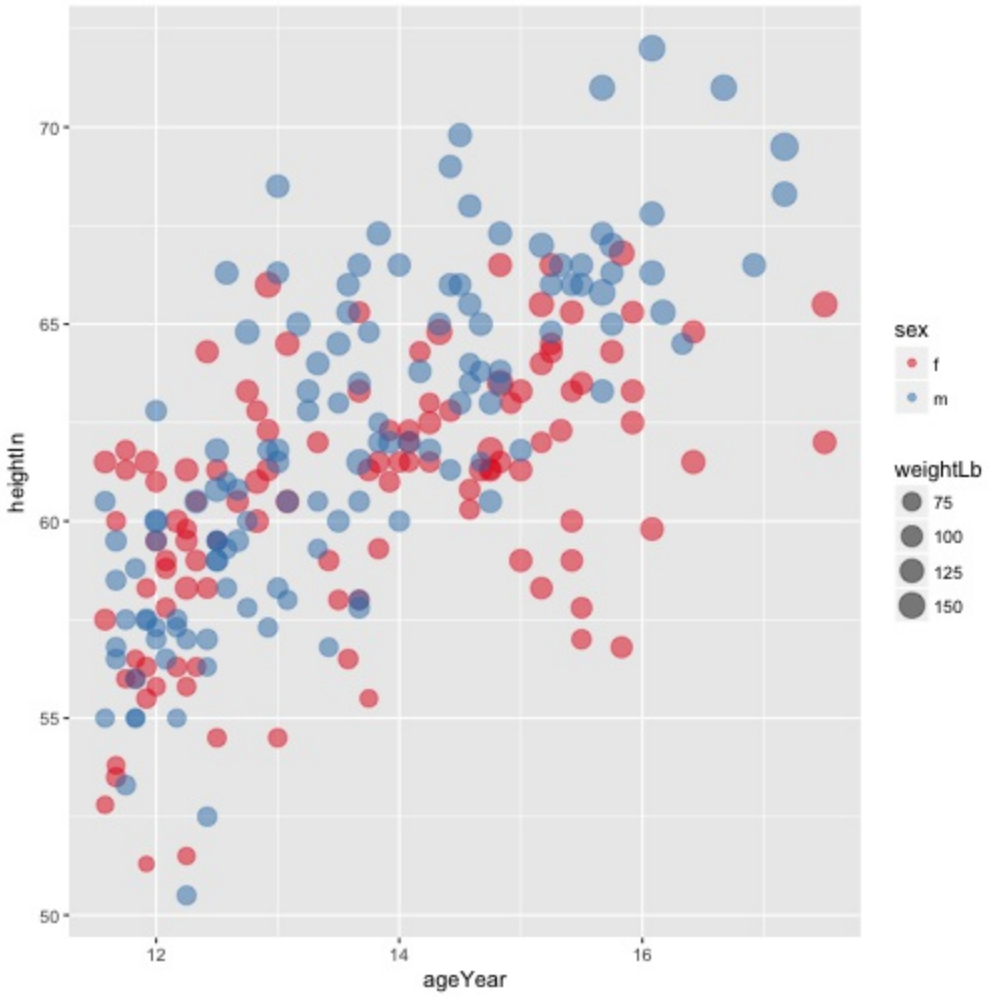
See Recipe \@ref(RECIPE-SCATTER-BALLOON) for details on making the area of points proportional to the value.

When it comes to color, there are actually two aesthetic attributes that can be used: `colour` and `fill`. For most point shapes, you use colour. However, shapes 21–25 have an outline with a solid region in the middle where the color is controlled by fill. These outlined shapes can be useful when using a color scale with light colors, as in Figure \@ref(fig:FIG-SCATTER-CONTINUOUS-FILL), because the outline sets them off from the background. In this example, we also set the fill gradient to go from black to white and make the points larger so that the fill is easier to see:





When we map a continuous variable to an aesthetic, that doesn't prevent us from mapping a categorical variable to other aesthetics. In Figure \@ref(fig:FIG-SCATTER-CONTINUOUS-SIZE-CATEGORICAL-COLOR), we'll map weightLb to size, and also map sex to colour. Because there is a fair amount of overplotting, we'll make the points 50% transparent by setting alpha=.5. We'll also use scale_size_area() to make the area of the points proportional to the value (see Recipe \@ref(RECIPE-SCATTER-BALLOON)), and change the color palette to one that is a little more appealing:



When a variable is mapped to size, it's a good idea to *not* map a variable to shape. This is because it is difficult to compare the sizes of different shapes; for example, a size 4 triangle could appear larger than a size 3.5 circle. Also, some of the shapes really are different sizes: shapes 16 and 19 are both circles, but at any given numeric size, shape 19 circles are visually larger than shape 16 circles.

See Also

To use different colors from the default, see Recipe \@ref(RECIPE-COLORS-PALETTE-CONTINUOUS).

See Recipe \@ref(RECIPE-SCATTER-BALLOON) for creating a balloon plot.

2.5 Dealing with Overplotting

Problem

You have many points and they obscure each other.

Solution

With large data sets, the points in a scatter plot may obscure each other and prevent the viewer from accurately assessing the distribution of the data. This is called *overplotting*. If the amount of overplotting is low, you may be able to alleviate it by using smaller points, or by using a different shape (like shape 1, a hollow circle) through which other points can be seen. Figure \@ref(fig:FIG-SCATTER-BASIC-SHAPE-SIZE) in Recipe \@ref(RECIPE-SCATTER-BASIC-SCATTER) demonstrates both of these solutions.

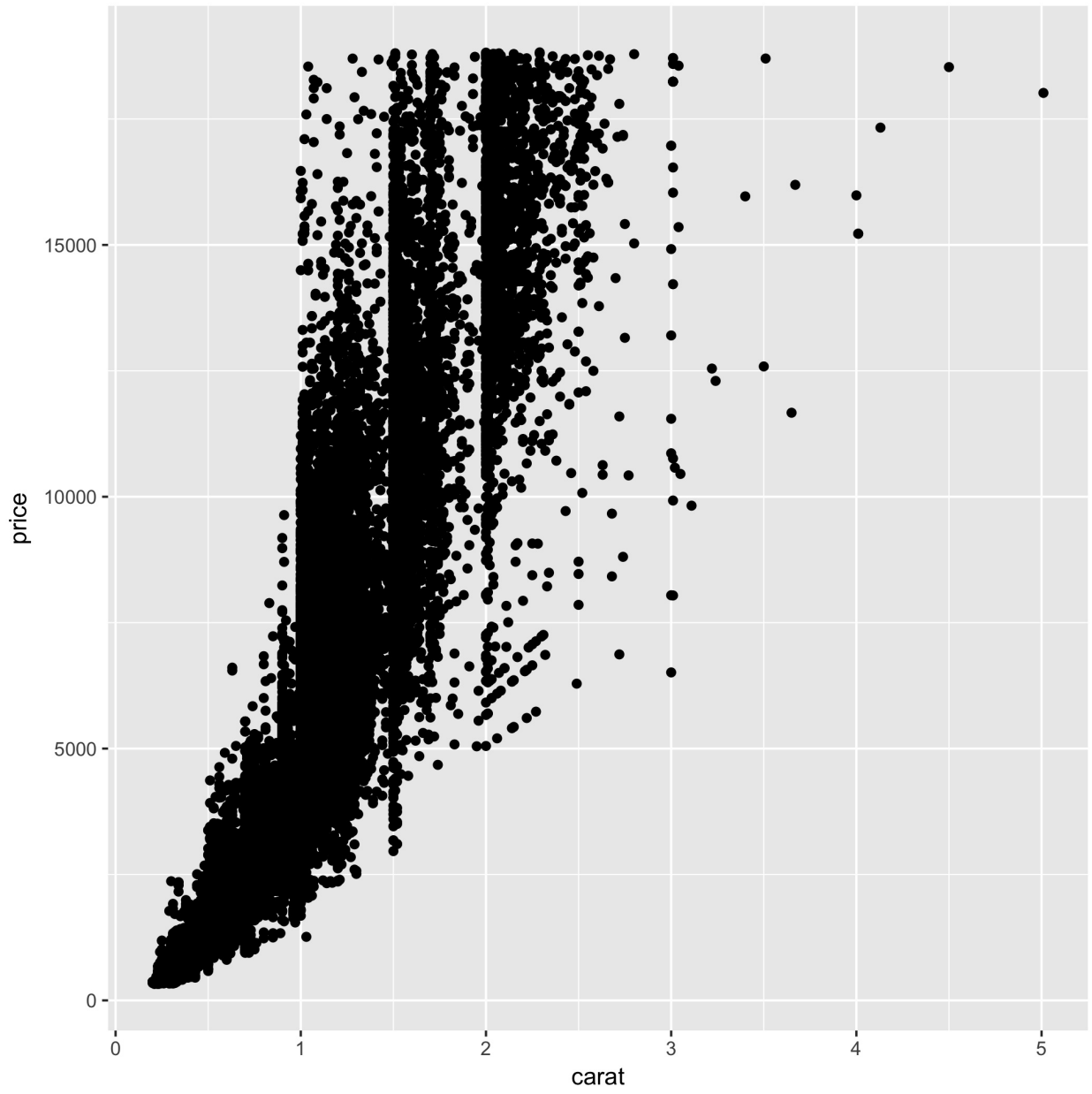
If there's a high degree of overplotting, there are a number of possible solutions:

- Make the points semi-transparent
- Bin the data into rectangles (better for quantitative analysis)
- Bin the data into hexagons
- Use box plots

Discussion

The scatter plot in Figure \@ref(fig:FIG-SCATTER-OVERPLOT) contains about 54,000 points. They are heavily overplotted, making it impossible to get a sense of the relative density of points in different areas of the graph:

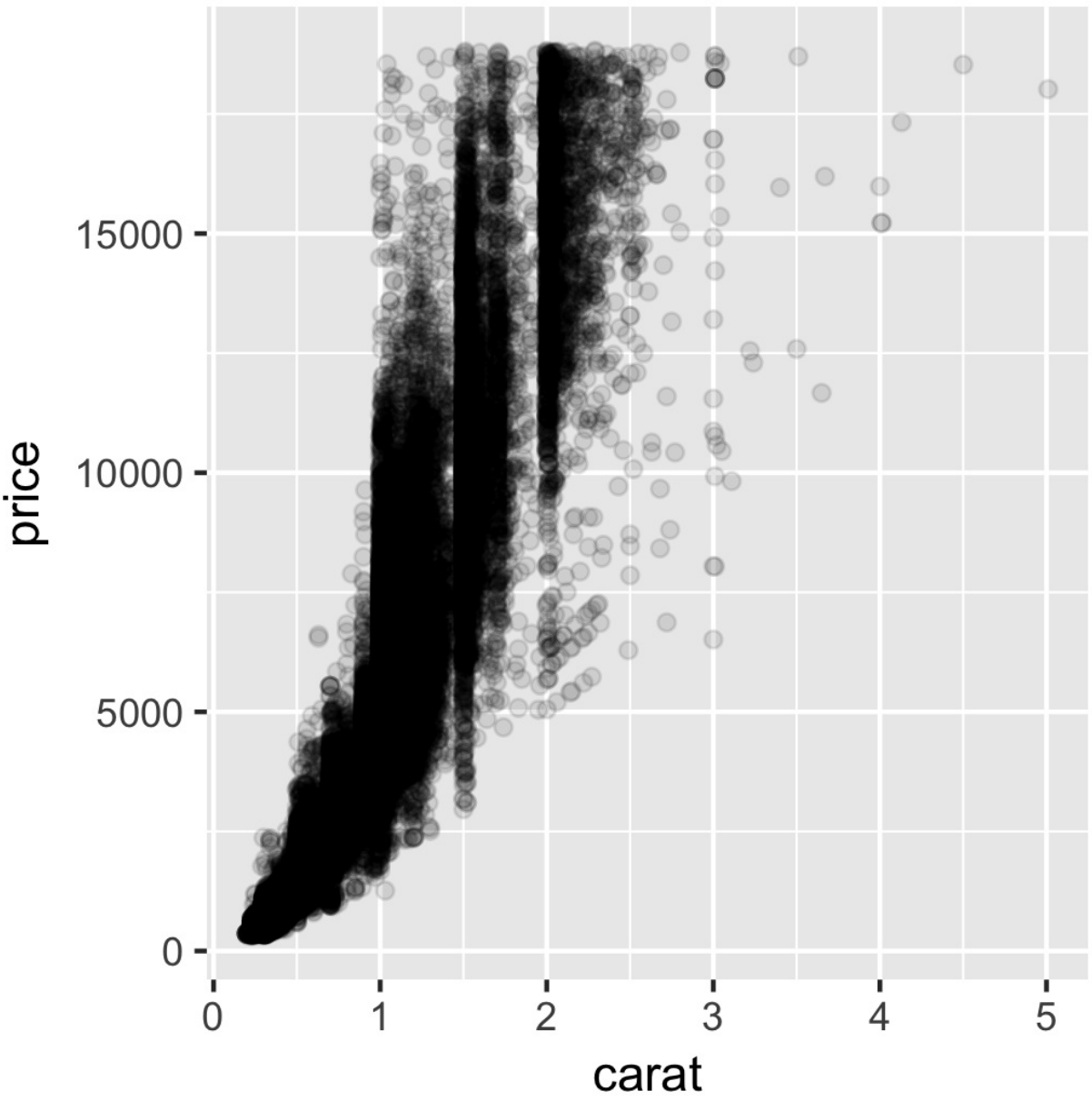
```
sp <- ggplot(diamonds, aes(x = carat, y = price))

sp + geom_point()
```

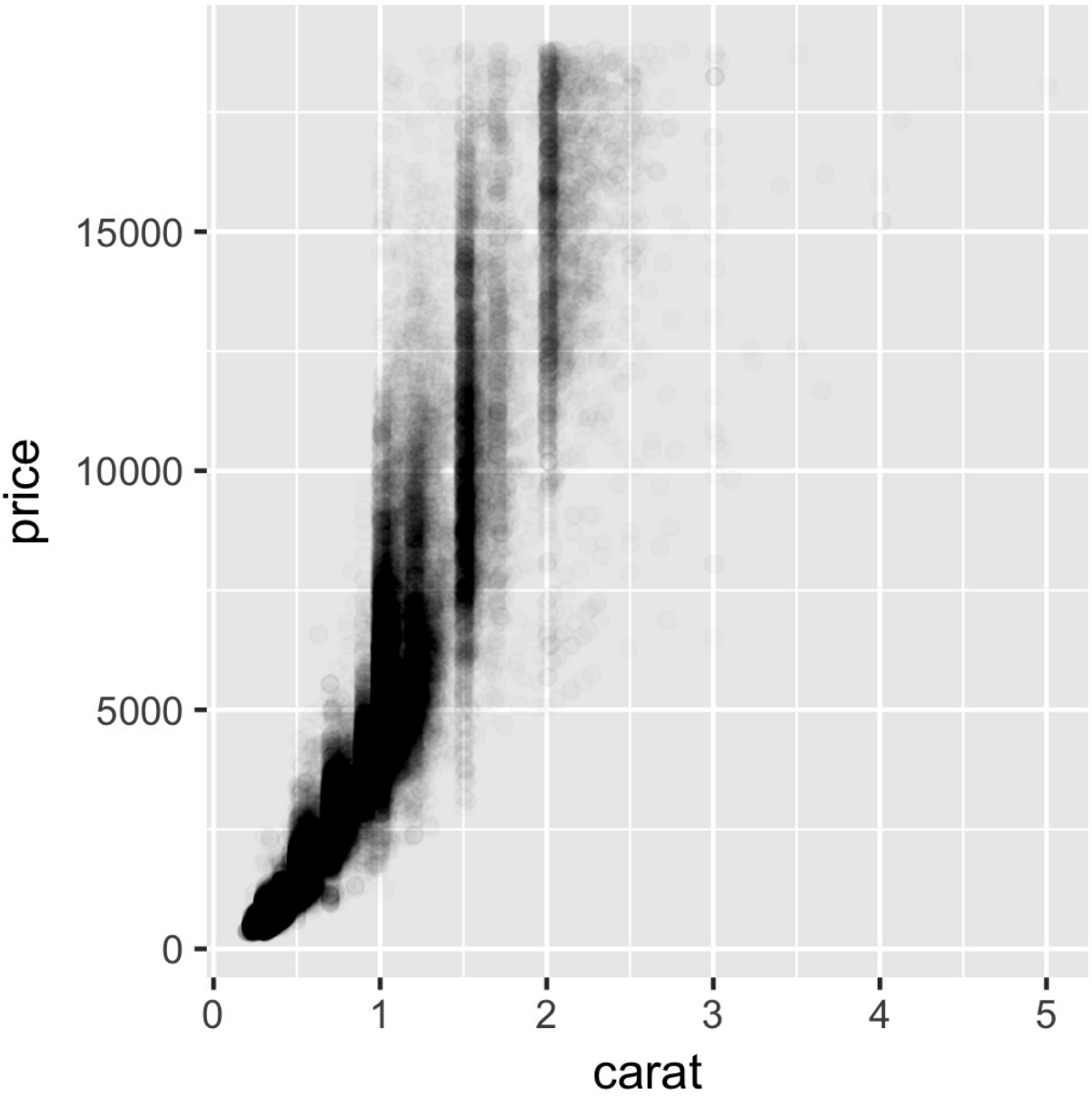


We can make the points semitransparent using alpha, as in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-ALPHA). Here, we'll make them 90% transparent and then 99% transparent, by setting `alpha=.1` and `alpha=.01`:

```
sp + geom_point(alpha = .1)
```

```
sp + geom_point(alpha = .01)
```


price
15000
10000
5000
0
0
1
2
3
4
5
carat



Now we can see that there are vertical bands at nice round values of carats, indicating that diamonds tend to be cut to those sizes. Still, the data is so dense that even when the points are 99% transparent, much of the graph appears solid black, and the data distribution is still somewhat obscured.

Note

For most graphs, vector formats (such as PDF, EPS, and SVG) result in smaller output files than bitmap formats (such as TIFF and PNG). But in cases where there are tens of thousands of points, vector output files can be very large and slow to render-the scatter plot here with 99% transparent points is 1.5 MB! In these cases, high-resolution bitmaps will be smaller and faster to display on computer screens. See Chapter \@ref(CHAPTER-OUTPUT) for more information.

Another solution is to *bin* the points into rectangles and map the density of the points to the fill color of the rectangles, as shown in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BIN2D).

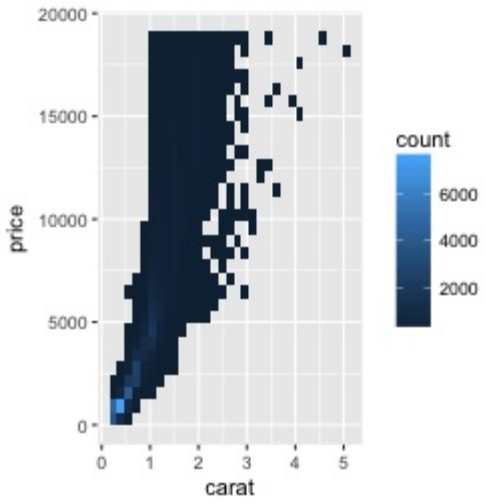
With the binned visualization, the vertical bands are barely visible. The density of points in the lower-left corner is much greater, which tells us that the vast majority of diamonds are small and inexpensive.

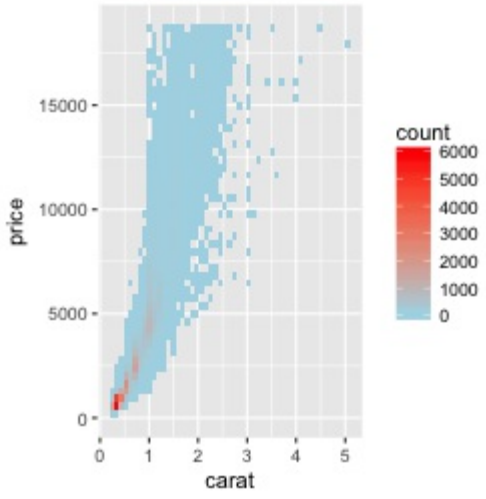
By default, `stat_bin_2d()` divides the space into 30 groups in the *x* and *y* directions, for a total of 900 bins. In the second version, we increase the number of bins with bins=50.

The default colors are somewhat difficult to distinguish because they don't vary much in luminosity. In the second version we set the colors by using `scale_fill_gradient()` and specifying the low and high colors. By default, the legend doesn't show an entry for the lowest values. This is because the range of the color scale starts not from zero, but from the smallest nonzero quantity in a bin-probably 1, in this case. To make the legend show a zero (as in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BIN2D), right), we can manually set the range from 0 to the maximum, 6000, using limits (Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BIN2D), left):

```
sp + stat_bin2d()
```

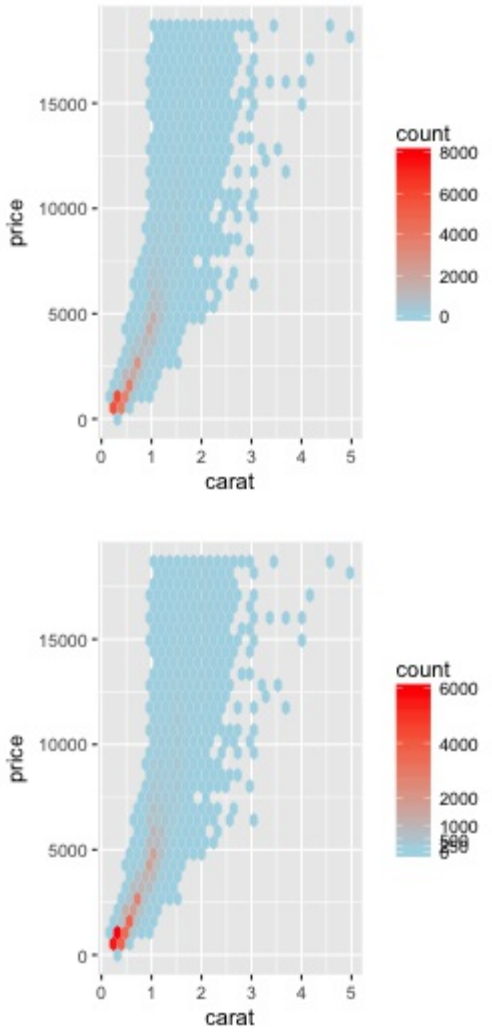
```
sp + stat_bin2d(bins = 50) +
    scale_fill_gradient(low = "lightblue", high = "red", limits = c(0, 6000))
```





Another alternative is to bin the data into hexagons instead of rectangles, with `stat_binhex()` (Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BINHEX)). It works just like `stat_bin2d()`. To use it, you must first install the hexbin package, with `install.packages("hexbin")`:

```
library(hexbin)

sp + stat_binhex() +
    scale_fill_gradient(low = "lightblue", high = "red",
                        limits = c(0, 8000))

sp + stat_binhex() +
    scale_fill_gradient(low = "lightblue", high = "red",
                        breaks = c(0, 250, 500, 1000, 2000, 4000, 6000),
                        limits = c(0, 6000))
```

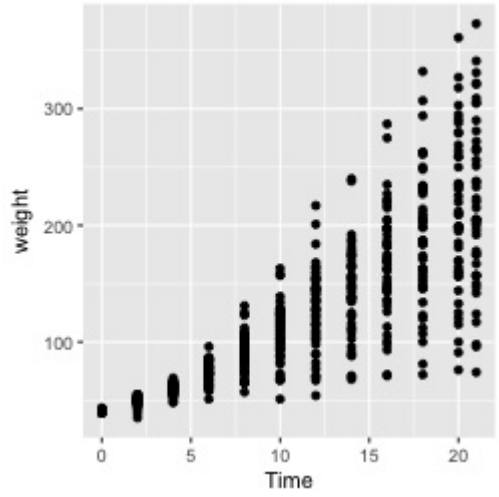


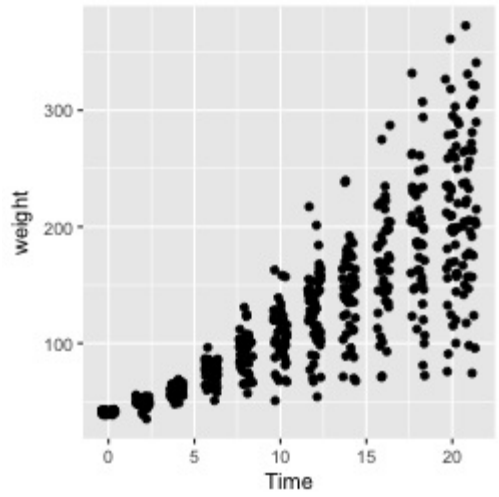
For both of these methods, if you manually specify the range, and there is a bin that falls outside that range because it has too many or too few points, that bin will show up as grey rather than the color at the high or low end of the range, as seen in the graph on the right in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BINHEX).

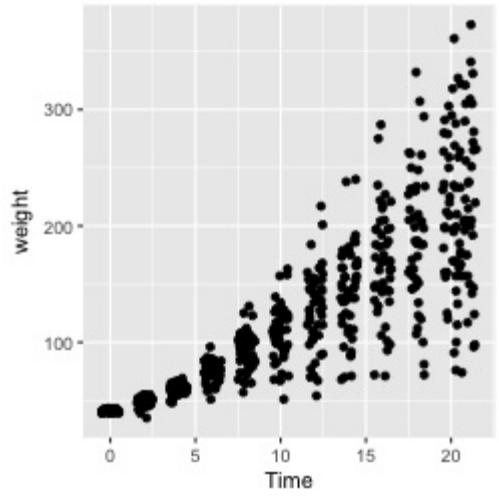
Overplotting can also occur when the data is *discrete* on one or both axes, as shown in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-JITTER). In these cases, you can randomly *jitter* the points with `position_jitter()`. By default the amount of jitter is 40% of the resolution of the data in each direction, but these amounts can be controlled with width and height:

```
sp1 <- ggplot(ChickWeight, aes(x = Time, y = weight))

sp1 + geom_point()

sp1 + geom_point(position = "jitter")
# Could also use geom_jitter(), which is equivalent
```

```
sp1 + geom_point(position = position_jitter(width = .5, height = 0))
```







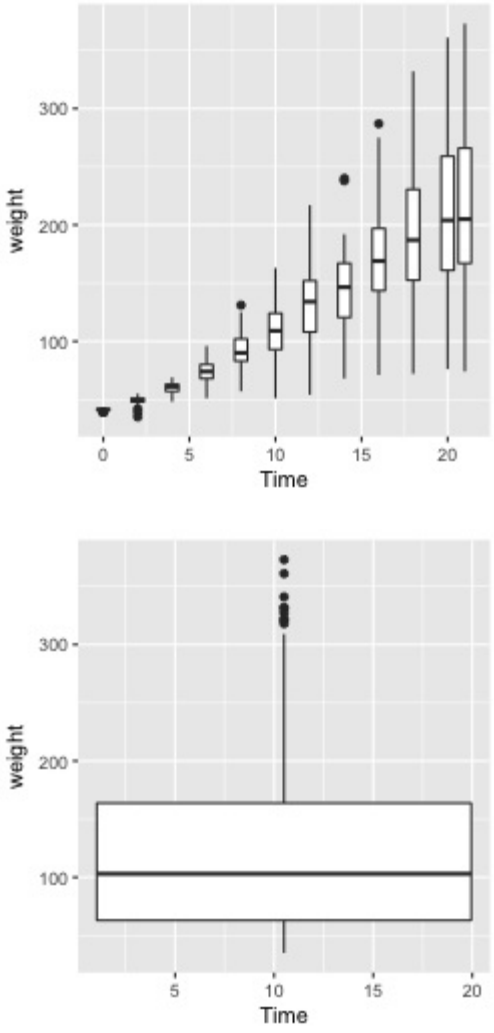
When the data has one discrete axis and one continuous axis, it might make sense to use box plots, as shown in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BOXPLOT). This will convey a different story than a standard scatter plot because it will obscure the *number* of data points at each location on the discrete axis. This may be problematic in some cases, but desirable in

others.

With the `ChickWeights` data, the x-axis is conceptually discrete, but since it is stored numerically, ggplot doesn't know how to group the data for each box. If you don't tell it how to group the data, you get a result like the graph on the right in Figure \@ref(fig:FIG-SCATTER-OVERPLOT-BOXPLOT). To tell it how to group the data, use `aes(group=...)`. In this case, we'll group by each distinct value of `Time`:

```
sp1 + geom_boxplot(aes(group = Time))
```

```
sp1 + geom_boxplot() # Without groups
#> Warning: Continuous x aesthetic -- did you forget aes(group=...)?
```



See Also

Instead of binning the data, it may be useful to display a 2D density estimate. To do this, see Recipe \@ref(RECIPE-DISTRIBUTION-DENSITY2D).

2.6 Adding Fitted Regression Model Lines

Problem

You want to add lines from a fitted regression model to a scatter plot.

Solution

To add a linear regression line to a scatter plot, add `stat_smooth()` and tell it to use `method=lm`. This instructs it to fit the data with the `lm()` (linear model) function. First we'll save the base plot object in `sp`, then we'll add different components to it:

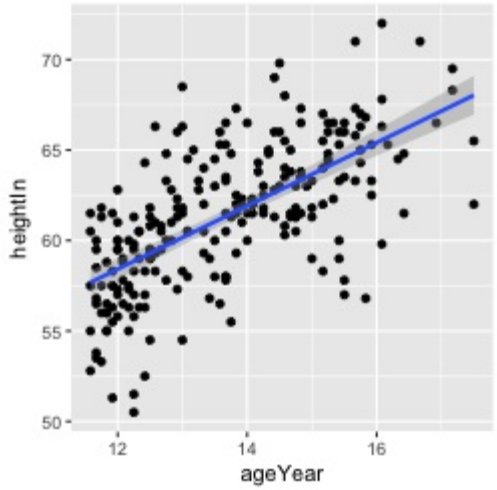
```
library(gcookbook) # For the data set

# The base plot
sp <- ggplot(heightweight, aes(x = ageYear, y = heightIn))

sp + geom_point() + stat_smooth(method = lm)
```

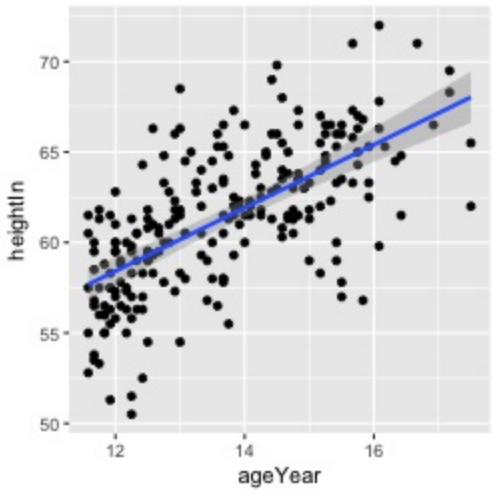
By default, `stat_smooth()` also adds a 95% confidence region for the regression fit. The confidence interval can be changed by setting `level`, or it can be disabled with `se=FALSE` (Figure \@ref(fig:FIG-SCATTER-FIT-LM)):

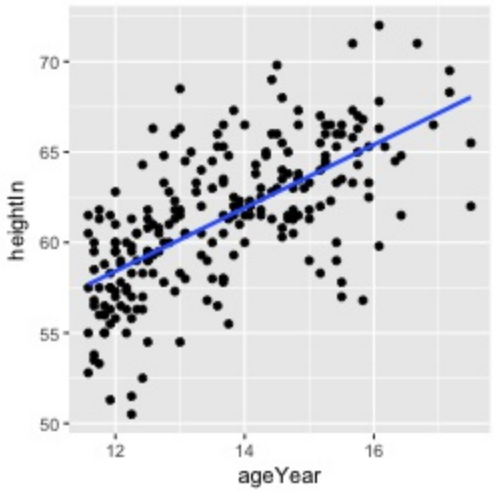
```
# 99% confidence region
sp + geom_point() + stat_smooth(method = lm, level = 0.99)

# No confidence region
sp + geom_point() + stat_smooth(method = lm, se = FALSE)
```

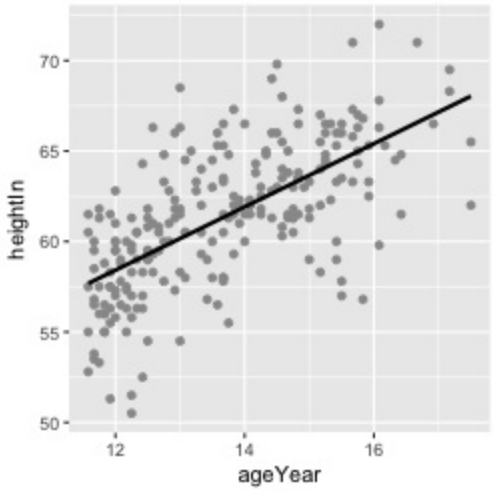
The default color of the fit line is blue. This can be change by setting `colour`. As with any other line, the attributes `linetype` and `size` can also be set. To emphasize the line, you can make the dots less prominent by setting `colour` (Figure \@ref(fig:FIG-SCATTER-FIT-LM), bottom right):

```
sp + geom_point(colour = "grey60") +
    stat_smooth(method = lm, se = FALSE, colour = "black")
```




heightIn
70
65
60
55
50
12
14
16
ageYear

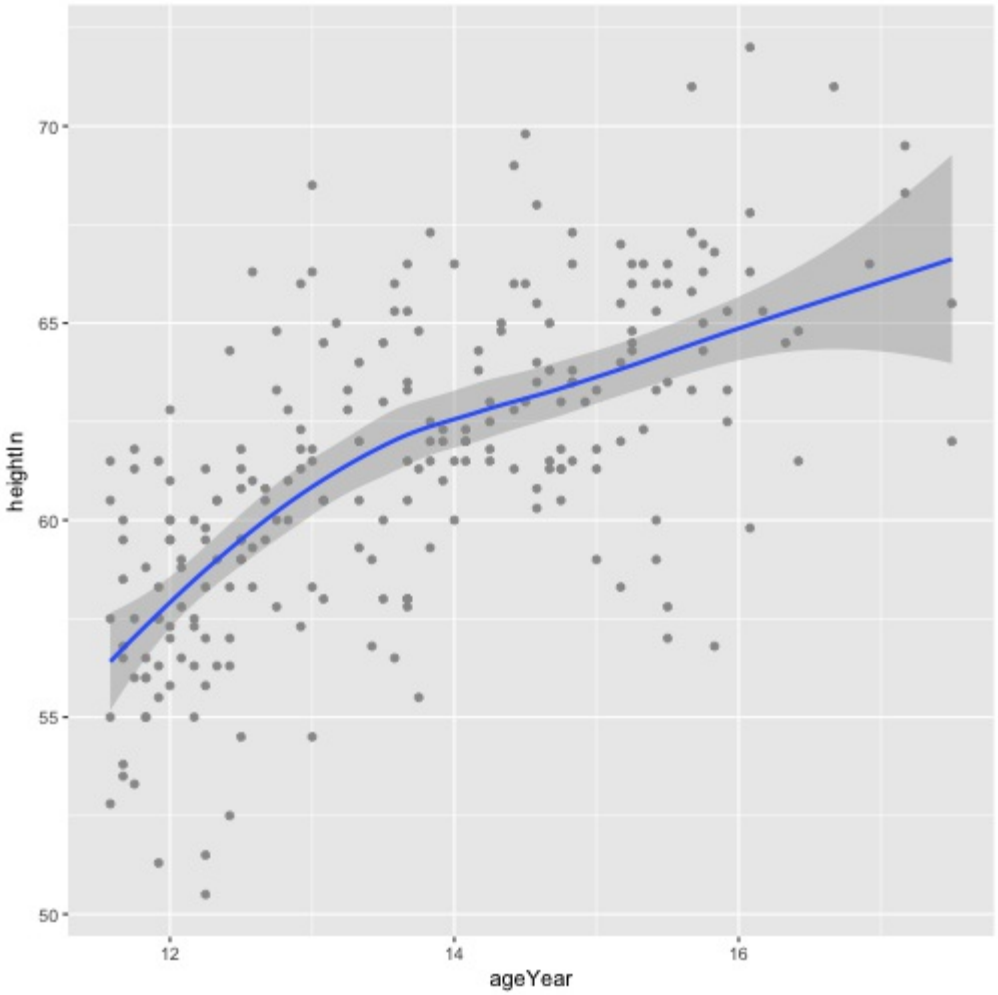

heightIn
70
65
60
55
50
12
14
16
ageYear


heightIn
70
65
60
55
50
12
14
16
ageYear

Discussion

The linear regression line is not the only way of fitting a model to the data--in fact, it's not even the default. If you add `stat_smooth()` without specifying the method, it will use a LOESS (locally weighted polynomial) curve, as shown in Figure \@ref(fig:FIG-SCATTER-FIT-LOESS):

```
sp + geom_point(colour = "grey60") + stat_smooth()
# Equivalent to:
sp + geom_point(colour = "grey60") + stat_smooth(method = loess)
```



It may be useful to send additional parameters to the modeling function, in this case `loess()`. If, for example, you wanted to use `loess(degree=1)`, you would call `stat_smooth(method=loess, method.args=list(degree=1))`. The same would be used for other modeling functions like `lm()` or `glm()`.

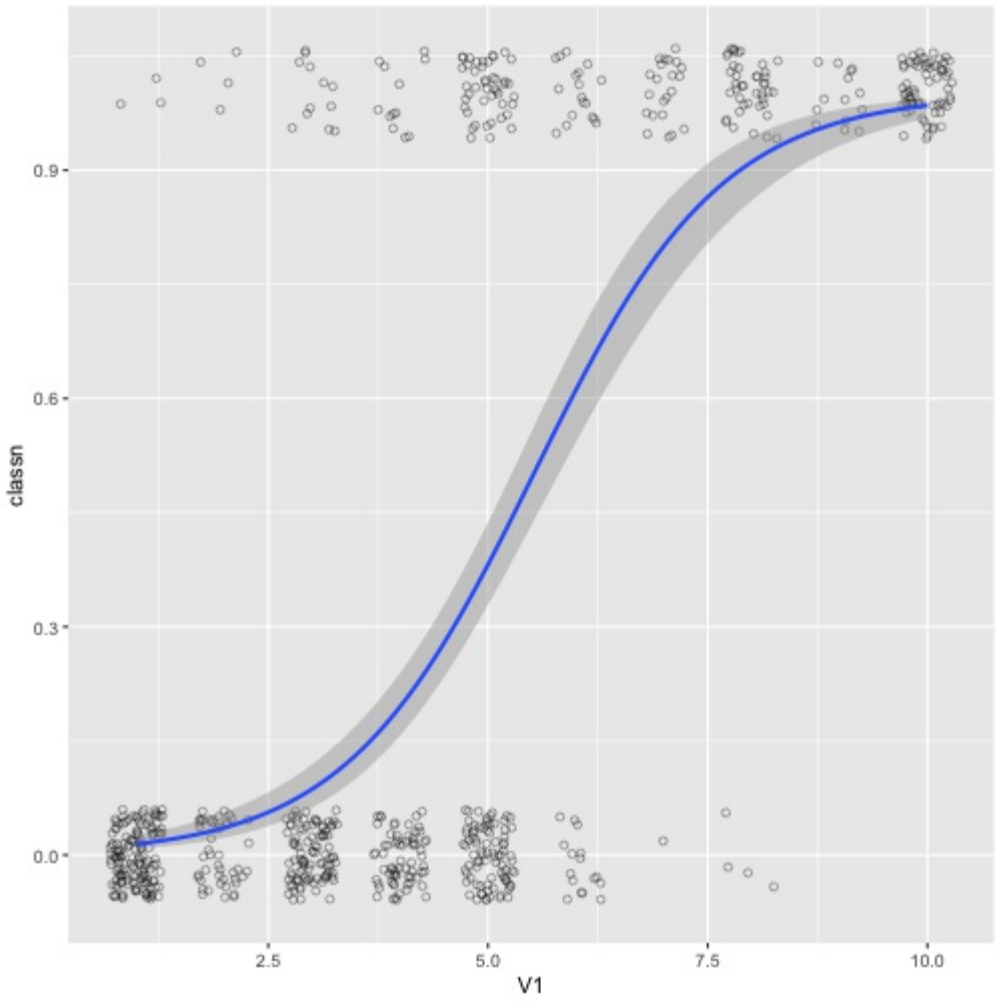
Another common type of model fit is a logistic regression. Logistic regression isn't appropriate for `heightweight`, but it's perfect for the `biopsy` data set in the MASS package. In the `biopsy` data, there are nine different measured attributes of breast cancer biopsies, as well as the class of the tumor, which is either benign or malignant. To prepare the data for logistic regression, we must convert the factor `class`, with the levels `benign` and `malignant`, to a vector with numeric values of 0 and 1. We'll make a copy of the `biopsy` data frame, then store the numeric coded `class` in a

column called `classn`:

```
library(MASS) # For the data set
b <- biopsy
b$classn[b$class=="benign"] <- 0
b$classn[b$class=="malignant"] <- 1
b
#>        ID V1 V2 V3 V4 V5 V6 V7 V8 V9     class classn
#> 1 1000025  5  1  1  1  2  1  3  1  1    benign      0
#> 2 1002945  5  4  4  5  7 10  3  2  1    benign      0
#> 3 1015425  3  1  1  1  2  2  3  1  1    benign      0
#> 4 1016277  6  8  8  1  3  4  3  7  1    benign      0
#> 5 1017023  4  1  1  3  2  1  3  1  1    benign      0
#> 6 1017122  8 10 10  8  7 10  9  7  1 malignant      1
#> 7 1018099  1  1  1  1  2 10  3  1  1    benign      0
#> 8 1018561  2  1  2  1  2  1  3  1  1    benign      0
#> # ... with 691 more rows
```

Although there are many attributes we could examine, for this example we'll just look at the relationship of `V1` (clump thickness) and the `class` of the tumor. Because there is a large degree of overplotting, we'll jitter the points and make them semitransparent (`alpha=0.4`), hollow (`shape=21`), and slightly smaller (`size=1.5`). Then we'll add a fitted logistic regression line (Figure \@ref(fig:FIG-SCATTER-FIT-LOGISTIC)) by telling `stat_smooth()` to use the `glm()` function with `family=binomial`:

```
ggplot(b, aes(x = V1, y = classn)) +
    geom_point(position = position_jitter(width = 0.3, height = 0.06), alpha = 0.4,
               shape = 21, size = 1.5) +
    stat_smooth(method = glm, method.args = list(family = binomial))
```



If your scatter plot has points grouped by a factor, using `colour` or `shape`, one fit line will be drawn for each group. First we'll make the base plot object `sps`, then we'll add the LOESS lines to it. We'll also make the points less prominent by making them semitransparent, using `alpha=.4` (Figure \@ref(fig:FIG-SCATTER-FIT-GROUP)):

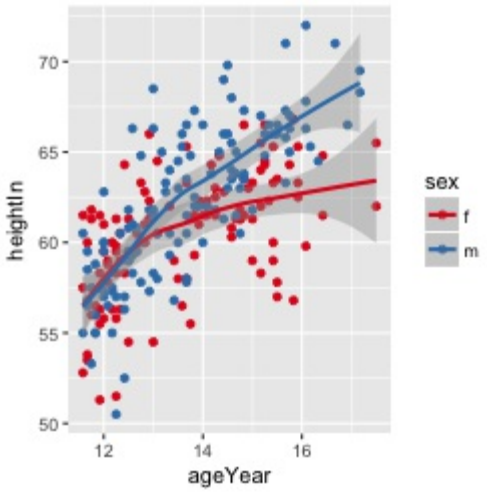
```
sps <- ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = sex)) +
    geom_point() +
    scale_colour_brewer(palette = "Set1")

sps + geom_smooth()
```

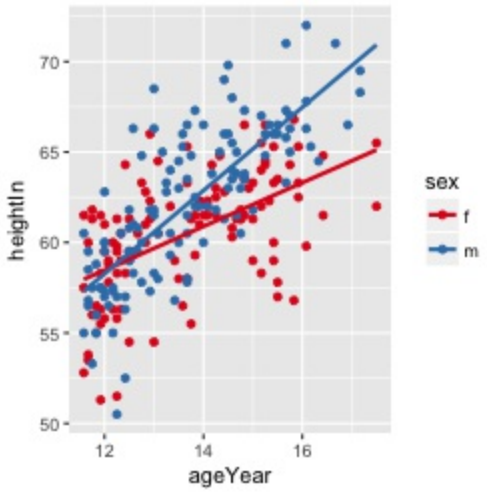
Notice that the blue line, for males, doesn't run all the way to the right side of the graph. There are two reasons for this. The first is that, by default, `stat_smooth()` limits the prediction to within the range of the predictor data (on the x-axis). The second is that even if it extrapolates, the `loess()` function only offers prediction within the *x* range of the data.

If you want the lines to extrapolate from the data, as shown in the right-hand image of Figure \@ref(fig:FIG-SCATTER-FIT-GROUP), you must use a model method that allows extrapolation, like `lm()`, and pass `stat_smooth()` the option `fullrange=TRUE`:

```
sps + geom_smooth(method = lm, se = FALSE, fullrange = TRUE)
```

```
sps <- ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = sex)) +
    geom_point() +
    scale_colour_brewer(palette = "Set1")

sps + geom_smooth()
#> `geom_smooth()` using method = 'loess'

sps + geom_smooth(method = lm, se = FALSE, fullrange = TRUE)
```





In this example with the `heightweight` data set, the default settings for `stat_smooth()` (with `loess` and no extrapolation) may make more sense than the extrapolated linear predictions, because we don’t grow linearly and we don’t grow forever.

TODO: Explain `geom_smooth` vs. `stat_smooth`.

2.7 Adding Fitted Lines from an Existing Model

Problem

You have already created a fitted regression model object for a data set, and you want to plot the lines for that model.

Solution

Usually the easiest way to overlay a fitted model is to simply ask `stat_smooth()` to do it for you, as described in Recipe \@ref(RECIPE-SCATTER-FITLINES). Sometimes, however, you may want to create the model yourself and then add it to your graph. This allows you to be sure that the model you're using for other calculations is the same one that you see.

In this example, we'll build a quadratic model using `lm()` with `ageYear` as a predictor of heightIn. Then we'll use the `predict()` function and find the predicted values of heightIn across the range of values for the predictor, `ageYear`:

```
library(gcookbook) # For the data set
model <- lm(heightIn ~ ageYear + I(ageYear^2), heightweight)
model
#>
#> Call:
#> lm(formula = heightIn ~ ageYear + I(ageYear^2), data = heightweight)
#>
#> Coefficients:
#>  (Intercept)       ageYear  I(ageYear^2)
#>     -10.3136        8.6673       -0.2478

# Create a data frame with ageYear column, interpolating across range
xmin <- min(heightweight$ageYear)
xmax <- max(heightweight$ageYear)
predicted <- data.frame(ageYear = seq(xmin, xmax, length.out = 100))
# Calculate predicted values of heightIn
predicted$heightIn <- predict(model, predicted)
predicted
#>    ageYear heightIn
#> 1 11.58000 56.82624
#> 2 11.63980 57.00047
#> 3 11.69960 57.17294
#> 4 11.75939 57.34363
#> 5 11.81919 57.51255
#> 6 11.87899 57.67969
#> 7 11.93879 57.84507
#> 8 11.99859 58.00867
#> # ... with 92 more rows
```

We can now plot the data points along with the values predicted from the model (as you'll see in Figure \@ref(fig:FIG-SCATTER-FIT-MODEL)):

```
sp <- ggplot(heightweight, aes(x = ageYear, y = heightIn)) +
      geom_point(colour = "grey40")

sp + geom_line(data = predicted, size = 1)
```

Discussion

Any model object can be used, so long as it has a corresponding `predict()` method. For example, `lm` has `predict.lm()`, loess has `predict.loess()`, and so on. Adding lines from a model can be simplified by using the function `predictvals()`, defined below If you simply pass in a model, it will do the work of finding the variable names and range of the predictor, and will return a data frame with predictor and predicted values. That data frame can then be passed to `geom_line()` to draw the fitted line, as we did earlier:

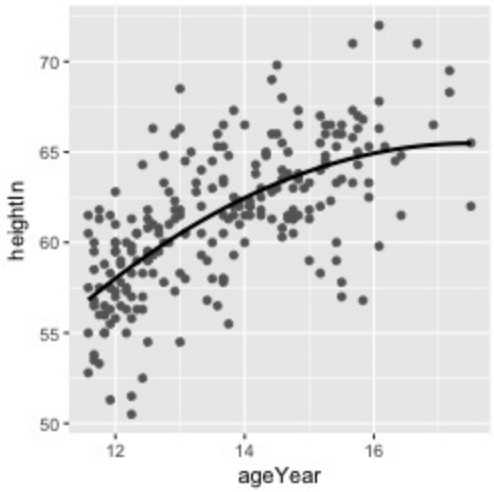
```
# Given a model, predict values of yvar from xvar
# This supports one predictor and one predicted variable
# xrange: If NULL, determine the x range from the model object. If a vector with
#   two numbers, use those as the min and max of the prediction range.
# samples: Number of samples across the x range.
# ...: Further arguments to be passed to predict()
predictvals <- function(model, xvar, yvar, xrange = NULL, samples = 100, ...) {

  # If xrange isn't passed in, determine xrange from the models.
  # Different ways of extracting the x range, depending on model type
  if (is.null(xrange)) {
    if (any(class(model) %in% c("lm", "glm")))
      xrange <- range(model$model[[xvar]])
    else if (any(class(model) %in% "loess"))
      xrange <- range(model$x)
  }

  newdata <- data.frame(x = seq(xrange[1], xrange[2], length.out = samples))
  names(newdata) <- xvar
  newdata[[yvar]] <- predict(model, newdata = newdata, ...)
  newdata
}
```

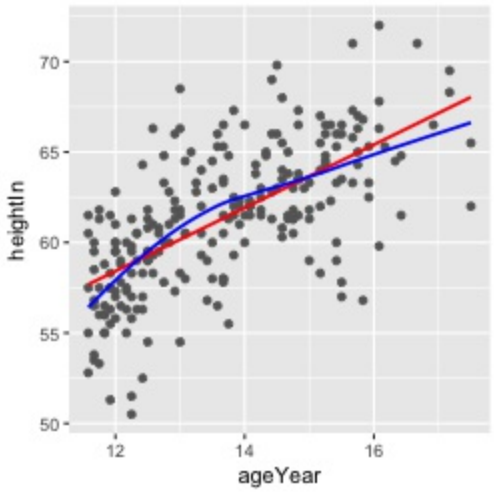
With the heightweight data set, we'll make a linear model with `lm()` and a LOESS model with `loess()` (Figure \@ref(fig:FIG-SCATTER-FIT-MODEL)):

```
modlinear <- lm(heightIn ~ ageYear, heightweight)

modloess <- loess(heightIn ~ ageYear, heightweight)
```

Then we can call `predictvals()` on each model, and pass the resulting data frames to `geom_line()`:

```
lm_predicted    <- predictvals(modlinear, "ageYear", "heightIn")
loess_predicted <- predictvals(modloess, "ageYear", "heightIn")

sp + geom_line(data = lm_predicted, colour = "red", size = .8) +
     geom_line(data = loess_predicted, colour = "blue", size = .8)

# From first block above
sp <- ggplot(heightweight, aes(x = ageYear, y = heightIn)) +
      geom_point(colour = "grey40")

sp + geom_line(data = predicted, size = 1)

# From second block above
lm_predicted    <- predictvals(modlinear, "ageYear", "heightIn")
loess_predicted <- predictvals(modloess, "ageYear", "heightIn")

sp + geom_line(data = lm_predicted, colour = "red", size = .8) +
     geom_line(data = loess_predicted, colour = "blue", size = .8)
```





For `glm` models that use a nonlinear link function, you need to specify `type="response"` to the `predictvals()` function. This is because the default behavior is to return predicted values in the scale of the linear predictors, instead of in the scale of the response (*y*) variable.

To illustrate this, we'll use the `biopsy` data set from the MASS package. As we did in Recipe \@ref(RECIPE-SCATTER-FITLINES), we'll use `V1` to predict `class`. Since logistic regression uses values from 0 to 1, while `class` is a factor, we'll first have to convert `class` to 0s and 1s:

```
library(MASS) # For the data set
b <- biopsy

b$classn[b$class=="benign"]    <- 0
b$classn[b$class=="malignant"] <- 1
```

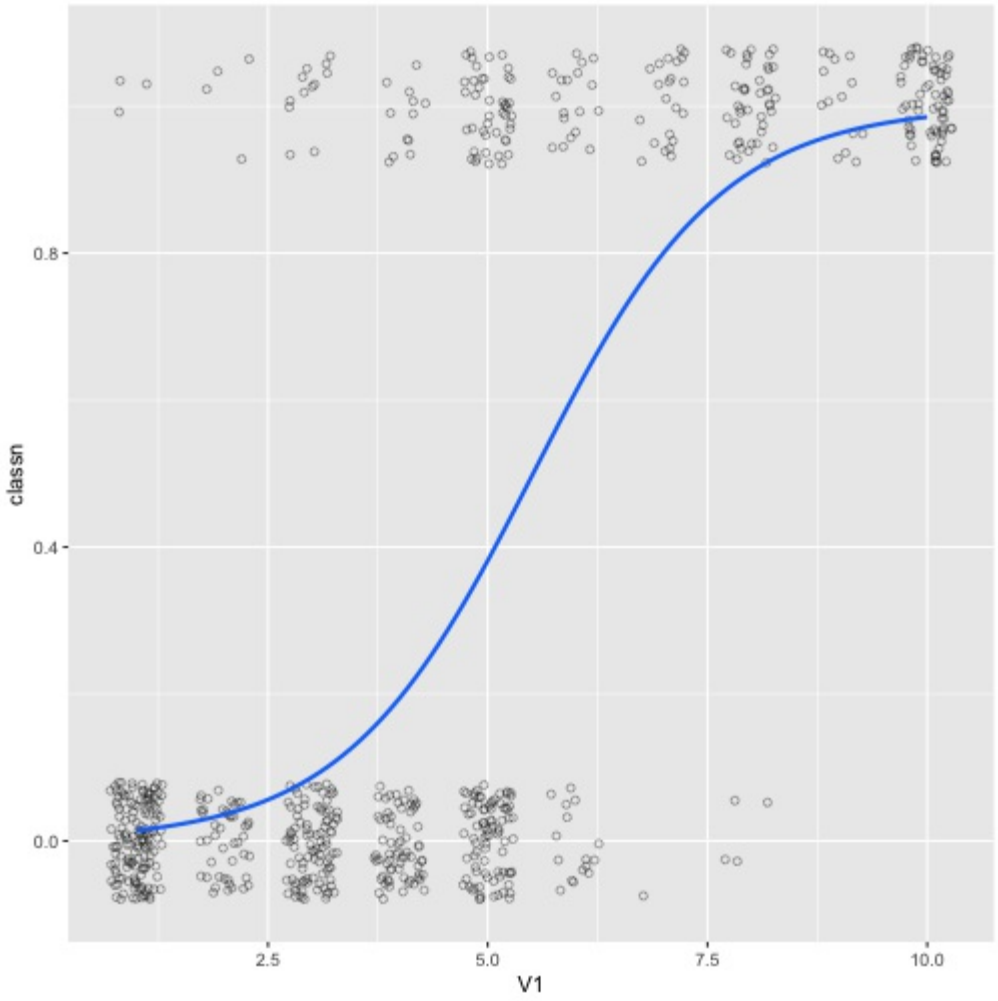
Next, we'll perform the logistic regression:

```
fitlogistic <- glm(classn ~ V1, b, family = binomial)
```

Finally, we'll make the graph with jittered points and the `fitlogistic` line. We'll make the line in a shade of blue by specifying a color in RGB values, and slightly thicker, with `size=1` (Figure \@ref(fig:FIG-SCATTER-FIT-MODEL-LOGISTIC)):

```
# Get predicted values
glm_predicted <- predictvals(fitlogistic, "V1", "classn", type = "response")
```

```
ggplot(b, aes(x = V1, y = classn)) +
    geom_point(position = position_jitter(width = .3, height = .08), alpha = 0.4,
               shape = 21, size = 1.5) +
    geom_line(data = glm_predicted, colour = "#1177FF", size = 1)
```



2.8 Adding Fitted Lines from Multiple Existing Models

Problem

You have already created a fitted regression model object for a data set, and you want to plot the lines for that model.

Solution

Use the `predictvals()` function from the previous recipe along with `dlply()` and `ldply()` from the plyr package.

With the `heightweight` data set, we'll make a linear model for each of the levels of `sex`, and put those model objects in a list. The model building is done with a function, `make_model()`, defined here. If you pass it a data frame, it simply returns an lm object. The model can be customized for your data.

```
make_model <- function(data) {
    lm(heightIn ~ ageYear, data)
}
```

With this function, we can use the `dlply()` function to build a model for each subset of data. This will split the data frame into subsets by the grouping variable sex, and apply `make_model()` to each subset. In this case, the `heightweight` data will be split into two data frames, one for males and one for females, and `make_model()` will be run on each subset. With `dlply()`, the models are put into a list and the list is returned:

```
library(gcookbook) # For the data set
library(dplyr)

# Create a lm for each value of sex; this returns a data frame
models <- heightweight %>%
    group_by(sex) %>%
    do(model = lm(heightIn ~ ageYear, .)) %>%
    ungroup()

# Print the data frame
models
#>   sex
#> 1   f
#> 2   m
#>
model
#> 1
43.96343, 1.2089, -2.073521, 2.717578, 3.923092, -1.186872, -0.7091224, 1.309741, -3.604672,
-1.76472, 1.921929, -4.271296, 3.537505, -0.3914211, 4.724154, -0.6779714, -7.073521,
0.2330547, 2.090878, 0.4253166, 2.527542, 1.203066, 3.631992, -4.571296, -2.489097, -0.397997,
0.6953278, 0.8174792, -0.789196, -3.096934, 0.3808152, -2.375746, 3.206453, -2.283584,
5.322029, -0.4624951, -4.901384, -2.313573, 1.315254, 0.005290943, 2.695328, -4.002447,
1.404228, -1.382521, 0.6119662, 1.015254, 2.225317, -0.1958712, -5.162495, 1.027542, 1.029767,
-0.4947091, 1.608579, 0.9864275, 1.324254, 0.4330547, -2.972458, -0.7669453, 1.717578,
1.508678, 1.900841, -2.873521, 2.100841, -2.604672, -2.56917, -0.4724583, 0.5263795,
-5.179134, -2.670233, 3.326379, 1.219804, 0.09087761, 3.699679, 1.928704, 1.63083, 1.296391,
-0.07352125, 4.608579, 4.810903, 0.7241544, 0.7275417, 0.1308296, -0.4947091, 0.3097411,
3.513029, -6.300321, 4.100841, 3.126479, 0.9997779, -5.701384, 3.197553, -0.3024472, -2.26472,
0.7141912, -5.085809, 1.526379, -4.574683, 6.417578, -3.880296, -2.168008, -7.272458,
-3.119185, 1.809741, 0.2086782, 1.697553, 2.529767, -3.602546, 3.131992, 2.810903, 0.7986157,
0.5152541, -1.289196, -0.7969341, -637.6823, 19.15641, 3.962784, -1.029333, -0.1118592,
1.613269, -3.095354, -1.886846, 2.063638, -4.421564, 3.371407, 0.01412235, 4.821891,
-0.696322, -7.179817, 0.1549016, 2.688141, 0.4210371, 2.47929, 1.638511, 3.495795, -4.721564,
-2.287585, -0.02059595, 1.204646, 1.047133, -0.427625, -2.661489, 1.255984, -2.438069,
3.495909, -2.111974, 5.303678, -0.6285931, -4.377995, -1.628365, 1.58888, 0.3967632, 3.204646,
-3.537101, 1.737657, -1.152867, 0.8715212, 1.28888, 2.221037, 0.297617, -5.328593, 0.9792898,
0.9375424, -0.1032368, 2.014122, 1.671635, 1.261931, 0.3549016, -3.02071, -0.8450984,
1.787173, 1.754162, 2.380258, -2.979817, 2.580258, -2.095354, -2.603351, -0.5207102,
0.5801436, -5.095468, -2.762458, 3.380144, 1.245425, 0.6881408, 4.281112, 1.778436, 1.596649,
1.863753, -0.1798167, 5.014122, 5.012415, 0.821891, 0.6792898, 0.09664893, -0.1032368,
0.6132685, 3.830628, -5.718888, 4.580258, 3.020183, 1.421151, -5.177995, 3.662899, 0.1628987,
-2.386846, 0.9297738, -4.870226, 1.580144, -4.578963, 6.487173, -3.694614, -2.304205,
-7.32071, -2.244016, 2.113269, 0.454162, 2.162899, 2.437542, -2.977141, 2.995795, 3.012415,
1.322005, 0.7888803, -0.927625, -0.3614895, 2, 58.37352, 59.58242, 59.37691, 60.18687,
63.20912, 61.19026, 62.60467, 58.26472, 60.07807, 58.0713, 57.9625, 61.89142, 59.77585,
```

```
58.97797, 58.37352, 58.56695, 63.20912, 59.07468, 58.77246, 62.09693, 58.16801, 58.0713,
60.4891, 61.698, 62.60467, 60.68252, 61.5892, 62.09693, 65.11918, 58.67575, 61.09355,
60.28358, 58.97797, 57.9625, 62.70138, 63.81357, 60.98475, 61.79471, 62.60467, 62.30245,
61.39577, 60.68252, 60.88803, 60.98475, 59.07468, 62.49587, 57.9625, 58.77246, 58.47023,
61.79471, 61.89142, 63.81357, 58.67575, 58.56695, 58.77246, 58.56695, 59.58242, 60.79132,
62.39916, 58.37352, 62.39916, 62.60467, 58.86917, 58.77246, 59.47362, 59.67913, 58.47023,
59.47362, 59.2802, 63.20912, 63.10032, 58.0713, 58.86917, 63.00361, 58.37352, 61.89142,
60.4891, 59.77585, 58.77246, 58.86917, 61.79471, 61.19026, 61.28697, 63.10032, 62.39916,
58.37352, 62.00022, 62.70138, 62.30245, 62.30245, 58.26472, 60.58581, 60.58581, 59.47362,
59.07468, 59.58242, 60.3803, 58.16801, 58.77246, 65.11918, 61.19026, 60.79132, 62.30245,
58.47023, 63.40255, 58.16801, 60.4891, 62.70138, 60.98475, 61.5892, 62.09693, 0, 1, -10.53565,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158,
0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, 0.0949158, -144.3451, 15.84614,
0.05025019, 0.007968611, -0.1497985, -0.04441007, -0.1182451, 0.1083085, 0.01364823,
0.1184056, 0.1240852, -0.08101203, 0.02942494, 0.07107545, 0.1026289, 0.09253177, -0.1497985,
0.0660269, 0.08180361, -0.09174019, 0.113357, 0.1184056, -0.007808099, -0.07091494,
-0.1182451, -0.01790519, -0.06523532, -0.09174019, -0.2495073, 0.08685216, -0.03936152,
0.002920064, 0.07107545, 0.1240852, -0.1232936, -0.1813519, -0.0336819, -0.07596348,
-0.1182451, -0.1024684, -0.05513823, -0.01790519, -0.02863336, -0.0336819, 0.0660269,
-0.1125655, 0.1240852, 0.08180361, 0.09758032, -0.07596348, -0.08101203, -0.1813519,
0.08685216, 0.09253177, 0.08180361, 0.09253177, 0.03952203, -0.02358481, -0.1075169,
0.1026289, -0.1075169, -0.1182451, 0.07675506, 0.08180361, 0.04520165, 0.03447348, 0.09758032,
0.04520165, 0.05529874, -0.1497985, -0.1441189, 0.1184056, 0.07675506, -0.1390703, 0.1026289,
-0.08101203, -0.007808099, 0.02942494, 0.08180361, 0.07675506, -0.07596348, -0.04441007,
-0.04945861, -0.1441189, -0.1075169, 0.1026289, -0.08669165, -0.1232936, -0.1024684,
-0.1024684, 0.1083085, -0.01285665, -0.01285665, 0.04520165, 0.0660269, 0.03952203,
-0.002128483, 0.113357, 0.08180361, -0.2495073, -0.04441007, -0.02358481, -0.1024684,
0.09758032, -0.1598956, 0.113357, -0.007808099, -0.1232936, -0.0336819, -0.06523532,
-0.09174019, 1.094916, 1.039522, 1, 2, 1e-07, 2, 109, lm(formula = heightIn ~ ageYear, data =
.), heightIn ~ ageYear, 56.3, 62.3, 63.3, 59, 62.5, 62.5, 59, 56.5, 62, 53.8, 61.5, 61.5,
64.5, 58.3, 51.3, 58.8, 65.3, 59.5, 61.3, 63.3, 61.8, 53.5, 58, 61.3, 63.3, 61.5, 60.8, 59,
65.5, 56.3, 64.3, 58, 64.3, 57.5, 57.8, 61.5, 62.3, 61.8, 65.3, 58.3, 62.8, 59.3, 61.5, 62,
61.3, 62.3, 52.8, 59.8, 59.5, 61.3, 63.5, 64.8, 60, 59, 55.8, 57.8, 61.3, 62.3, 64.3, 55.5,
64.5, 60, 56.3, 58.3, 60, 54.5, 55.8, 62.8, 60.5, 63.3, 66.8, 60, 60.5, 64.3, 58.3, 66.5,
65.3, 60.5, 59.5, 59, 61.3, 61.5, 64.8, 56.8, 66.5, 61.5, 63, 57, 65.5, 62, 56, 61.3, 55.5,
61, 54.5, 66, 56.5, 56, 51.5, 62, 63, 61, 64, 61, 59.8, 61.3, 63.3, 63.5, 61.5, 60.3, 61.3,
11.92, 12.92, 12.75, 13.42, 15.92, 14.25, 15.42, 11.83, 13.33, 11.67, 11.58, 14.83, 13.08,
12.42, 11.92, 12.08, 15.92, 12.5, 12.25, 15, 11.75, 11.67, 13.67, 14.67, 15.42, 13.83, 14.58,
15, 17.5, 12.17, 14.17, 13.5, 12.42, 11.58, 15.5, 16.42, 14.08, 14.75, 15.42, 15.17, 14.42,
13.83, 14, 14.08, 12.5, 15.33, 11.58, 12.25, 12, 14.75, 14.83, 16.42, 12.17, 12.08, 12.25,
12.08, 12.92, 13.92, 15.25, 11.92, 15.25, 15.42, 12.33, 12.25, 12.83, 13, 12, 12.83, 12.67,
15.92, 15.83, 11.67, 12.33, 15.75, 11.92, 14.83, 13.67, 13.08, 12.25, 12.33, 14.75, 14.25,
14.33, 15.83, 15.25, 11.92, 14.92, 15.5, 15.17, 15.17, 11.83, 13.75, 13.75, 12.83, 12.5,
12.92, 13.58, 11.75, 12.25, 17.5, 14.25, 13.92, 15.17, 12, 16.08, 11.75, 13.67, 15.5, 14.08,
14.58, 15
#> 2 30.65796, 2.300934, 2.5042, -0.2541742, -0.969166, 0.08036725, 1.380367, 3.197226,
0.1023333, -0.9472, -8.344399, -1.160325, -0.8294076, 1.2299, -2.537425, 6.696293, -4.393933,
-0.4196327, -4.311725, 0.005133869, -1.864992, -0.6126585, 0.5873415, 1.990142, -0.1383587,
-3.371967, -1.360325, 0.2528, -1.009858, -1.303707, 1.396293, 4.530834, -2.029408, 2.519192,
-1.356975, 2.779434, -1.611725, -0.3224334, -0.5850913, -0.2055745, 3.794425, -0.7055745,
5.162575, 0.5378082, 1.9785, 1.388275, 1.230834, 5.7299, -2.235558, 1.730834, -1.844399,
0.5864079, -1.055108, 1.369659, 0.08036725, 0.1430252, -2.754174, -2.871033, 0.9901421,
-2.2701, 0.9299005, 4.038742, 0.5687253, 7.9299, -1.269166, -2.912659, 3.628967, -6.735558,
-2.878007, 4.286408, -3.089759, 0.9219927, -0.5976667, -0.9135921, 1.985474, -0.3107914,
5.7785, 0.02012565, -1.953241, -0.5850913, 3.395359, 4.820126, 1.436875, 2.162575, 1.413975,
-1.720566, -1.596733, -4.096733, 1.294425, -0.4798744, -0.4196327, 2.380367, -3.413592,
4.095359, -1.646266, 1.279434, -0.1939326, -1.0215, -1.878007, 1.471526, -0.7098579, 2.670592,
1.730834, -0.6649924, 2.154667, -2.935558, 4.343025, -2.564059, 0.9892086, -3.660325,
-2.302774, 0.1775666, -4.736492, 4.805134, -3.732208, -4.111725, 1.654667, -0.9808079,
-2.194866, -3.086025, -1.280808, -2.878007, 4.388275, -1.897667, -0.611725, -0.6869584,
-0.3037074, -693.8519, 36.68783, -1.128675, -0.09305696, 1.206943, 3.049405, -0.1615379,
-1.197156, -8.510866, -1.324565, -1.025931, 1.042561, -2.764283, 6.520642, -4.546484,
-0.593057, -4.51771, -0.1752478, -2.168382, -0.8464734, 0.3535266, 1.839817, -0.393046,
-3.614965, -1.524565, 0.002843815, -1.160183, -1.479358, 1.220642, 4.371325, -2.225931,
```

2.280924, -1.63003, 2.57818, -1.81771, -0.579347, -0.7423742, -0.4368847, 3.563115,
-0.9368847, 4.935717, 0.3179082, 1.749416, 1.18229, 1.071325, 5.542561, -2.406756, 1.571325,
-2.010866, 0.3247632, -1.272503, 1.145306, -0.09305696, -0.1300298, -2.94374, -3.086202,
0.8398166, -2.457439, 0.7425614, 3.846672, 0.3165427, 7.742561, -1.428675, -3.146473,
3.413798, -6.906756, -3.032786, 4.024763, -3.386191, 0.7672145, -0.8615379, -1.175237, 1.696,
-0.4889467, 5.549416, -0.1903122, -2.114976, -0.7423742, 3.191879, 4.609688, 1.189145,
1.935717, 1.228862, -1.92182, -1.832775, -4.332775, 1.063115, -0.6903122, -0.593057, 2.206943,
-3.675237, 3.891879, -1.868393, 1.07818, -0.3464844, -1.250584, -2.032786, 1.302833,
-0.8601834, 2.474069, 1.571325, -0.9683818, 1.960371, -3.106756, 4.06997, -2.839618,
0.8110533, -3.824565, -2.450595, -0.07934702, -4.935519, 4.624752, -4.012221, -4.31771,
1.460371, -1.219076, -2.375248, -3.271138, -1.519076, -3.032786, 4.18229, -2.161538,
-0.8177101, -0.8999009, -0.479358, 2, 62.2958, 60.75417, 58.26917, 59.41963, 59.41963,
57.30277, 66.89767, 65.7472, 58.8444, 58.66032, 61.32941, 60.5701, 63.83743, 59.60371,
57.69393, 59.41963, 62.11172, 59.99487, 70.16499, 64.41266, 64.41266, 57.50986, 66.13836,
65.17197, 58.66032, 65.7472, 57.50986, 59.60371, 59.60371, 58.26917, 61.32941, 64.78081,
67.65697, 61.72057, 62.11172, 66.32243, 58.08509, 64.20557, 64.20557, 64.20557, 63.83743,
63.26219, 64.0215, 62.11172, 58.26917, 60.5701, 59.23556, 58.26917, 58.8444, 66.71359,
63.05511, 63.63034, 59.41963, 67.65697, 60.75417, 62.87103, 57.50986, 60.5701, 60.5701,
60.96126, 65.93127, 60.5701, 58.26917, 64.41266, 62.87103, 59.23556, 57.87801, 66.71359,
69.58976, 57.87801, 66.89767, 66.71359, 69.01453, 59.81079, 64.0215, 62.47987, 58.45324,
58.08509, 61.90464, 62.47987, 65.56313, 63.83743, 60.38602, 61.72057, 64.59673, 64.59673,
64.20557, 62.47987, 59.41963, 59.41963, 66.71359, 61.90464, 63.44627, 61.72057, 57.69393,
64.0215, 57.87801, 59.02847, 57.50986, 61.32941, 58.26917, 70.16499, 61.14533, 59.23556,
67.65697, 67.86406, 59.81079, 58.66032, 57.30277, 66.32243, 61.53649, 59.99487, 68.23221,
62.11172, 61.14533, 64.78081, 59.99487, 60.38602, 64.78081, 57.87801, 62.11172, 66.89767,
62.11172, 62.68696, 59.60371, 0, 1, -11.18034, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272, 0.08944272,
0.08944272, -152.5839, 15.94476, 0.1038544, 0.07249615, 0.07249615, 0.1301954, -0.1313326,
-0.09997432, 0.08817528, 0.0931926, 0.02044142, 0.04113788, -0.0479196, 0.06747882, 0.1195335,
0.07249615, -0.0008821985, 0.05681701, -0.2203901, -0.06359873, -0.06359873, 0.1245509,
-0.1106361, -0.08429519, 0.0931926, -0.09997432, 0.1245509, 0.06747882, 0.06747882, 0.1038544,
0.02044142, -0.07363338, -0.152029, 0.009779612, -0.0008821985, -0.1156535, 0.1088717,
-0.05795424, -0.05795424, -0.05795424, -0.0479196, -0.03224047, -0.05293692, -0.0008821985,
0.1038544, 0.04113788, 0.07751347, 0.1038544, 0.08817528, -0.1263153, -0.02659598,
-0.04227511, 0.07249615, -0.152029, 0.03612056, -0.02157865, 0.1245509, 0.04113788,
0.04113788, 0.03047607, -0.1049916, 0.04113788, 0.1038544, -0.06359873, -0.02157865,
0.07751347, 0.1145162, -0.1263153, -0.2047109, 0.1145162, -0.1313326, -0.1263153, -0.1890318,
0.06183434, -0.05293692, -0.01091684, 0.09883709, 0.1088717, 0.00476229, -0.01091684,
-0.094957, -0.0479196, 0.0461552, 0.009779612, -0.06861605, -0.06861605, -0.05795424,
-0.01091684, 0.07249615, 0.07249615, -0.1263153, 0.00476229, -0.03725779, 0.009779612,
0.1195335, -0.05293692, 0.1145162, 0.08315796, 0.1245509, 0.02044142, 0.1038544, -0.2203901,
0.02545875, 0.07751347, -0.152029, -0.1576735, 0.06183434, 0.0931926, 0.1301954, -0.1156535,
0.01479693, 0.05681701, -0.1677082, -0.0008821985, 0.02545875, -0.07363338, 0.05681701,
0.0461552, -0.07363338, 0.1145162, -0.0008821985, -0.1313326, -0.0008821985, -0.01656133,
0.06747882, 1.089443, 1.036121, 1, 2, 1e-07, 2, 123, lm(formula = heightIn ~ ageYear, data =
.), heightIn ~ ageYear, 64.8, 60.5, 57.3, 59.5, 60.8, 60.5, 67, 64.8, 50.5, 57.5, 60.5, 61.8,
61.3, 66.3, 53.3, 59, 57.8, 60, 68.3, 63.8, 65, 59.5, 66, 61.8, 57.3, 66, 56.5, 58.3, 61,
62.8, 59.3, 67.3, 66.3, 64.5, 60.5, 66, 57.5, 64, 68, 63.5, 69, 63.8, 66, 63.5, 59.5, 66.3,
57, 60, 57, 67.3, 62, 65, 59.5, 67.8, 58, 60, 58.5, 58.3, 61.5, 65, 66.5, 68.5, 57, 61.5,
66.5, 52.5, 55, 71, 66.5, 58.8, 66.3, 65.8, 71, 59.5, 69.8, 62.5, 56.5, 57.5, 65.3, 67.3, 67,
66, 61.8, 60, 63, 60.5, 65.5, 62, 59, 61.8, 63.3, 66, 61.8, 63, 57.5, 63, 56, 60.5, 56.8, 64,
60, 69.5, 63.3, 56.3, 72, 65.3, 60.8, 55, 55, 66.5, 56.8, 64.8, 64.5, 58, 62.8, 63.8, 57.8,
57.3, 63.5, 55, 66.5, 65, 61.5, 62, 59.3, 13.75, 13.08, 12, 12.5, 12.5, 11.58, 15.75, 15.25,
12.25, 12.17, 13.33, 13, 14.42, 12.58, 11.75, 12.5, 13.67, 12.75, 17.17, 14.67, 14.67, 11.67,
15.42, 15, 12.17, 15.25, 11.67, 12.58, 12.58, 12, 13.33, 14.83, 16.08, 13.5, 13.67, 15.5,
11.92, 14.58, 14.58, 14.58, 14.42, 14.17, 14.5, 13.67, 12, 13, 12.42, 12, 12.25, 15.67, 14.08,
14.33, 12.5, 16.08, 13.08, 14, 11.67, 13, 13, 13.17, 15.33, 13, 12, 14.67, 14, 12.42, 11.83,
15.67, 16.92, 11.83, 15.75, 15.67, 16.67, 12.67, 14.5, 13.83, 12.08, 11.92, 13.58, 13.83,
15.17, 14.42, 12.92, 13.5, 14.75, 14.75, 14.58, 13.83, 12.5, 12.5, 15.67, 13.58, 14.25, 13.5,
11.75, 14.5, 11.83, 12.33, 11.67, 13.33, 12, 17.17, 13.25, 12.42, 16.08, 16.17, 12.67, 12.17,

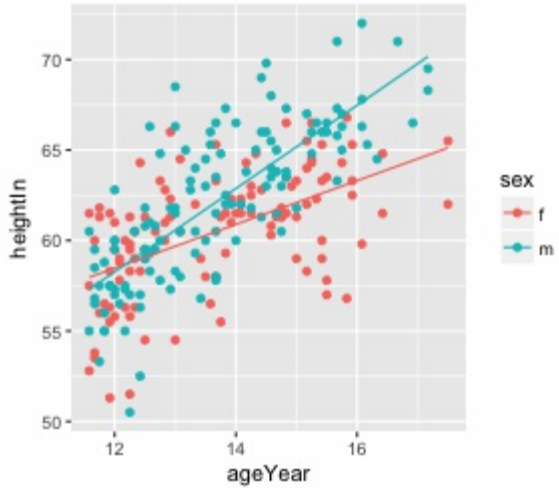
```
11.58, 15.5, 13.42, 12.75, 16.33, 13.67, 13.25, 14.83, 12.75, 12.92, 14.83, 11.83, 13.67,
15.75, 13.67, 13.92, 12.58

# Print out the model column of the data frame
models$model
#> [[1]]
#>
#> Call:
#> lm(formula = heightIn ~ ageYear, data = .)
#>
#> Coefficients:
#> (Intercept)      ageYear
#>      43.963        1.209
#>
#>
#> [[2]]
#>
#> Call:
#> lm(formula = heightIn ~ ageYear, data = .)
#>
#> Coefficients:
#> (Intercept)      ageYear
#>      30.658        2.301
```

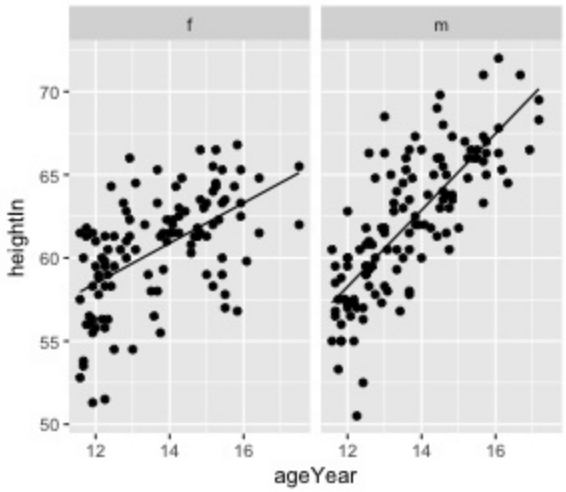
Now that we have the list of model objects, we can run `predictvals()` to get predicted values from each model, using the `ldply()` function:

```
predvals <- models %>%
    group_by(sex) %>%
    do(predictvals(.$model[[1]], xvar = "ageYear", yvar = "heightIn"))
```

Finally, we can plot the data with the predicted values (Figure \@ref(fig:FIG-SCATTER-FIT-MODEL-MULTI)):

```
ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = sex)) +
    geom_point() +
    geom_line(data = predvals)

ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = sex)) +
    geom_point() +
    geom_line(data = predvals)

# Using facets instead of colors for the groups
ggplot(heightweight, aes(x = ageYear, y = heightIn)) +
    geom_point() +
    geom_line(data = predvals) +
    facet_grid(. ~ sex)
```




f
m
70
65
60
55
50
heightIn
12
14
16
12
14
16
ageYear

Discussion

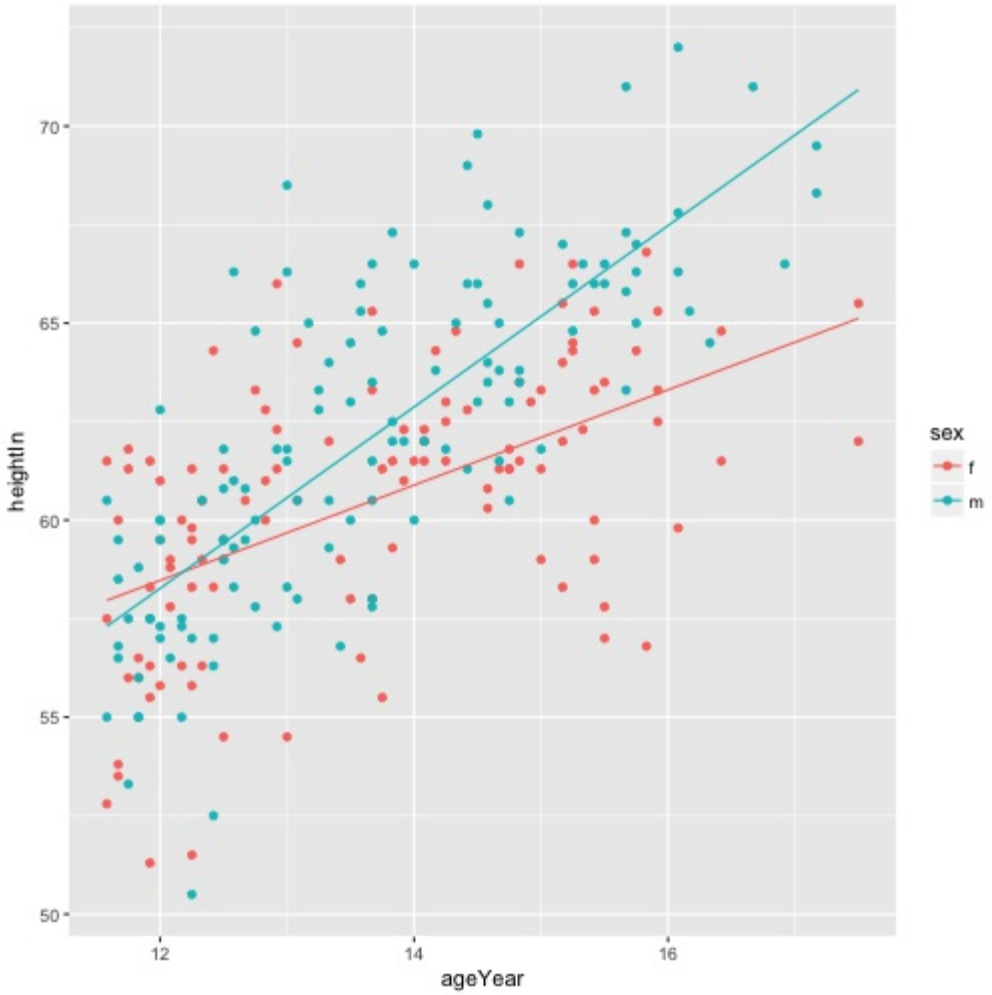
The `dlply()` and `ldply()` calls are used for splitting the data into parts, running functions on those parts, and then reassembling the output.

With the preceding code, the *x* range of the predicted values for each group spans the *x* range of each group, and no further; for the males, the prediction line stops at the oldest male, while for females, the prediction line continues further right, to the oldest female. To form prediction lines that have the same *x* range across all groups, we can simply pass in `xrange`, like this:

```
predvals <- models %>%
    group_by(sex) %>%
    do(predictvals(.$model[[1]], xvar = "ageYear", yvar = "heightIn",
                   xrange = range(heightweight$ageYear)))
```

Then we can plot it, the same as we did before:

```
ggplot(heightweight, aes(x = ageYear, y = heightIn, colour = sex)) +
    geom_point() +
    geom_line(data = predvals)
```



As you can see in Figure \@ref(fig:FIG-SCATTER-FIT-MODEL-MULTI-RANGE), the line for males now extends as far to the right as the line for females. Keep in mind that extrapolating past

the data isn’t always appropriate, though; whether or not it’s justified will depend on the nature of your data and the assumptions you bring to the table.

2.9 Adding Annotations with Model Coefficients

Problem

You want to add numerical information about a model to a plot.

Solution

To add simple text to a plot, simply add an annotation. In this example, we'll create a linear model and use the `predictvals()` function defined in Recipe \@ref(RECIPE-SCATTER-FITLINES-MODEL) to create a prediction line from the model. Then we'll add an annotation:

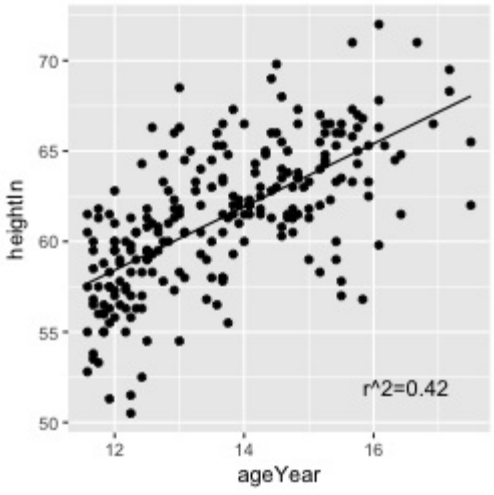
```
library(gcookbook) # For the data set
model <- lm(heightIn ~ ageYear, heightweight)
summary(model)
#>
#> Call:
#> lm(formula = heightIn ~ ageYear, data = heightweight)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -8.3517 -1.9006  0.1378  1.9071  8.3371
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  37.4356     1.8281   20.48   <2e-16 ***
#> ageYear       1.7483     0.1329   13.15   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 2.989 on 234 degrees of freedom
#> Multiple R-squared:  0.4249,    Adjusted R-squared:  0.4225
#> F-statistic: 172.9 on 1 and 234 DF,  p-value: < 2.2e-16
```

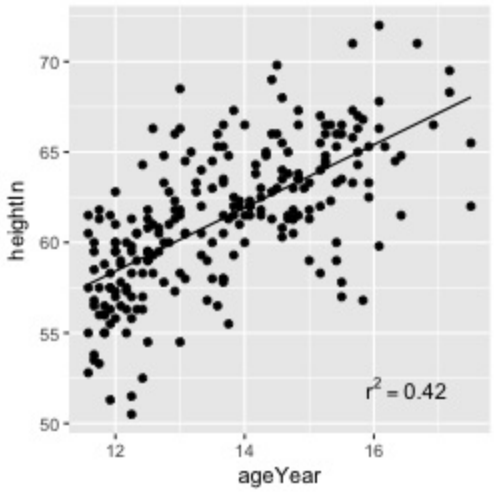
This shows that the *r^2* value is 0.4249. We'll create a graph and manually add the text using `annotate()` (Figure \@ref(fig:FIG-SCATTER-FIT-MODEL-TEXT)):

```
# First generate prediction data
pred <- predictvals(model, "ageYear", "heightIn")
sp <- ggplot(heightweight, aes(x = ageYear, y = heightIn)) + geom_point() +
    geom_line(data = pred)

sp + annotate("text", label = "r^2=0.42", x = 16.5, y = 52)
```

Instead of using a plain text string, it's also possible to enter formulas using R's math expression syntax, by using `parse=TRUE`:

```
sp + annotate("text", label = "r^2 == 0.42", parse = TRUE, x = 16.5, y = 52)
```




70
65
60
55
50
heightIn
12
14
16
ageYear
r2 = 0.42

Discussion

Text geoms in ggplot do not take expression objects directly; instead, they take character strings that can be turned into expressions with R's `parse()` function.

If you use a math expression, the syntax must be correct for it to be a valid R expression object. You can test validity by wrapping it in `expression()` and seeing if it throws an error (make sure *not* to use quotes around the expression). In the example here, `==` is a valid construct in an expression to express equality, but `=` is not:

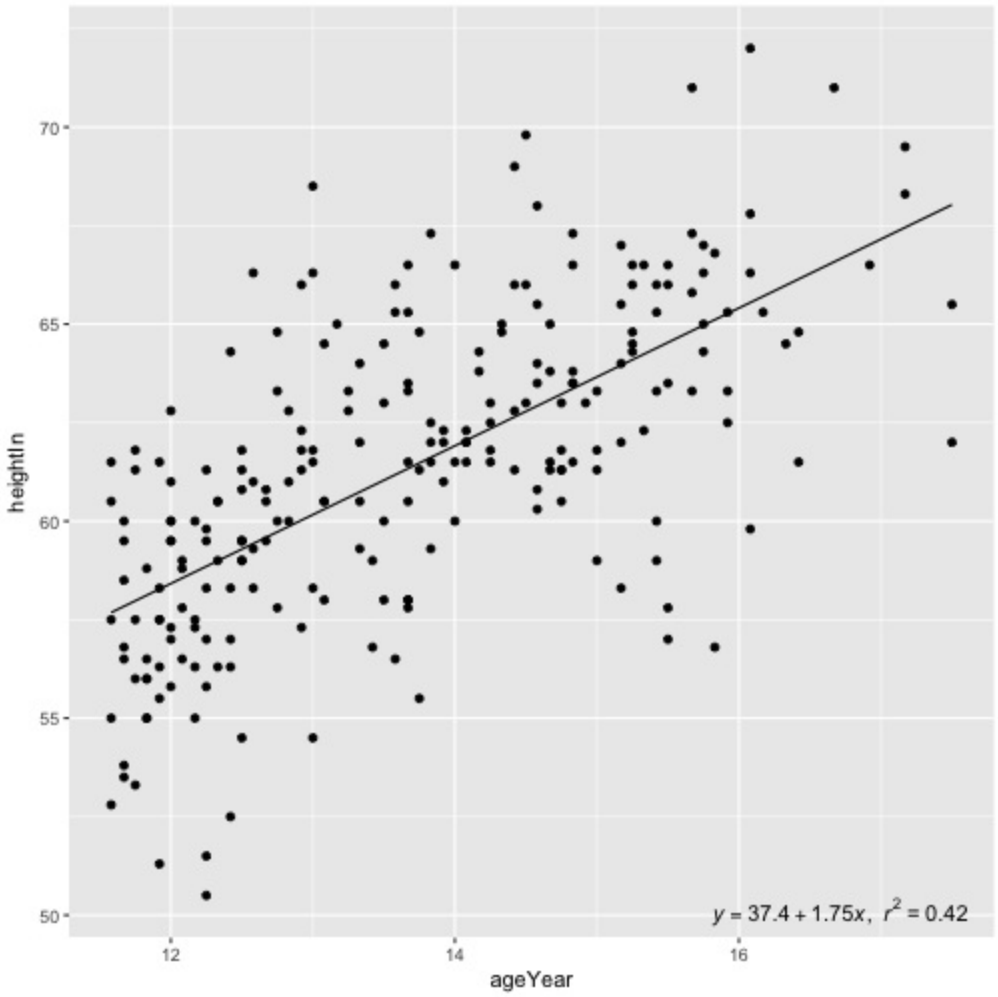
```
expression(r^2 == 0.42) # Valid
expression(r^2 = 0.42)  # Not valid
#> Error: unexpected '=' in "expression(r\^2 ="
```

It's possible to automatically extract values from the model object and build an expression using those values. In this example, we'll create a string, which, when parsed, yields a valid expression:

```
# Use sprintf() to construct our string.
# The %.3g and %.2g are replaced with numbers with 3 significant digits and 2
# significant digits, respectively. The numbers are supplied after the string.
eqn <- sprintf(
    "italic(y) == %.3g + %.3g * italic(x) * ',' ~~ italic(r)^2 ~ '=' ~ %.2g",
    coef(model)[1],
    coef(model)[2],
    summary(model)$r.squared
)

eqn
#> [1] "italic(y) == 37.4 + 1.75 * italic(x) * ',' ~~ italic(r)^2 ~ '=' ~ 0.42"

# Test validity by using parse()
parse(text = eqn)
#> expression(italic(y) == 37.4 + 1.75 * italic(x) * "," ~ ~italic(r)^2 ~
#>     "=" ~ 0.42)
```

Now that we have the expression string, we can add it to the plot. In this example we'll put the text in the bottom-right corner, by setting `x=Inf` and `y=-Inf` and using horizontal and vertical adjustments so that the text all fits inside the plotting area (Figure \@ref(fig:FIG-SCATTER-FIT-MODEL-TEXT-AUTO)):

```
sp + annotate("text", label = eqn, parse = TRUE, x = Inf, y = -Inf, hjust = 1.1, vjust = -.5)
```


70
65
60
55
50
heightIn
12
14
16
ageYear
$y = 37.4 + 1.75x,\ r^2 = 0.42$

See Also

The math expression syntax in R can be a bit tricky. See Recipe \@ref(RECIPE-ANNOTATE-TEXT-MATH) for more information.

2.10 Adding Marginal Rugs to a Scatter Plot

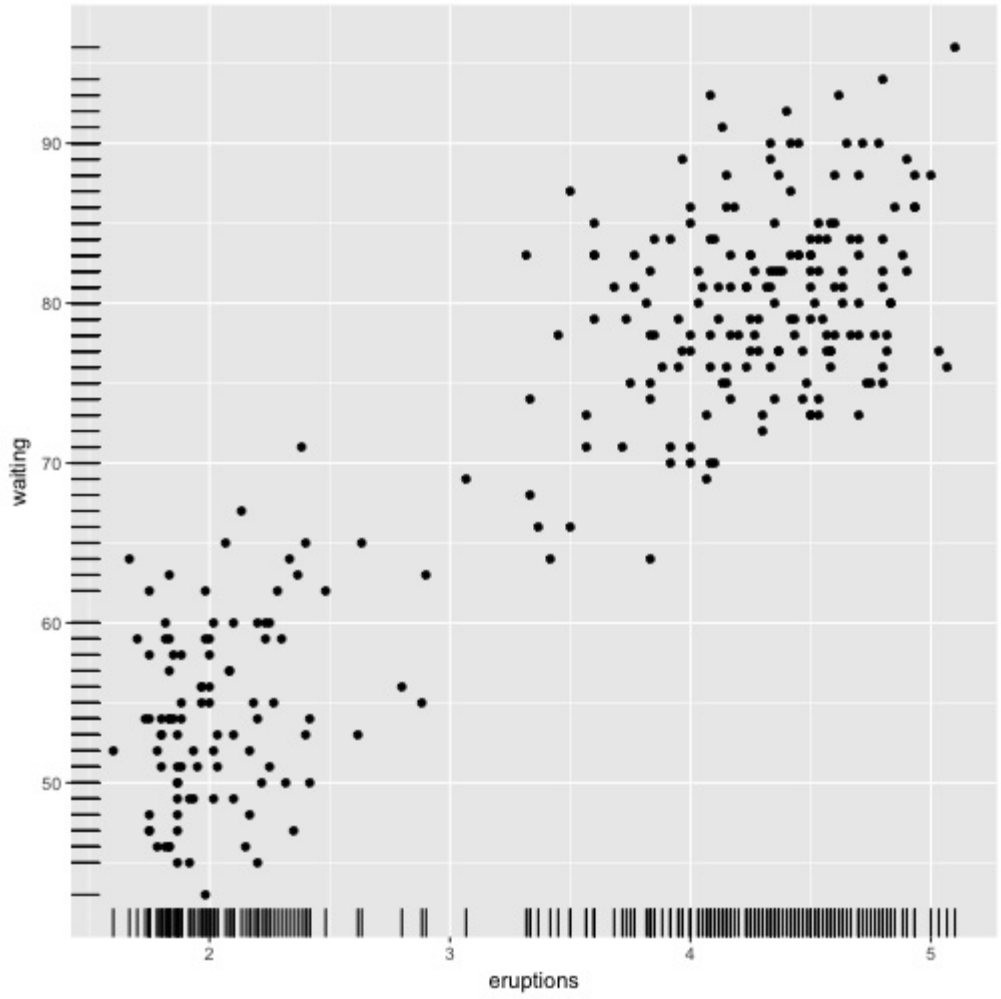
Problem

You want to add marginal rugs to a scatter plot.

Solution

Use `geom_rug()`. For this example (Figure \@ref(fig:FIG-SCATTER-RUG)), we'll use the `faithful` data set, which contains data about the Old Faithful geyser in two columns: `eruptions`, which is the length of each eruption, and `waiting`, which is the length of time to the next eruption:

```
ggplot(faithful, aes(x = eruptions, y = waiting)) +
    geom_point() +
    geom_rug()
```

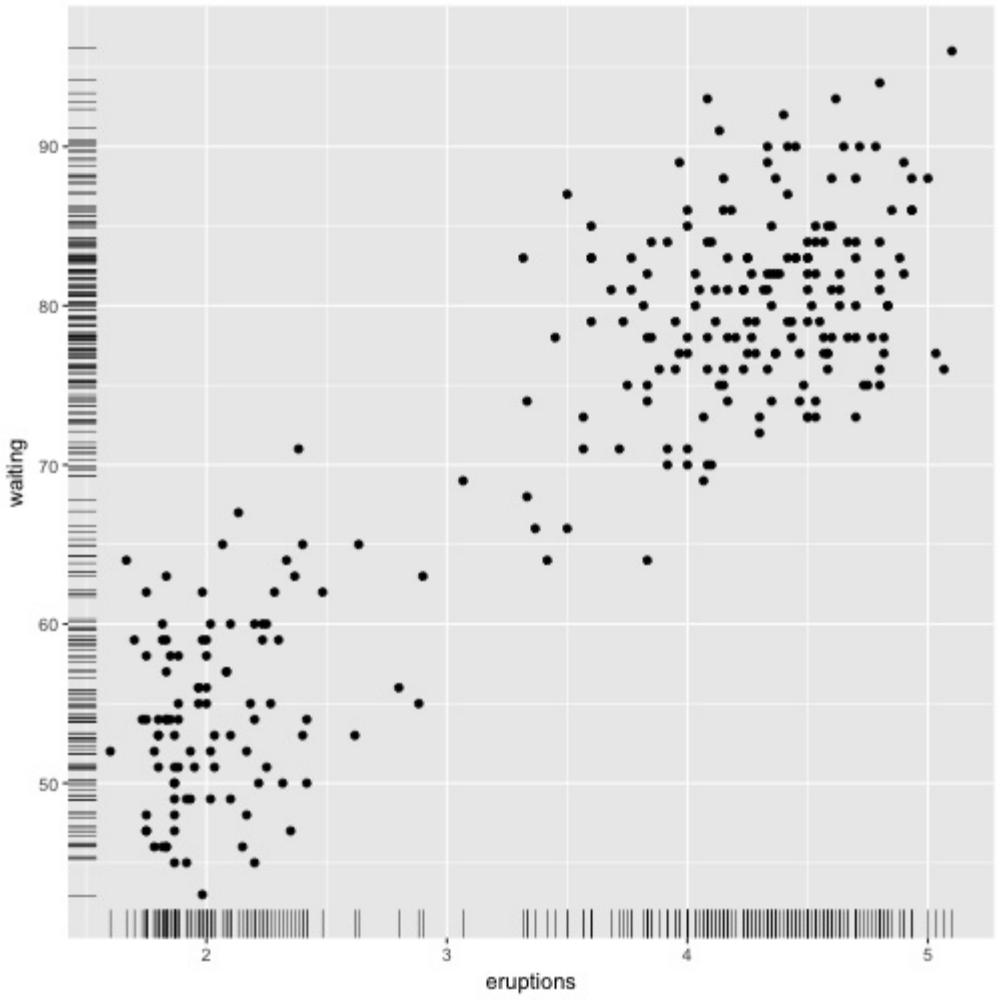


Discussion

A marginal rug plot is essentially a one-dimensional scatter plot that can be used to visualize the distribution of data on each axis.

In this particular data set, the marginal rug is not as informative as it could be. The resolution of the `waiting` variable is in whole minutes, and because of this, the rug lines have a lot of overplotting. To reduce the overplotting, we can jitter the line positions and make them slightly thinner by specifying size (Figure \@ref(fig:FIG-SCATTER-RUG-JITTER)). This helps the viewer see the distribution more clearly:

```
ggplot(faithful, aes(x = eruptions, y = waiting)) +
    geom_point() +
    geom_rug(position = "jitter", size = 0.2)
```



See Also

For more about overplotting, see Recipe \@ref(RECIPE-SCATTER-OVERPLOT).

2.11 Labeling Points in a Scatter Plot

Problem

You want to add labels to points in a scatter plot.

Solution

For annotating just one or a few points, you can use `annotate()` or `geom_text()`. For this example, we'll use the countries data set and visualize the relationship between health expenditures and infant mortality rate per 1,000 live births. To keep things manageable, we'll just take the subset of countries that spent more than $2,000 USD per capita:

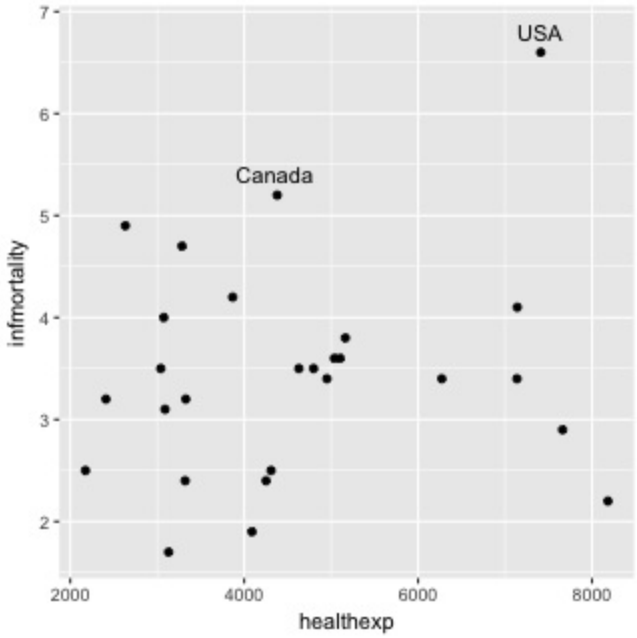
```
library(gcookbook) # For the data set
subset(countries, Year==2009 & healthexp>2000)
#>           Name Code Year      GDP laborrate healthexp infmortality
#> 254    Andorra  AND 2009       NA        NA  3089.636          3.1
#> 560  Australia  AUS 2009 42130.82      65.2  3867.429          4.2
#> 611    Austria  AUT 2009 45555.43      60.4  5037.311          3.6
#> 968    Belgium  BEL 2009 43640.20      53.5  5104.019          3.6
#> 1733    Canada  CAN 2009 39599.04      67.8  4379.761          5.2
#> 2702   Denmark  DNK 2009 55933.35      65.4  6272.729          3.4
#> 3365   Finland  FIN 2009 44576.73      60.9  4309.604          2.5
#> 3416    France  FRA 2009 40663.05      56.1  4797.966          3.5
#> # ... with 19 more rows
```

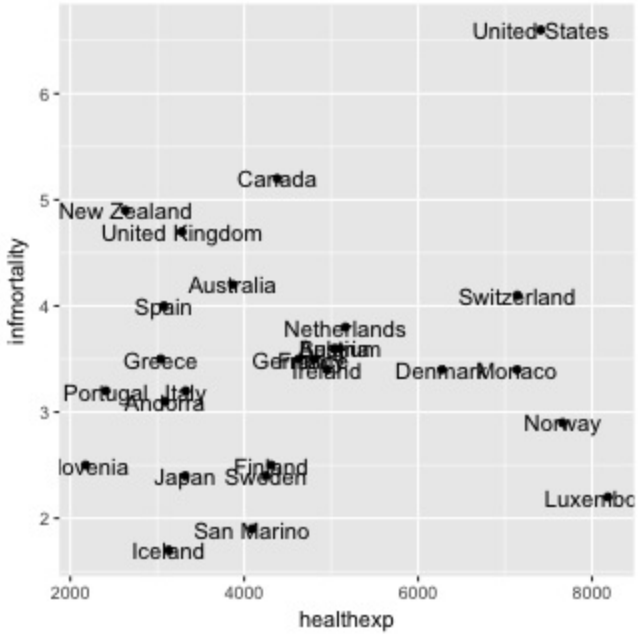
We'll save the basic scatter plot object in sp and add then add things to it. To manually add annotations, use `annotate()`, and specify the coordinates and label (Figure \@ref(fig:FIG-SCATTER-LABEL), left). It may require some trial-and-error tweaking to get them positioned just right:

```
sp <- ggplot(subset(countries, Year==2009 & healthexp>2000),
             aes(x = healthexp, y = infmortality)) +
      geom_point()

sp + annotate("text", x = 4350, y = 5.4, label = "Canada") +
     annotate("text", x = 7400, y = 6.8, label = "USA")
```

To automatically add the labels from your data (Figure \@ref(fig:FIG-SCATTER-LABEL), right), use `geom_text()` and map a column that is a factor or character vector to the label aesthetic. In this case, we'll use `Name`, and we'll make the font slightly smaller to reduce crowding. The default value for `size` is 5, which doesn't correspond directly to a point size:

```
sp + geom_text(aes(label = Name), size = 4)
```


USA
Canada
infmortality
healthexp
7
6
5
4
3
2
2000
4000
6000
8000

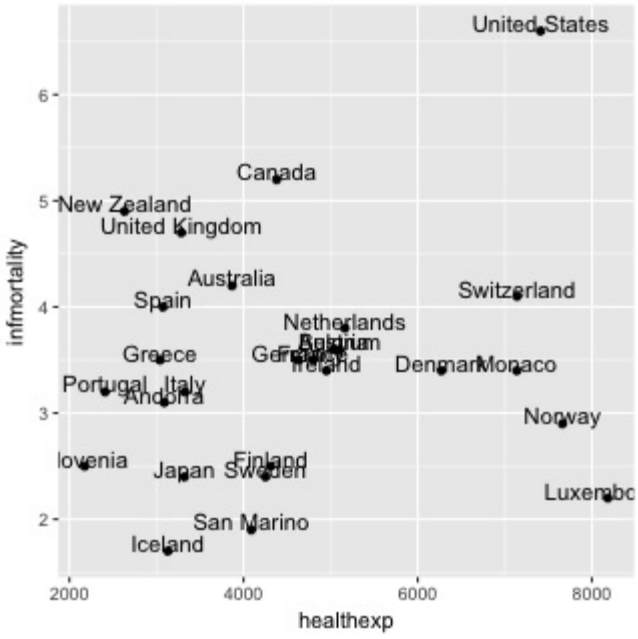

United States
Canada
New Zealand
United Kingdom
Australia
Switzerland
Spain
Netherlands
Greece
Ireland
Denmark
Monaco
Portugal
Andorra
Norway
Japan
Sweden
San Marino
Iceland
infmortality
healthexp
6
5
4
3
2
2000
4000
6000
8000

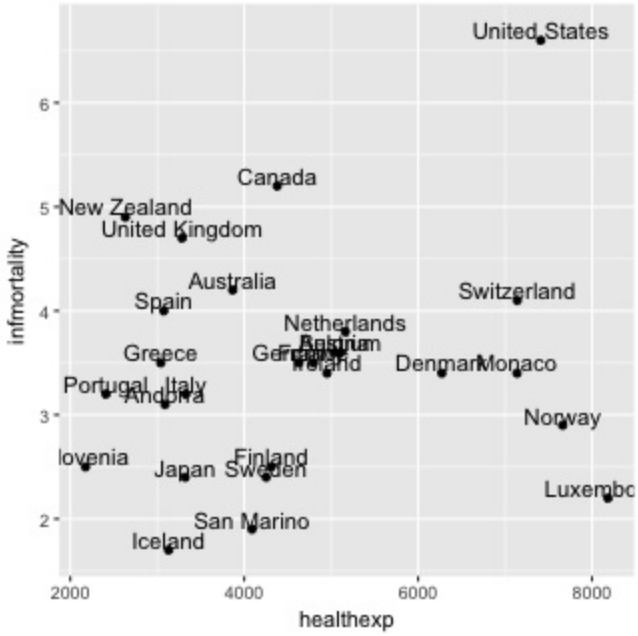
Discussion

The automatic method for placing annotations centers each annotation on the *x* and *y* coordinates. You'll probably want to shift the text vertically, horizontally, or both.

Setting `vjust=0` will make the baseline of the text on the same level as the point (Figure \@ref(fig:FIG-SCATTER-LABEL-VJUST), left), and setting vjust=1 will make the top of the text level with the point. This usually isn't enough, though -- you can increase or decrease `vjust` to shift the labels higher or lower, or you can add or subtract a bit to or from the y mapping to get the same effect (Figure \@ref(fig:FIG-SCATTER-LABEL-VJUST), right):

```
sp + geom_text(aes(label = Name), size = 4, vjust = 0)

# Add a little extra to y
sp + geom_text(aes(y = infmortality + .1, label = Name), size = 4)
```

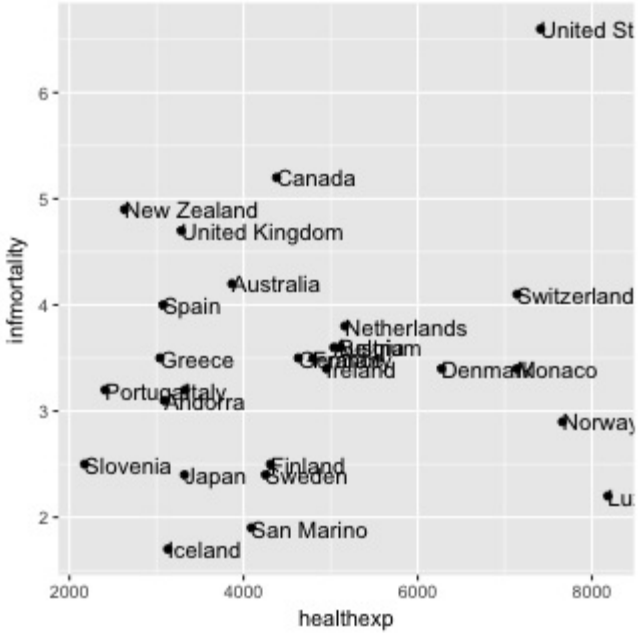


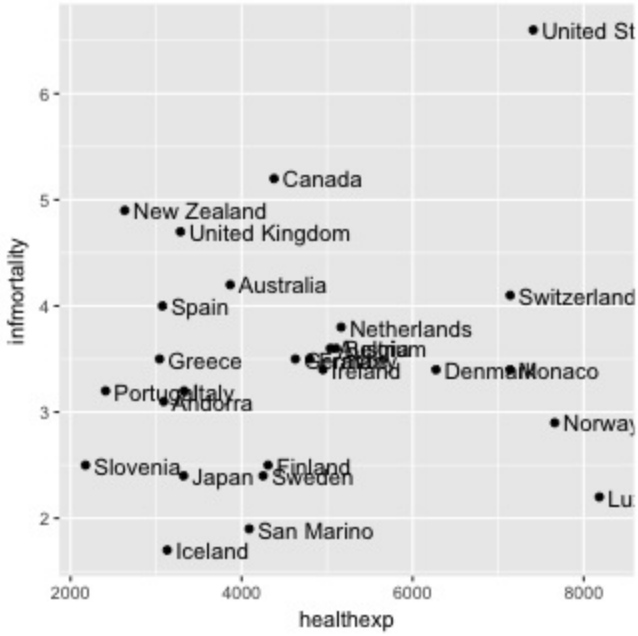


It often makes sense to right- or left-justify the labels relative to the points. To left-justify, set `hjust=0` (Figure \@ref(fig:FIG-SCATTER-LABEL-HJUST), left), and to right-justify, set `hjust=1`. As was the case with `vjust`, the labels will still slightly overlap with the points. This time, though, it's not a good idea to try to fix it by increasing or decreasing `hjust`. Doing so will shift the labels a distance proportional to the length of the label, making longer labels move further than shorter ones. It's better to just set hjust to 0 or 1, and then add or subtract a bit to or from `x` (Figure \@ref(fig:FIG-SCATTER-LABEL-HJUST), right):

```
sp + geom_text(aes(label = Name), size = 4, hjust = 0)
```

```
sp + geom_text(aes(x = healthexp+100, label = Name), size = 4, hjust = 0)
```





Note

If you are using a logarithmic axis, instead of adding to x or y, you'll need to *multiply* the x or y value by a number to shift the labels a consistent amount.

If you want to label just some of the points but want the placement to be handled automatically, you can add a new column to your data frame containing just the labels you want. Here's one way to do that: first we'll make a copy of the data we're using, then we'll copy the `Name` column into `Name1`, converting from a factor to a character vector, for reasons we'll see below.

```
cdat <- subset(countries, Year==2009 & healthexp>2000)

cdat$Name1 <- as.character(cdat$Name)
```

Next, we'll use the `%in%` operator to find *where* each name that we want to keep is. This returns a logical vector indicating which entries in the first vector, `cdat$Name1`, are present in the second vector, in which we specify the names of the countries we want to show:

```
idx <- cdat$Name1 %in% c("Canada", "Ireland", "United Kingdom",
    "United States", "New Zealand", "Iceland", "Japan", "Luxembourg",
    "Netherlands", "Switzerland")
idx
#>  [1] FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE
#> [12]  TRUE FALSE  TRUE  TRUE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE
#> [23] FALSE FALSE  TRUE  TRUE  TRUE
```

Then we'll use that boolean vector to overwrite all the *other* entries in `Name1` with a blank string. This is where the conversion from factor to character vector helps -- if `Name1` had remained a factor, then we would not be able to replace values with just any string.

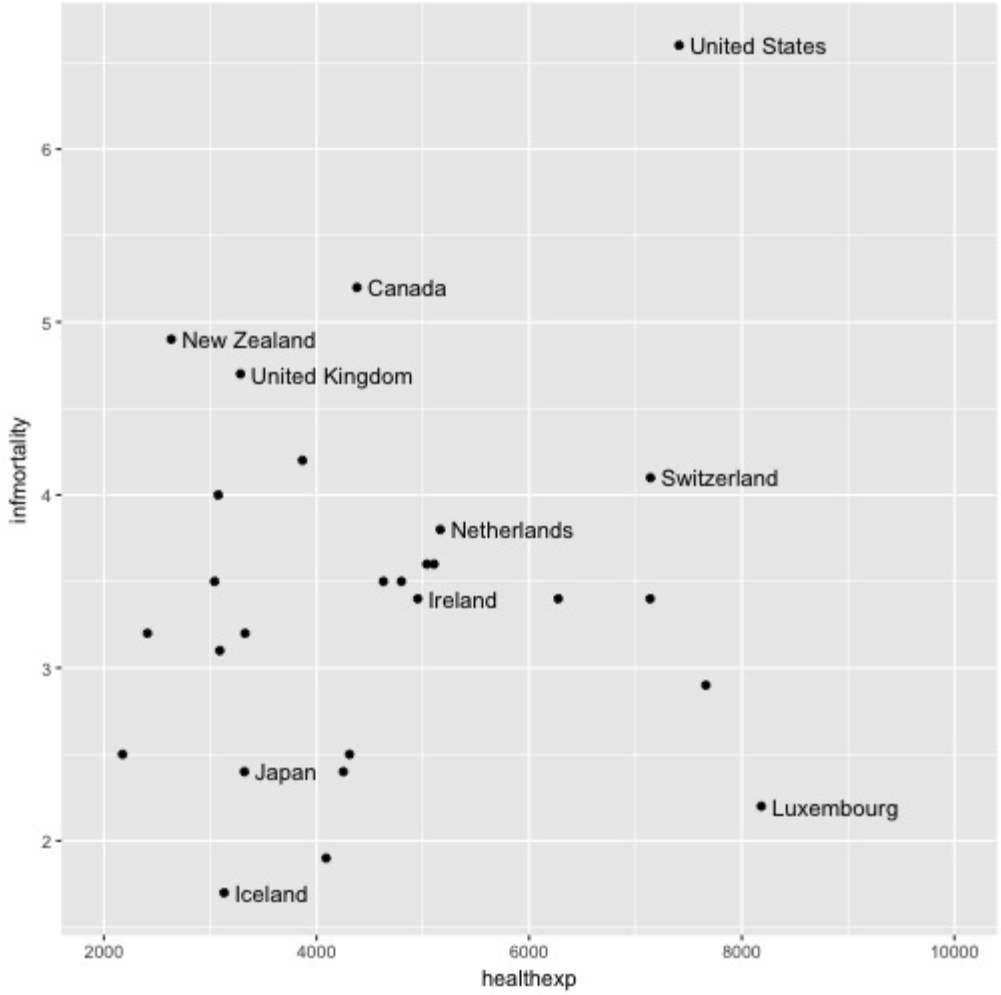
```
cdat$Name1[!idx] <- ""

cdat
#>           Name Code Year      GDP laborrate healthexp infmortality  Name1
#> 254    Andorra  AND 2009       NA        NA  3089.636          3.1
#> 560  Australia  AUS 2009 42130.82      65.2  3867.429          4.2
#> 611    Austria  AUT 2009 45555.43      60.4  5037.311          3.6
#> 968    Belgium  BEL 2009 43640.20      53.5  5104.019          3.6
#> 1733    Canada  CAN 2009 39599.04      67.8  4379.761          5.2 Canada
```

```
#> 2702   Denmark  DNK 2009 55933.35      65.4  6272.729          3.4
#> 3365   Finland  FIN 2009 44576.73      60.9  4309.604          2.5
#> 3416    France  FRA 2009 40663.05      56.1  4797.966          3.5
#> # ... with 19 more rows
```

TODO: Print first 2 and last 3 rows of data. via chunk option?

Now we can make the plot (Figure \@ref(fig:FIG-SCATTER-LABEL-SELECT)). This time, we'll also expand the *x* range so that the text will fit:

```
ggplot(cdat, aes(x = healthexp, y = infmortality)) +
    geom_point() +
    geom_text(aes(x = healthexp+100, label = Name1), size = 4, hjust = 0) +
    xlim(2000, 10000)
```



If any individual position adjustments are needed, you have a couple of options. One option is to copy the columns used for the *x* and *y* coordinates and modify the numbers for the individual items to move the text around. Make sure to use the original numbers for the coordinates of the points, of course! Another option is to save the output to a vector format such as PDF or SVG (see Recipes Recipe \@ref(RECIPE-OUTPUT-VECTOR) and Recipe \@ref(RECIPE-OUTPUT-VECTOR-SVG)), then edit it in a program like Illustrator or Inkscape.

See Also

For more on controlling the appearance of the text, see Recipe \@ref(RECIPE-APPEARANCE-TEXT-APPEARANCE).

If you want to manually edit a PDF or SVG file, see Recipe \@ref(RECIPE-OUTPUT-EDIT-VECTOR).

2.12 Creating a Balloon Plot

Problem

You want to make a balloon plot, where the area of the dots is proportional to their numerical value.

Solution

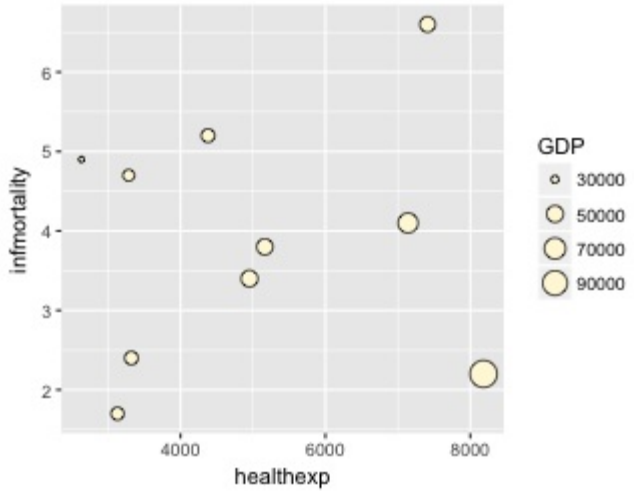
TODO: Check if the content from size/color recipe should be moved here.

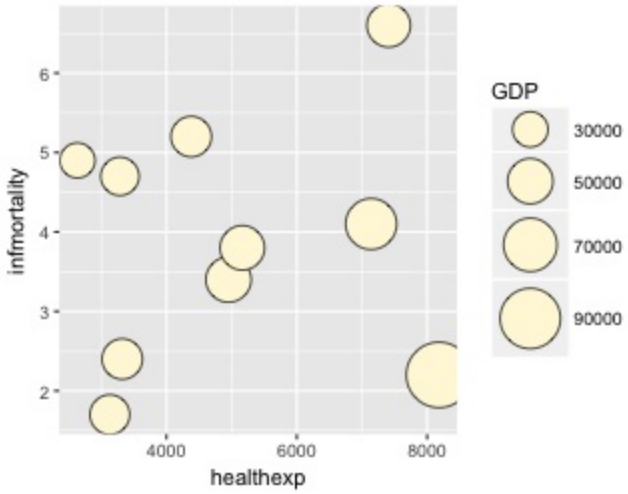
Use `geom_point()` with `scale_size_area()`. For this example, we'll use a subset of `countries`:

```
library(gcookbook) # For the data set
cdat <- subset(countries, Year==2009 & Name %in% c("Canada", "Ireland",
    "United Kingdom", "United States", "New Zealand", "Iceland", "Japan",
    "Luxembourg", "Netherlands", "Switzerland"))
cdat
#>             Name Code Year       GDP laborrate healthexp infmortality
#> 1733      Canada  CAN 2009  39599.04      67.8  4379.761          5.2
#> 4436     Iceland  ISL 2009  37972.24      77.5  3130.391          1.7
#> 4691     Ireland  IRL 2009  49737.93      63.6  4951.845          3.4
#> 4946       Japan  JPN 2009  39456.44      59.5  3321.466          2.4
#> 5864  Luxembourg  LUX 2009 106252.24      55.5  8182.855          2.2
#> 7088 Netherlands  NLD 2009  48068.35      66.1  5163.740          3.8
#> 7190 New Zealand  NZL 2009  29352.45      68.6  2633.625          4.9
#> 9587 Switzerland  CHE 2009  63524.65      66.9  7140.729          4.1
#> # ... with 2 more rows
```

If we just map `GDP` to `size`, the value of `GDP` gets mapped to the *radius* of the dots (Figure \@ref(fig:FIG-SCATTER-BALLOON), left), which is not what we want; a doubling of value results in a quadrupling of area, and this will distort the interpretation of the data. We instead want to map it to the *area*, and we can do this using `scale_size_area()` (Figure \@ref(fig:FIG-SCATTER-BALLOON), right):

```
p <- ggplot(cdat, aes(x = healthexp, y = infmortality, size = GDP)) +
     geom_point(shape = 21, colour = "black", fill = "cornsilk")

# GDP mapped to radius (default with scale_size_continuous)
p

# GDP mapped to area instead, and larger circles
p + scale_size_area(max_size = 15)
```

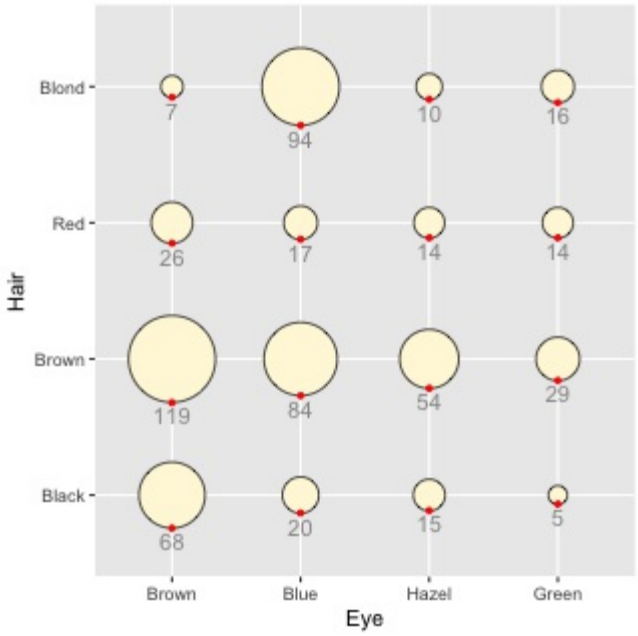


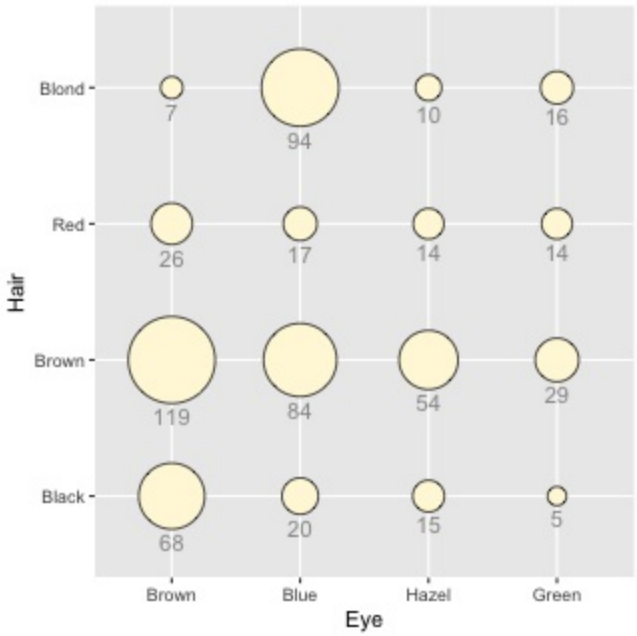

infmortality
6
5
4
3
2
4000
6000
8000
healthexp
GDP
30000
50000
70000
90000

Discussion

The example here is a scatter plot, but that is not the only way to use balloon plots. It may also be useful to use them to represent values on a grid, where the x- and y-axes are categorical, as in Figure \@ref(fig:FIG-SCATTER-BALLOON-CAT):

```
# Add up counts for male and female
hec <- HairEyeColor[,,"Male"] + HairEyeColor[,,"Female"]

# TODO: Switch to tidyr
# Convert to long format
library(reshape2)
hec <- melt(hec, value.name = "count")

# With red guide points
ggplot(hec, aes(x = Eye, y = Hair)) +
    geom_point(aes(size = count), shape = 21, colour = "black", fill = "cornsilk") +
    scale_size_area(max_size = 20, guide = FALSE) +
    geom_point(aes(y = as.numeric(Hair)-sqrt(count)/34), colour = "red", size = 1) +
    geom_text(aes(y = as.numeric(Hair)-sqrt(count)/34, label = count), vjust = 1.3,
              colour = "grey60", size = 4)

# Without red guide points
ggplot(hec, aes(x = Eye, y = Hair)) +
    geom_point(aes(size = count), shape = 21, colour = "black", fill = "cornsilk") +
    scale_size_area(max_size = 20, guide = FALSE) +
    geom_text(aes(y = as.numeric(Hair)-sqrt(count)/34, label = count), vjust = 1.3,
              colour = "grey60", size = 4)
```





In this example we've used a few tricks to add the text labels under the circles. First, we used `vjust=1.3` to justify the top of text slightly below the *y* coordinate. Next, we wanted to set the *y* coordinate so that it is at the bottom of each circle. This requires a little arithmetic: take the *numeric* value of Hair and subtract a small value from it, where the value depends in some way on count. This actually requires taking the square root of count, since the radius has a linear relationship with the square root of `count`. The number that this value is divided by (34 in this case) is found by trial and error; it depends on the particular data values, radius, text size, and output image size.

To help find the correct *y* offset, we can add guide points in red and adjusted the value until they lined up with the bottom of each circle. Once we have the correct value, we can place the text and remove the points.

The text under the circles is in a shade of grey. This is so that it doesn't jump out at the viewer and overwhelm the perceptual impact of the circles, but is still available if the viewer wants to know the exact values.

See Also

To add labels to the circles, see Recipes Recipe \@ref(RECIPE-SCATTER-LABELS) and Recipe \@ref(RECIPE-ANNOTATE-TEXT).

See Recipe \@ref(RECIPE-SCATTER-CONTINUOUS-SCATTER) for ways of mapping variables to other aesthetics in a scatter plot.

2.13 Making a Scatter Plot Matrix

Problem

You want to make a scatter plot matrix.

Solution

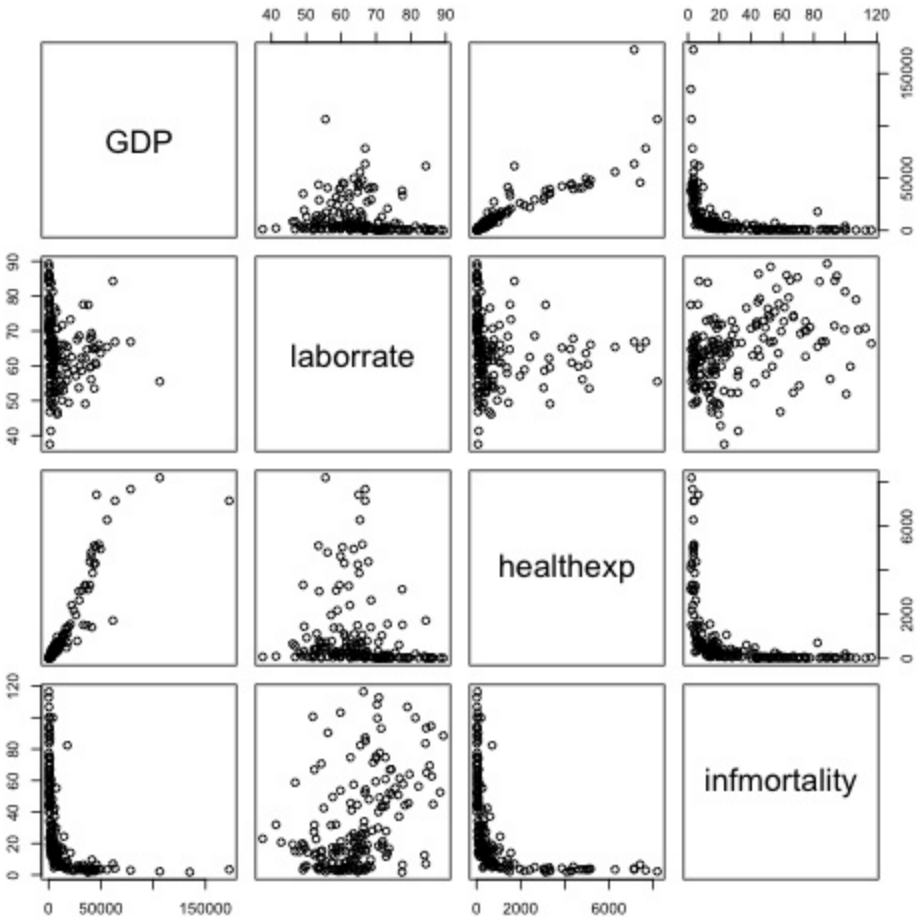
A scatter plot matrix is an excellent way of visualizing the pairwise relationships among several variables. To make one, use the `pairs()` function from R's base graphics.

For this example, we'll use a subset of the `countries` data. We'll pull out the data for the year 2009, and keep only the columns that are relevant:

```
library(gcookbook) # For the data set
c2009 <- subset(countries, Year==2009,
                select = c(Name, GDP, laborrate, healthexp, infmortality))
c2009
#>                     Name       GDP laborrate  healthexp infmortality
#> 50           Afghanistan        NA      59.8   50.88597        103.2
#> 101              Albania  3772.605      59.5  264.60406         17.2
#> 152              Algeria  4022.199      58.5  267.94653         32.0
#> 203       American Samoa        NA        NA         NA           NA
#> 254              Andorra        NA        NA 3089.63589          3.1
#> 305               Angola  4068.576      81.3  203.80787         99.9
#> 356 Antigua and Barbuda 12501.867        NA  652.53169          7.3
#> 407            Argentina  7665.073      65.0  730.17301         12.7
#> # ... with 208 more rows
```

To make the scatter plot matrix (Figure \@ref(fig:FIG-SCATTER-SPLOM)), we'll use columns 2 through 5 -- using the `Name` column wouldn't make sense, and it would produce strange-looking results:

```
pairs(c2009[,2:5])
```


GDP
laborrate
healthexp
infmortality
40 50 60 70 80 90
0 20 40 60 80 120
0 50000 150000
0 2000 6000

Discussion

We didn't use ggplot here because it doesn't make scatter plot matrices (at least, not well).

TODO: look more into ggally?

You can also use customized functions for the panels. To show the correlation coefficient of each pair of variables instead of a scatter plot, we'll define the function `panel.cor`. This will also show higher correlations in a larger font. Don't worry about the details for now -- just paste this code into your R session or script:

```
panel.cor <- function(x, y, digits = 2, prefix = "", cex.cor, ...) {
    usr <- par("usr")
    on.exit(par(usr))
    par(usr = c(0, 1, 0, 1))
    r <- abs(cor(x, y, use = "complete.obs"))
    txt <- format(c(r, 0.123456789), digits = digits)[1]
    txt <- paste(prefix, txt, sep = "")
    if(missing(cex.cor)) cex.cor <- 0.8/strwidth(txt)
    text(0.5, 0.5, txt, cex =  cex.cor * (1 + r) / 2)
}
```

To show histograms of each variable along the diagonal, we'll define `panel.hist`:

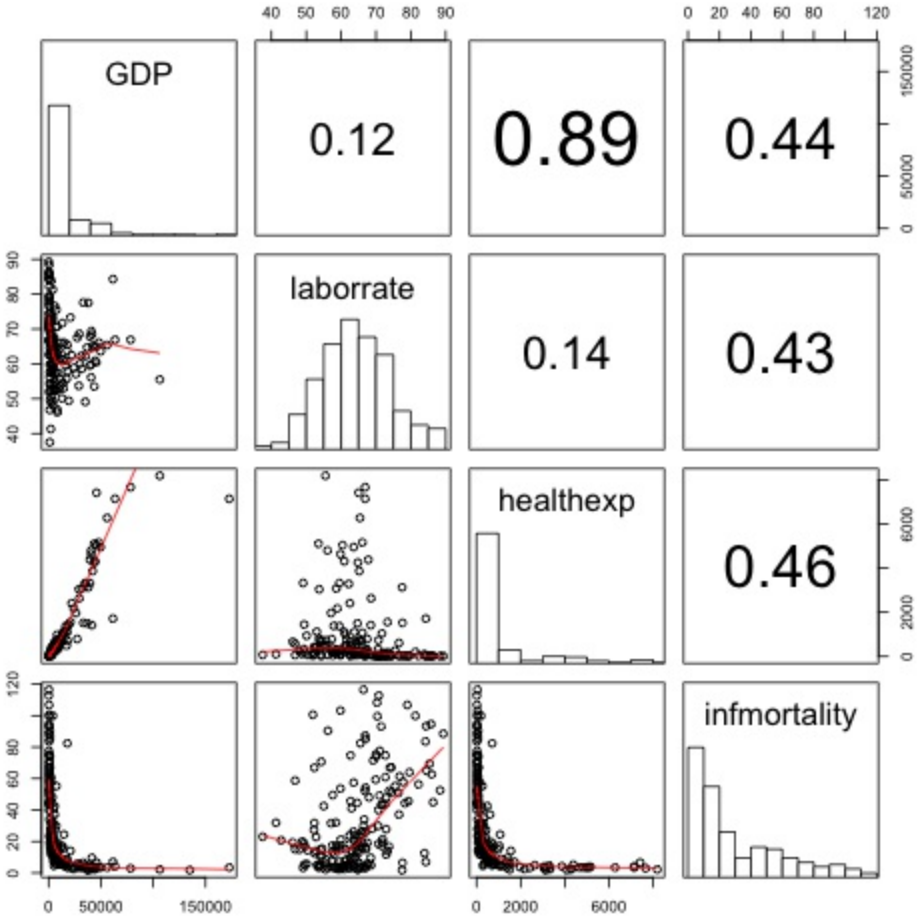
```
panel.hist <- function(x, ...) {
    usr <- par("usr")
    on.exit(par(usr))
    par(usr = c(usr[1:2], 0, 1.5) )
    h <- hist(x, plot = FALSE)
    breaks <- h$breaks
    nB <- length(breaks)
    y <- h$counts
    y <- y/max(y)
    rect(breaks[-nB], 0, breaks[-1], y, col = "white", ...)
}
```

Both of these panel functions are taken from the pairs help page, so if it's more convenient, you can simply open that help page, then copy and paste. The last line of this version of the `panel.cor` function is slightly modified, however, so that the changes in font size aren't as extreme as with the original.

Now that we've defined these functions we can use them for our scatter plot matrix, by telling `pairs()` to use `panel.cor` for the upper panels and `panel.hist` for the diagonal panels.

We'll also throw in one more thing: `panel.smooth` for the lower panels, which makes a scatter plot and adds a LOWESS smoothed line, as shown in Figure \@ref(fig:FIG-SCATTER-SPLOM-PANELS). (LOWESS is slightly different from LOESS, which we saw in Recipe \@ref(RECIPE-SCATTER-FITLINES), but the differences aren't important for this sort of rough exploratory visualization):

```
pairs(c2009[,2:5], upper.panel = panel.cor,
                   diag.panel  = panel.hist,
                   lower.panel = panel.smooth)
```



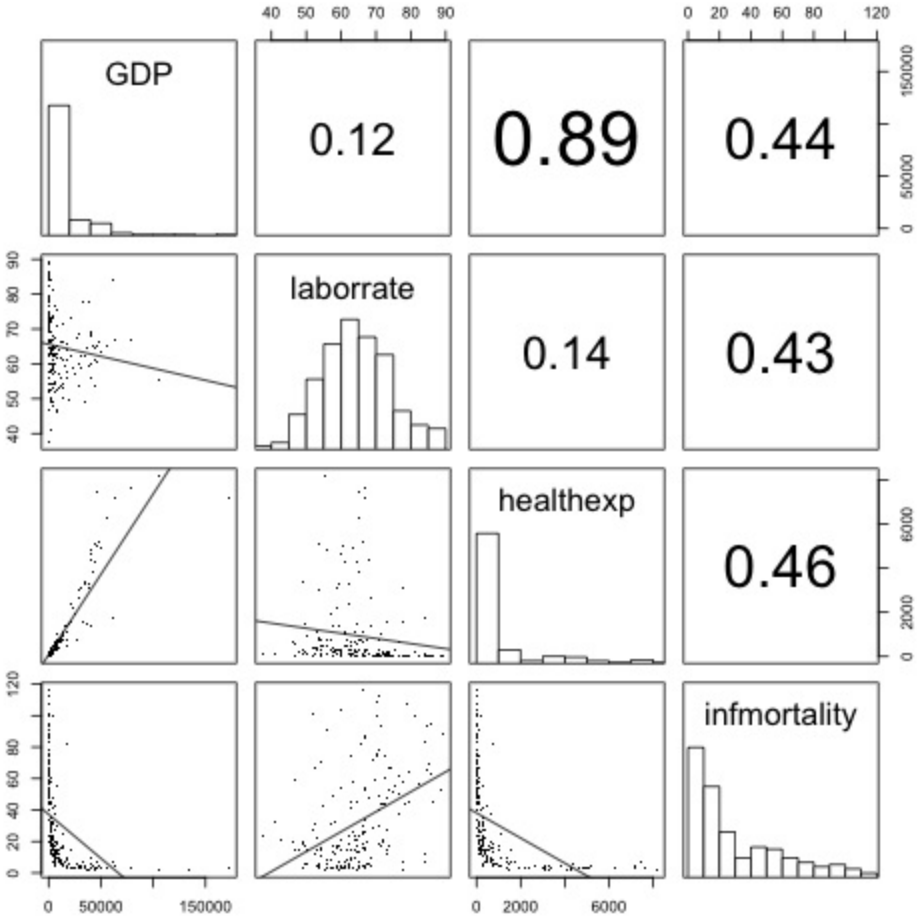
It may be more desirable to use linear regression lines instead of LOWESS lines. The `panel.lm` function will do the trick (unlike the previous panel functions, this one isn't in the pairs help page):

```
panel.lm <- function (x, y, col = par("col"), bg = NA, pch = par("pch"),
                          cex = 1, col.smooth = "black", ...) {
    points(x, y, pch = pch, col = col, bg = bg, cex = cex)
    abline(stats::lm(y ~ x),  col = col.smooth, ...)
}
```

This time the default line color is black instead of red, though you can change it here (and with `panel.smooth`) by setting `col.smooth` when you call `pairs()`.

We'll also use small points in the visualization, so that we can distinguish them a bit better (Figure \@ref(fig:FIG-SCATTER-SPLOM-PANELS2)). This is done by setting `pch="."`:

```
pairs(c2009[,2:5], pch = ".",
                   upper.panel = panel.cor,
                   diag.panel  = panel.hist,
                   lower.panel = panel.lm)
```



The size of the points can also be controlled using the cex parameter. The default value for cex is 1; make it smaller for smaller points and larger for larger points. Values below .5 might not render properly with PDF output.

See Also

To create a correlation matrix, see Recipe \@ref(RECIPE-MISCGRAPH-CORRMATRIX).

The `ggpairs()` function from the GGally package can also make scatter plot matrices.

Chapter 3. Summarized Data Distributions

This chapter explores how to visualize summarized distributions of data.

3.1 Making a Basic Histogram

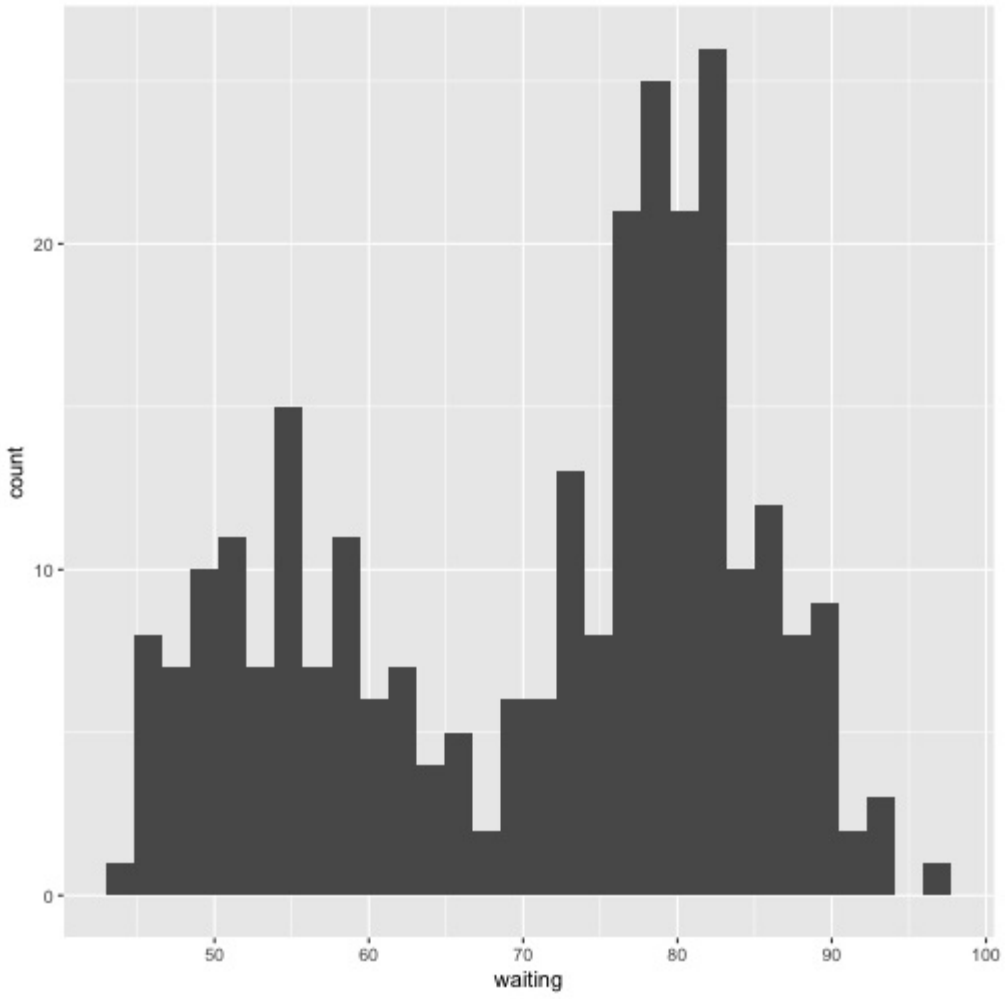
Problem

You want to make a histogram.

Solution

Use `geom_histogram()` and map a continuous variable to x (Figure \@ref(fig:FIG-DISTRIBUTION-HIST-BASIC)):

```
ggplot(faithful, aes(x=waiting)) + geom_histogram()
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Discussion

All `geom_histogram()` requires is one column from a data frame or a single vector of data. For this example we'll use the faithful data set, which contains data about the Old Faithful geyser in two columns: eruptions, which is the length of each eruption, and waiting, which is the length of time to the next eruption. We'll only use the waiting column in this example:

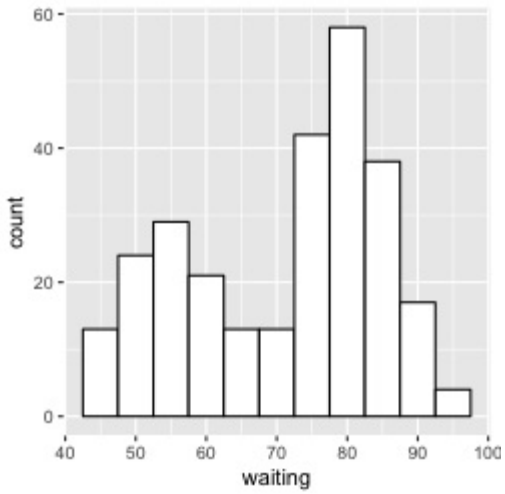
```
faithful
#>   eruptions waiting
#> 1     3.600      79
#> 2     1.800      54
#> 3     3.333      74
#> 4     2.283      62
#> 5     4.533      85
#> 6     2.883      55
#> 7     4.700      88
#> 8     3.600      85
#> # ... with 264 more rows
```

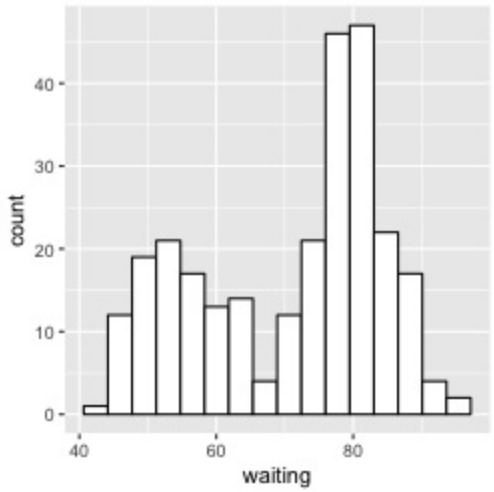
If you just want to get a quick look at some data that isn't in a data frame, you can get the same result by passing in NULL for the data frame and giving `ggplot()` a vector of values. This would have the same result as the previous code:

```
# Store the values in a simple vector
w <- faithful$waiting

ggplot(NULL, aes(x = w)) + geom_histogram()
```

By default, the data is grouped into 30 bins. This is an arbitrary value, and may be too fine or too coarse for your data. You can change the size of the bins by using binwidth, or you can divide the range of the data into a specific number of bins. The default colors -- a dark fill without an outline -- can make it difficult to see which bar corresponds to which value, so we'll also change the colors, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-HIST-WIDTH).

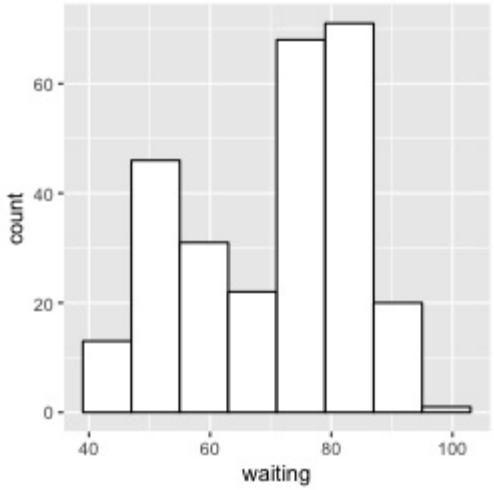
```
# Set the width of each bin to 5
ggplot(faithful, aes(x = waiting)) +
    geom_histogram(binwidth = 5, fill = "white", colour = "black")

# Divide the x range into 15 bins
binsize <- diff(range(faithful$waiting))/15
ggplot(faithful, aes(x = waiting)) +
    geom_histogram(binwidth = binsize, fill = "white", colour = "black")
```

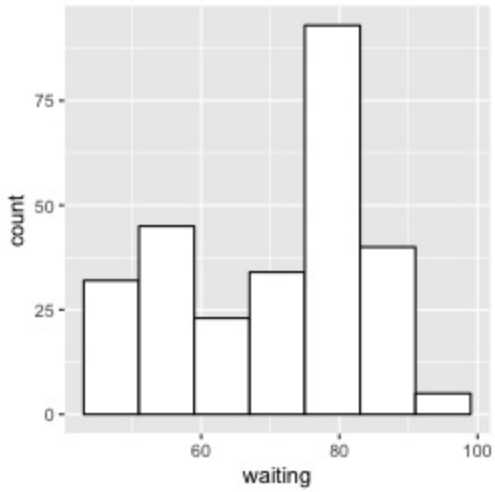




Sometimes the appearance of the histogram will be very dependent on the width of the bins and where exactly the boundaries between bins are. In Figure \@ref(fig:FIG-DISTRIBUTION-HIST-BOUNDARY), we'll use a bin width of 8. In the version on the left, we'll use the origin parameter to put boundaries at 31, 39, 47, etc., while in the version on the right, we'll shift it over by 4, putting boundaries at 35, 43, 51, etc.:

```
h <- ggplot(faithful, aes(x = waiting))  # Save the base object for reuse

h + geom_histogram(binwidth = 8, fill = "white", colour = "black", boundary = 31)

h + geom_histogram(binwidth = 8, fill = "white", colour = "black", boundary = 35)
```





The results look quite different, even though they have the same bin size. The `faithful` data set is not particularly small, with 272 observations; with smaller data sets, this is even more of an issue. When visualizing your data, it's a good idea to experiment with different bin sizes and boundary points.

If your data has discrete values, it may matter that the histogram bins are asymmetrical. They are *closed* on the lower bound and *open* on the upper bound. If you have bin boundaries at 1, 2, 3, etc., then the bins will be [1, 2), [2, 3), and so on. In other words, the first bin contains 1 but not 2, and the second bin contains 2 but not 3.

See Also

Frequency polygons provide a better way of visualizing multiple distributions without the bars interfering with each other. See Recipe \@ref(RECIPE-DISTRIBUTION-FREQPOLY).

3.2 Making Multiple Histograms from Grouped Data

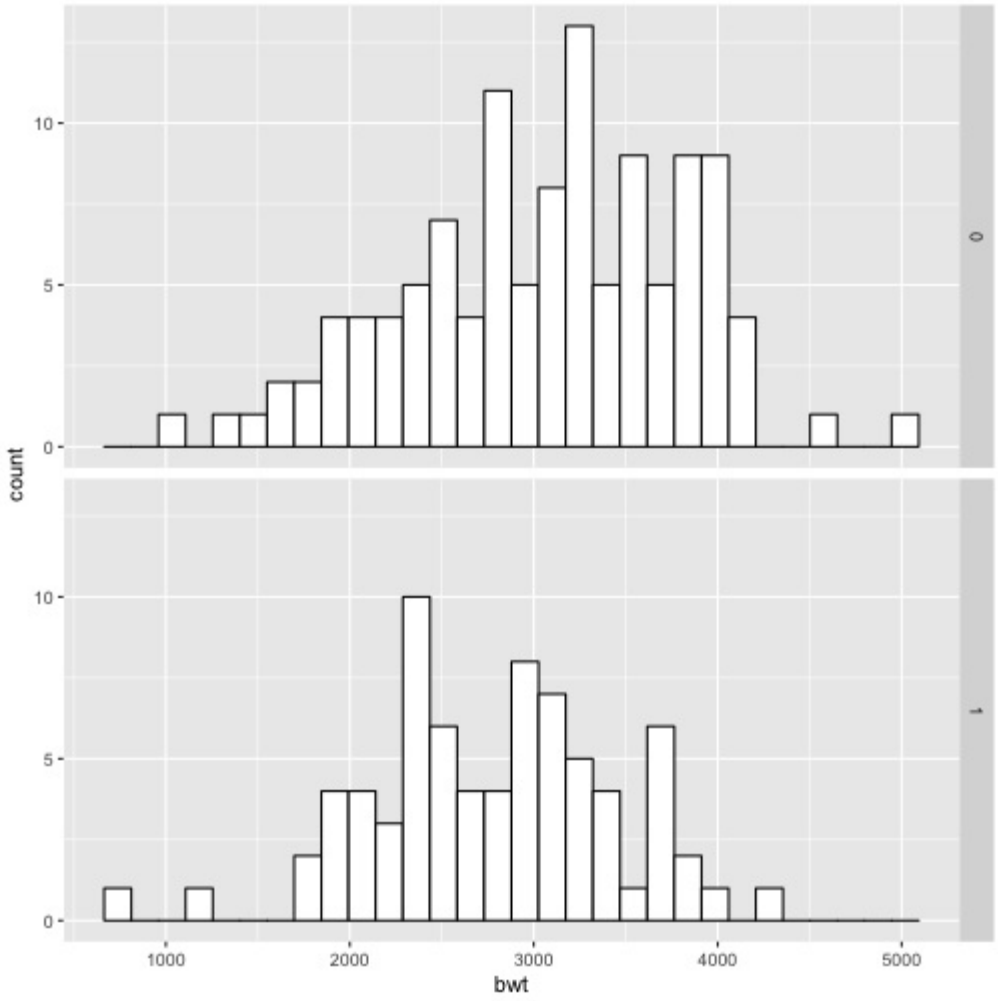
Problem

You want to make histograms of multiple groups of data.

Solution

Use `geom_histogram()` and use facets for each group, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-HISTOGRAM-FACET):

```
library(MASS) # For the data set

# Use smoke as the faceting variable
ggplot(birthwt, aes(x = bwt)) + geom_histogram(fill = "white", colour = "black") +
    facet_grid(smoke ~ .)
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Discussion

To make these plots, the data must all be in one data frame, with one column containing a categorical variable used for grouping.

For this example, we used the `birthwt` data set. It contains data about birth weights and a number of risk factors for low birth weight:

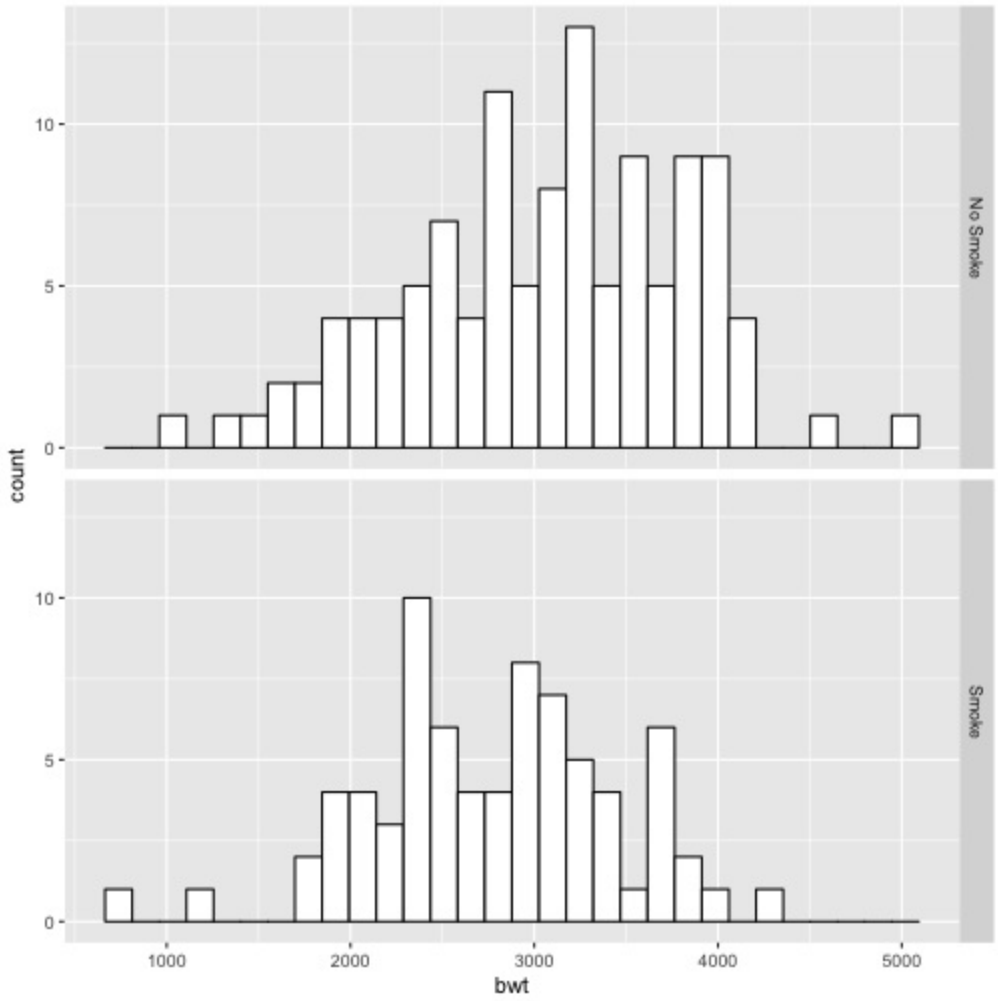
```
birthwt
#>    low age lwt race smoke ptl ht ui ftv  bwt
#> 85   0  19 182    2     0   0  0  1   0 2523
#> 86   0  33 155    3     0   0  0  0   3 2551
#> 87   0  20 105    1     1   0  0  0   1 2557
#> 88   0  21 108    1     1   0  0  1   2 2594
#> 89   0  18 107    1     1   0  0  1   0 2600
#> 91   0  21 124    3     0   0  0  0   0 2622
#> 92   0  22 118    1     0   0  0  0   1 2637
#> 93   0  17 103    3     0   0  0  0   1 2637
#> # ... with 181 more rows
```

One problem with the faceted graph is that the facet labels are just 0 and 1, and there's no label indicating that those values are for smoke. To change the labels, we need to change the names of the factor levels. First we'll take a look at the factor levels, then we'll assign new factor level names, in the same order:

```
birthwt1 <- birthwt # Make a copy of the data

# Convert smoke to a factor
birthwt1$smoke <- factor(birthwt1$smoke)
levels(birthwt1$smoke)
#> [1] "0" "1"

library(dplyr) # For the recode() function
birthwt1$smoke <- recode(birthwt1$smoke, "0" = "No Smoke", "1" = "Smoke")
```

Now when we plot it again, it shows the new labels (Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-HISTOGRAM-FACET-LABELS)).

```
ggplot(birthwt1, aes(x = bwt)) + geom_histogram(fill = "white", colour = "black") +
    facet_grid(smoke ~ .)
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



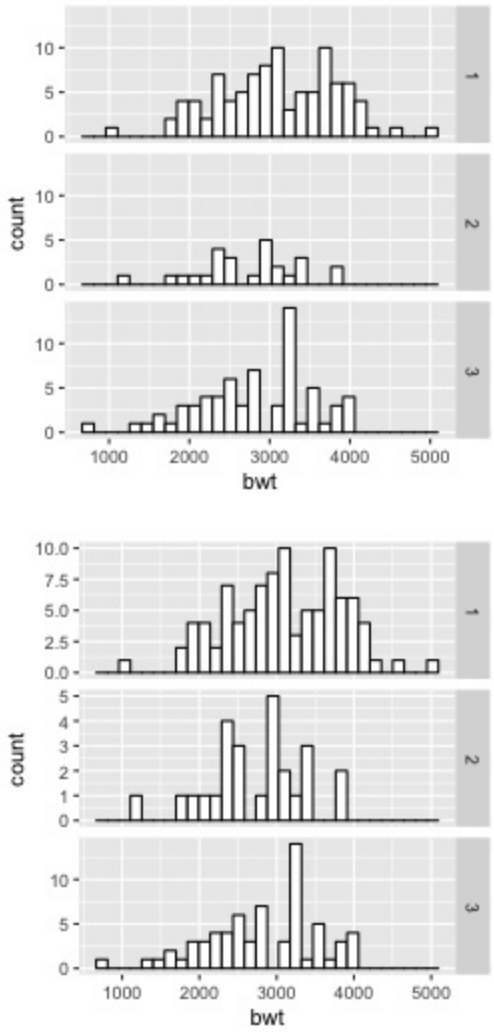
With facets, the axes have the same *y* scaling in each facet. If your groups have different sizes, it might be hard to compare the *shapes* of the distributions of each one. For example, see what happens when we facet the birth weights by race (Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-HISTOGRAM-FACET-SCALESFREE), left):

```
ggplot(birthwt, aes(x = bwt)) + geom_histogram(fill = "white", colour = "black") +
    facet_grid(race ~ .)
```

To allow the *y* scales to be resized independently (Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-HISTOGRAM-FACET-SCALESFREE), right), use `scales="free"`. Note that this will only allow the *y* scales to be free-the *x* scales will still be fixed because the histograms are aligned with respect to that axis:

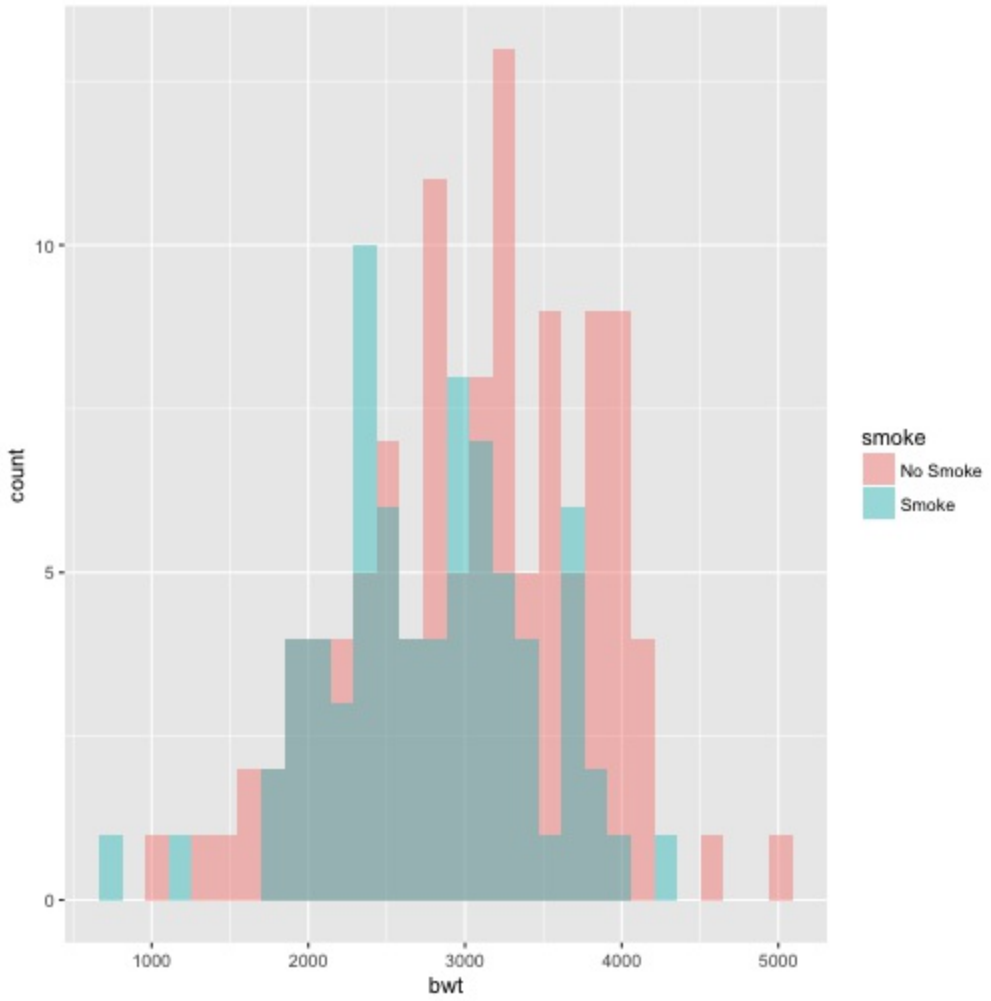
```
ggplot(birthwt, aes(x = bwt)) + geom_histogram(fill = "white", colour = "black") +
    facet_grid(race ~ ., scales = "free")
```

```
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Another approach is to map the grouping variable to fill, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-HISTOGRAM-FILL). The grouping variable must be a factor or character vector. In the birthwt data set, the desired grouping variable, smoke, is stored as a number, so we'll use the birthwt1 data set we created above, in which smoke is a factor:

```
# Convert smoke to a factor
birthwt1$smoke <- factor(birthwt1$smoke)

# Map smoke to fill, make the bars NOT stacked, and make them semitransparent
ggplot(birthwt1, aes(x = bwt, fill = smoke)) +
    geom_histogram(position = "identity", alpha = 0.4)
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



The position="identity" is important. Without it, ggplot() will stack the histogram bars on top of each other vertically, making it much more difficult to see the distribution of each group.

3.3 Making a Density Curve

Problem

You want to make a kernel density estimate curve.

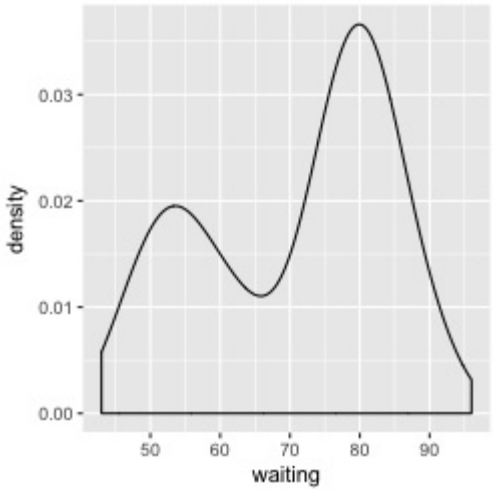
Solution

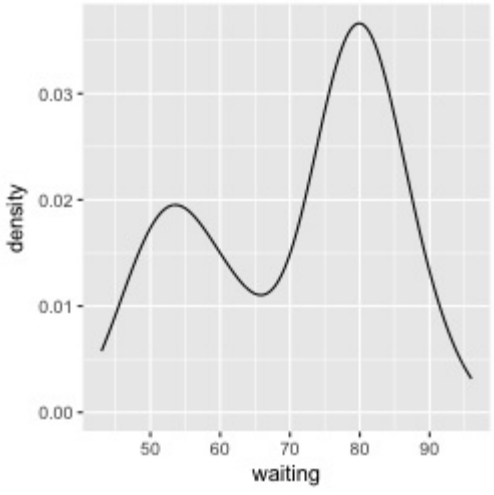
Use `geom_density()` and map a continuous variable to x (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY-BASIC)):

```
ggplot(faithful, aes(x = waiting)) + geom_density()
```

If you don’t like the lines along the side and bottom,you can use `geom_line(stat="density")` (see Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY-BASIC), right):

```
# The expand_limits() increases the y range to include the value 0
ggplot(faithful, aes(x = waiting)) + geom_line(stat = "density") +
    expand_limits(y = 0)
```





Discussion

Like `geom_histogram()`, `geom_density()` requires just one column from a data frame. For this example, we'll use the faithful data set, which contains data about the Old Faithful geyser in two columns: `eruptions`, which is the length of each eruption, and `waiting`, which is the length of time to the next eruption. We'll only use the waiting column in this example:

```
faithful
#>   eruptions waiting
#> 1     3.600      79
#> 2     1.800      54
#> 3     3.333      74
#> 4     2.283      62
#> 5     4.533      85
#> 6     2.883      55
#> 7     4.700      88
#> 8     3.600      85
#> # ... with 264 more rows
```

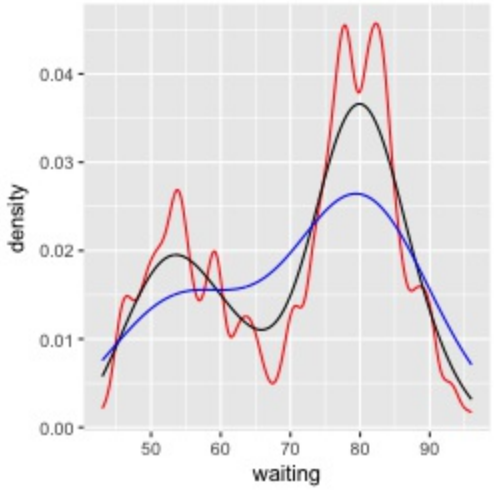
The second method mentioned earlier uses `geom_line()` and tells it to use the "density" statistical transformation. This is essentially the same as the first method, using `geom_density()`, except the former draws it with a closed polygon.

As with `geom_histogram()`, if you just want to get a quick look at data that isn't in a data frame, you can get the same result by passing in `NULL` for the data and giving `ggplot()` a vector of values. This would have the same result as the first solution:

```
# Store the values in a simple vector
w <- faithful$waiting

ggplot(NULL, aes(x = w)) + geom_density()
```

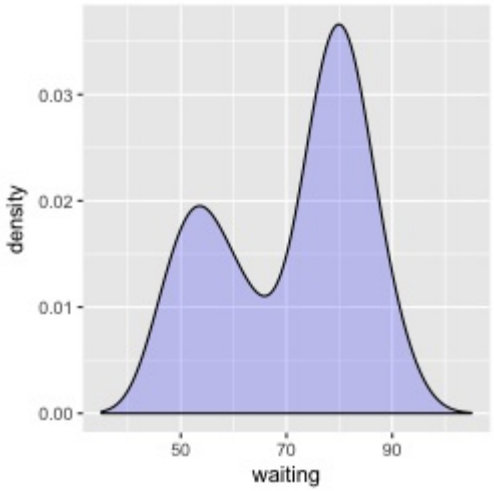
A kernel density curve is an estimate of the population distribution, based on the sample data. The amount of smoothing depends on the *kernel bandwidth*: the larger the bandwidth, the more smoothing there is. The bandwidth can be set with the `adjust` parameter, which has a default value of 1. Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY-ADJUST) shows what happens with a smaller and larger value of adjust:

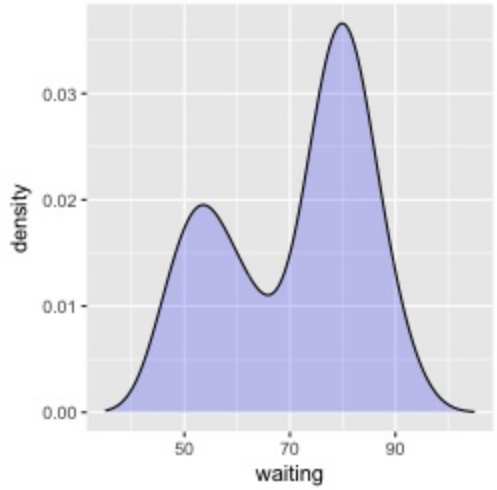
```
ggplot(faithful, aes(x = waiting)) +
    geom_line(stat = "density", adjust = .25, colour = "red") +
    geom_line(stat = "density") +
    geom_line(stat = "density", adjust = 2, colour = "blue")
```



In this example, the *x* range is automatically set so that it contains the data, but this results in the edge of the curve getting clipped. To show more of the curve, set the *x* limits (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY-WIDTH)). We'll also add an 80% transparent fill, with `alpha=.2`:

```
ggplot(faithful, aes(x = waiting)) +
    geom_density(fill = "blue", alpha = .2) +
    xlim(35, 105)

# This draws a blue polygon with geom_density(), then adds a line on top
ggplot(faithful, aes(x = waiting)) +
    geom_density(fill = "blue", colour = NA, alpha = .2) +
    geom_line(stat = "density") +
    xlim(35, 105)
```

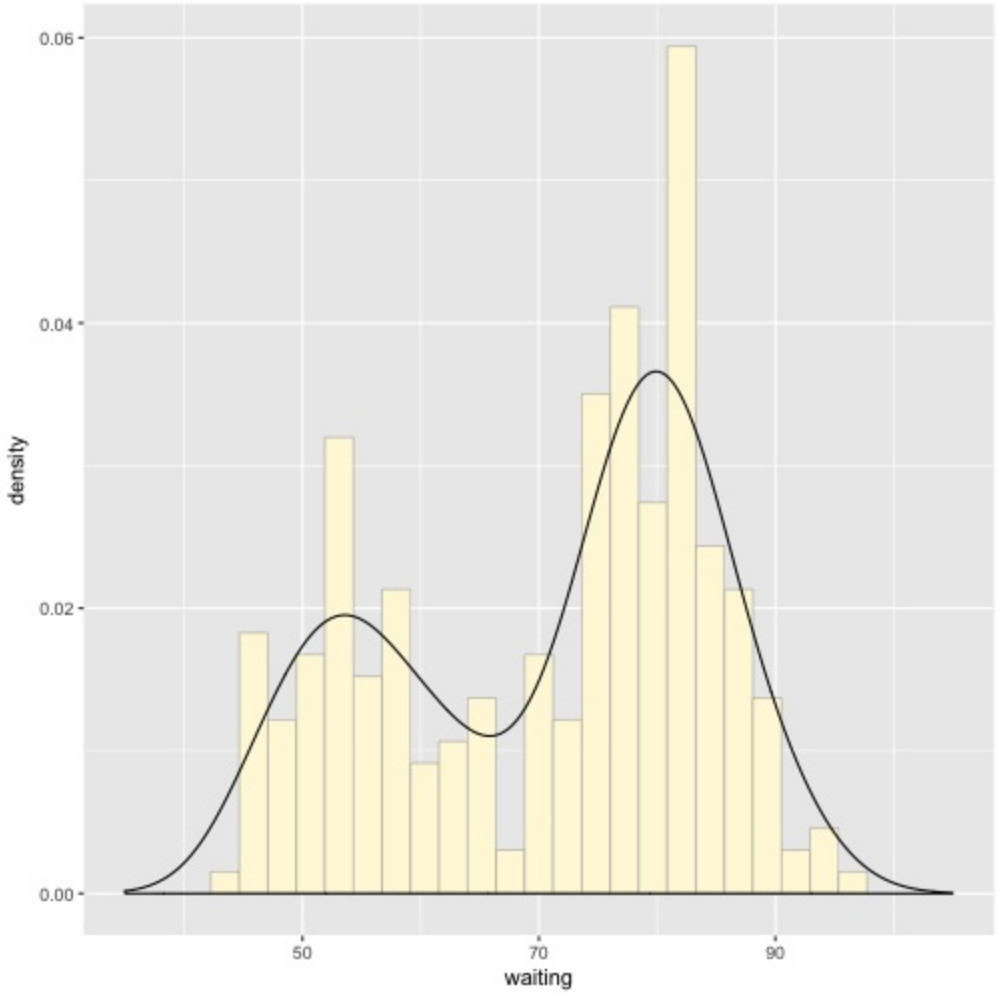




If this edge-clipping happens with your data, it might mean that your curve is too smooth -- if the curve is much wider than your data, it might not be the best model of your data. Or it could be because you have a small data set.

To compare the theoretical and observed distributions, you can overlay the density curve with the histogram. Since the *y* values for the density curve are small (the area under the curve always sums to 1), it would be barely visible if you overlaid it on a histogram without any transformation. To solve this problem, you can scale down the histogram to match the density curve with the mapping `y=..density..`. Here we'll add `geom_histogram()` first, and then layer `geom_density()` on top (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY-HIST)):

```
ggplot(faithful, aes(x = waiting, y = ..density..)) +
    geom_histogram(fill = "cornsilk", colour = "grey60", size = .2) +
    geom_density() +
    xlim(35, 105)
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```


0.06
0.04
0.02
0.00
density
50
70
90
waiting

See Also

See Recipe \@ref(RECIPE-DISTRIBUTION-VIOLIN) for information on violin plots, which are another way of representing density curves and may be more appropriate for comparing multiple distributions.

3.4 Making Multiple Density Curves from Grouped Data

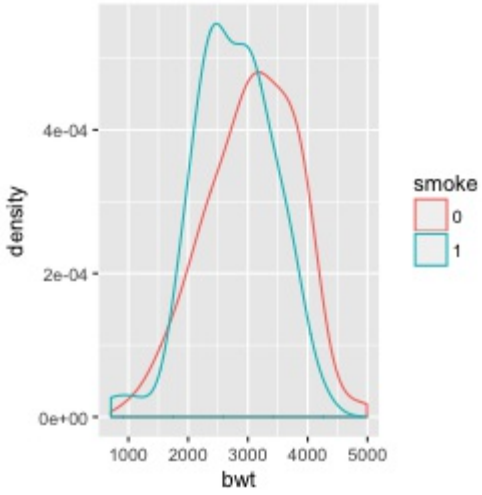
Problem

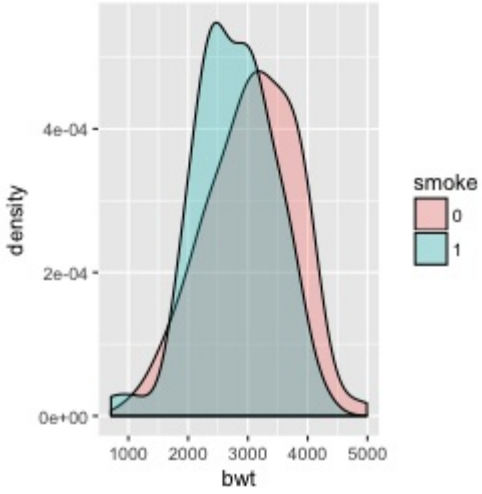
You want to make density curves of multiple groups of data.

Solution

Use `geom_density()`, and map the grouping variable to an aesthetic like `colour` or `fill`, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-DENSITY). The grouping variable must be a factor or character vector. In the `birthwt` data set, the desired grouping variable, `smoke`, is stored as a number, so we have to convert it to a factor first:

```
library(MASS) # For the data set
# Make a copy of the data
birthwt1 <- birthwt

# Convert smoke to a factor
birthwt1$smoke <- factor(birthwt1$smoke)

# Map smoke to colour
ggplot(birthwt1, aes(x = bwt, colour = smoke)) + geom_density()

# Map smoke to fill and make the fill semitransparent by setting alpha
ggplot(birthwt1, aes(x = bwt, fill = smoke)) + geom_density(alpha = .3)
```





Discussion

To make these plots, the data must all be in one data frame, with one column containing a categorical variable used for grouping.

For this example, we used the `birthwt` data set. It contains data about birth weights and a number of risk factors for low birth weight:

```
birthwt
#>    low age lwt race smoke ptl ht ui ftv  bwt
#> 85   0  19 182    2     0   0  0  1   0 2523
#> 86   0  33 155    3     0   0  0  0   3 2551
#> 87   0  20 105    1     1   0  0  0   1 2557
#> 88   0  21 108    1     1   0  0  1   2 2594
#> 89   0  18 107    1     1   0  0  1   0 2600
#> 91   0  21 124    3     0   0  0  0   0 2622
#> 92   0  22 118    1     0   0  0  0   1 2637
#> 93   0  17 103    3     0   0  0  0   1 2637
#> # ... with 181 more rows
```

We looked at the relationship between `smoke` (smoking) and `bwt` (birth weight in grams). The value of `smoke` is either 0 or 1, but since it's stored as a numeric vector, ggplot doesn't know that it should be treated as a categorical variable. To make it so ggplot knows to treat `smoke` as categorical, we can either convert that column of the data frame to a factor, or tell ggplot to treat it as a factor by using `factor(smoke)` inside of the `aes()` statement. For these examples, we converted it to a factor in the data.

Another method for visualizing the distributions is to use facets, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-DENSITY-FACET). We can align the facets vertically or horizontally. Here we'll align them vertically so that it's easy to compare the two distributions:

```
ggplot(birthwt1, aes(x = bwt)) + geom_density() + facet_grid(smoke ~ .)
```

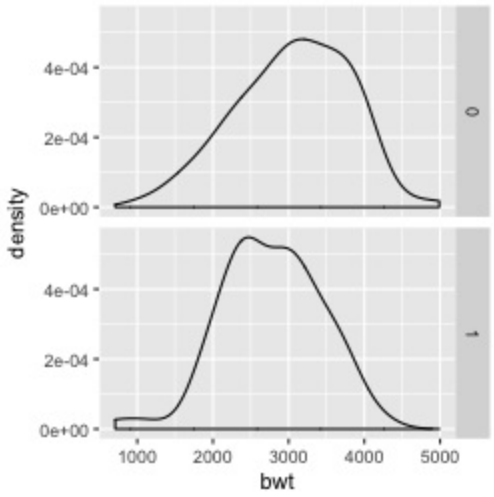
One problem with the faceted graph is that the facet labels are just 0 and 1, and there's no label indicating that those values are for smoke. To change the labels, we need to change the names of the factor levels. First we'll take a look at the factor levels, then we'll assign new factor level names:

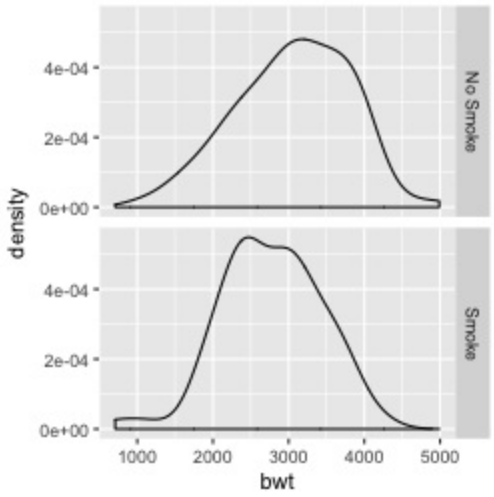
```
levels(birthwt1$smoke)
library(dplyr) # For the recode() function
birthwt1$smoke <- recode(birthwt1$smoke, "0" = "No Smoke", "1" = "Smoke")
```

Now when we plot it again, it shows the new labels (Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-DENSITY-FACET), right):

```
ggplot(birthwt1, aes(x = bwt)) + geom_density() + facet_grid(smoke ~ .)
```

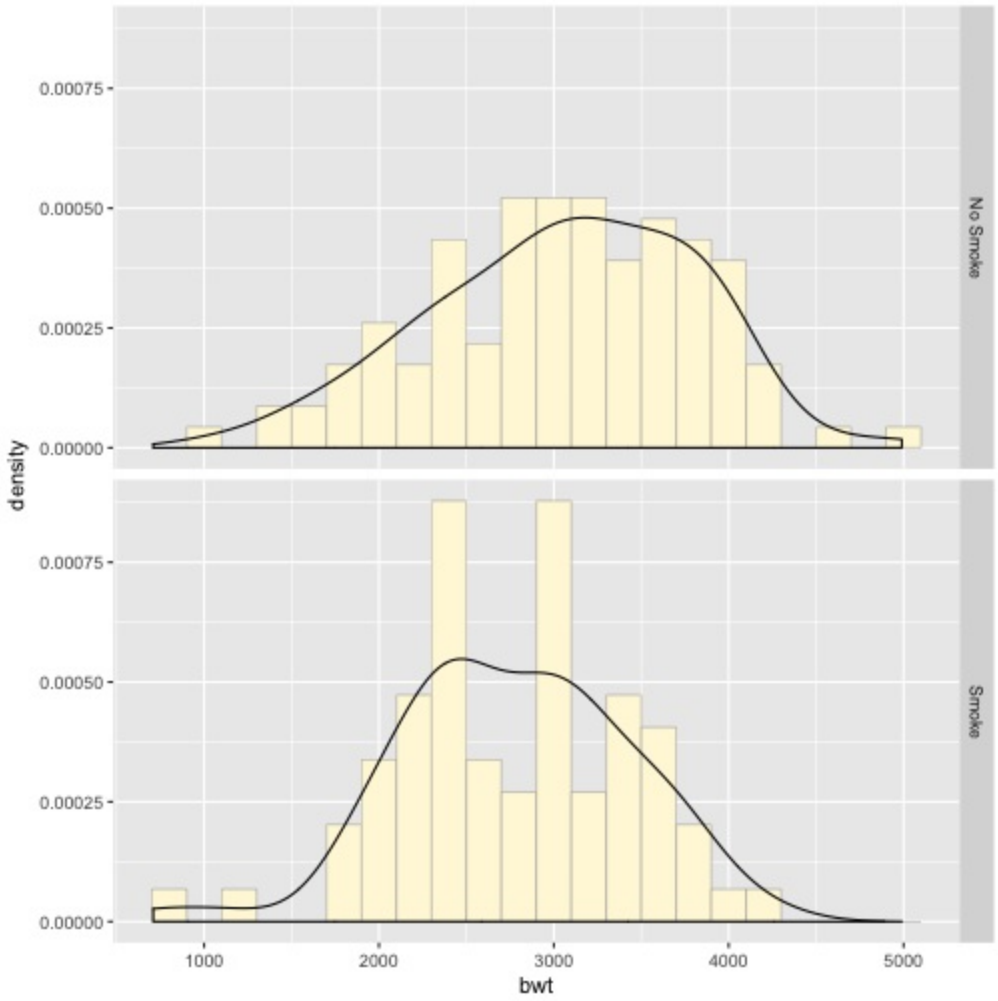
```
#> [1] "0" "1"
```





If you want to see the histograms along with the density curves, the best option is to use facets, since other methods of visualizing both histograms in a single graph can be difficult to interpret. To do this, map `y=..density..`, so that the histogram is scaled down to the height of the density curves. In this example, we'll also make the histogram bars a little less prominent by changing the colors (Figure \@ref(fig:FIG-DISTRIBUTION-MULTI-DENSITY-HIST)):

```
ggplot(birthwt1, aes(x = bwt, y = ..density..)) +
    geom_histogram(binwidth = 200, fill = "cornsilk", colour = "grey60", size = .2) +
    geom_density() +
    facet_grid(smoke ~ .)
```


density
0.00075
0.00050
0.00025
0.00000
No Smoke
Smoke
1000
2000
3000
4000
5000
bwt

3.5 Making a Frequency Polygon

Problem

You want to make a frequency polygon.

Solution

Use geom_`freqpoly()` (Figure \@ref(fig:FIG-DISTRIBUTION-FREQPOLY)):

```
ggplot(faithful, aes(x=waiting)) + geom_freqpoly()
```

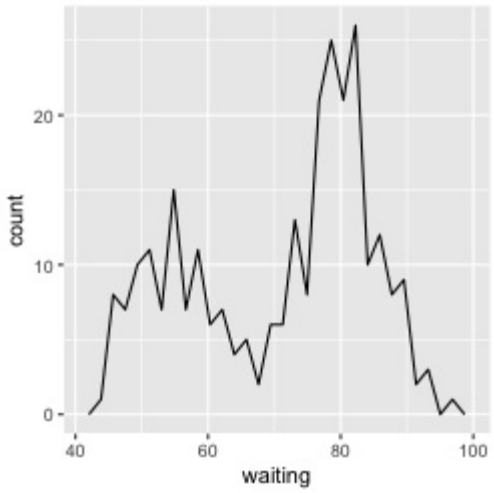
Discussion

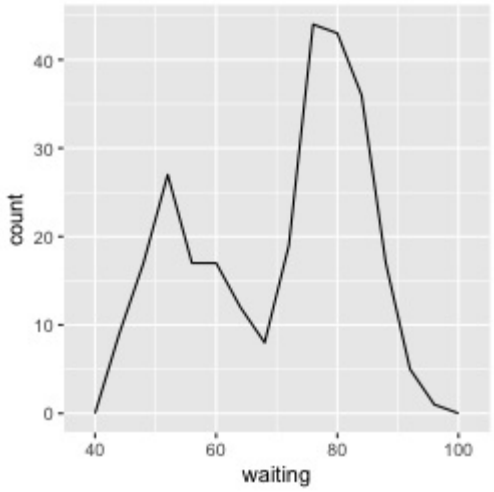
A frequency polygon appears similar to a kernel density estimate curve, but it shows the same information as a histogram. That is, like a histogram, it shows what is in the data, whereas a kernel density estimate is just that-an estimate-and requires you to pick some value for the bandwidth.

Also like a histogram, you can control the bin width for the frequency polygon (Figure \@ref(fig:FIG-DISTRIBUTION-FREQPOLY), right):

```
ggplot(faithful, aes(x = waiting)) + geom_freqpoly(binwidth = 4)
```

```
#> `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```





Or, instead of setting the width of each bin directly, you can divide the *x* range into a particular number of bins:

```
# Use 15 bins
binsize <- diff(range(faithful$waiting))/15
ggplot(faithful, aes(x = waiting)) + geom_freqpoly(binwidth = binsize)
```

See Also

Histograms display the same information, but with bars instead of lines. See Recipe \@ref(RECIPE-DISTRIBUTION-BASIC-HIST).

3.6 Making a Basic Box Plot

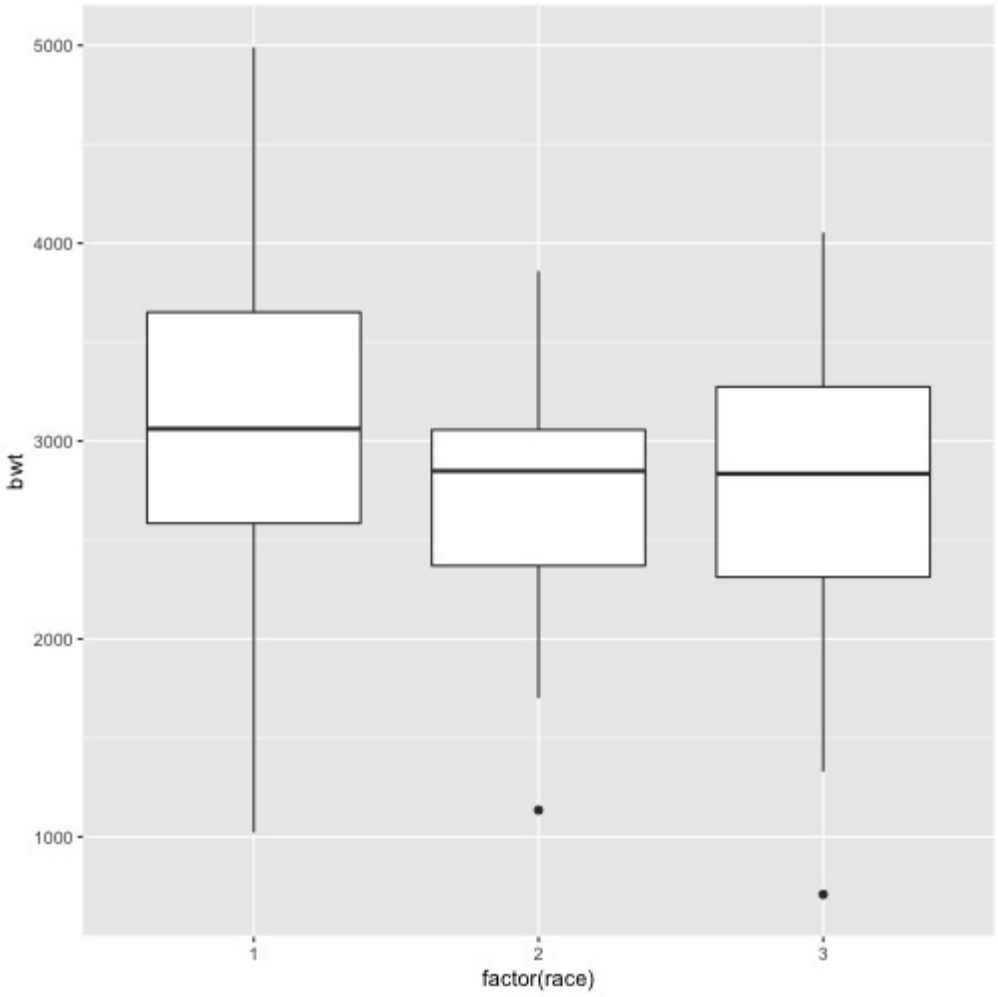
Problem

You want to make a box (or box-and-whiskers) plot.

Solution

Use `geom_boxplot()`, mapping a continuous variable to y and a discrete variable to x (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-BASIC)):

```
library(MASS) # For the data set

ggplot(birthwt, aes(x = factor(race), y = bwt)) + geom_boxplot()
```




```
# Use factor() to convert numeric variable to discrete
```

Discussion

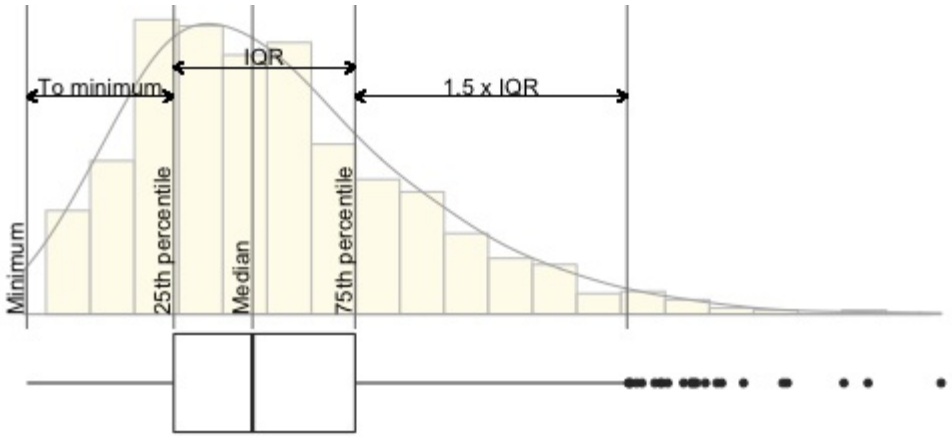
For this example, we used the `birthwt` data set from the MASS package It contains data about birth weights and a number of risk factors for low birth weight:

```
birthwt
#>    low age lwt race smoke ptl ht ui ftv  bwt
#> 85   0  19 182    2     0   0  0  1   0 2523
#> 86   0  33 155    3     0   0  0  0   3 2551
#> 87   0  20 105    1     1   0  0  0   1 2557
#> 88   0  21 108    1     1   0  0  1   2 2594
#> 89   0  18 107    1     1   0  0  1   0 2600
#> 91   0  21 124    3     0   0  0  0   0 2622
#> 92   0  22 118    1     0   0  0  0   1 2637
#> 93   0  17 103    3     0   0  0  0   1 2637
#> # ... with 181 more rows
```

In Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-BASIC), the data is divided into groups by race, and we visualize the distributions of `bwt` for each group. The value of `race` is 1, 2, or 3, but since it's stored as a numeric vector, ggplot doesn't know how to use it as a grouping variable. To make this work, we can modify the data frame by converting `race` to a factor, or tell ggplot to treat it as a factor by using `factor(race)` inside of the aes() statement. In the preceding example, we used `factor(race)`.

A box plot consists of a box and "whiskers." The box goes from the 25th percentile to the 75th percentile of the data, also known as the *inter-quartile range* (IQR). There's a line indicating the median, or 50th percentile of the data. The whiskers start from the edge of the box and extend to the furthest data point that is within 1.5 times the IQR. If there are any data points that are past the ends of the whiskers, they are considered outliers and displayed with dots. Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-DIAGRAM) shows the relationship between a histogram, a density curve, and a box plot, using a skewed data set.

```
#> Warning: Removed 1 rows containing missing values (geom_bar).
```



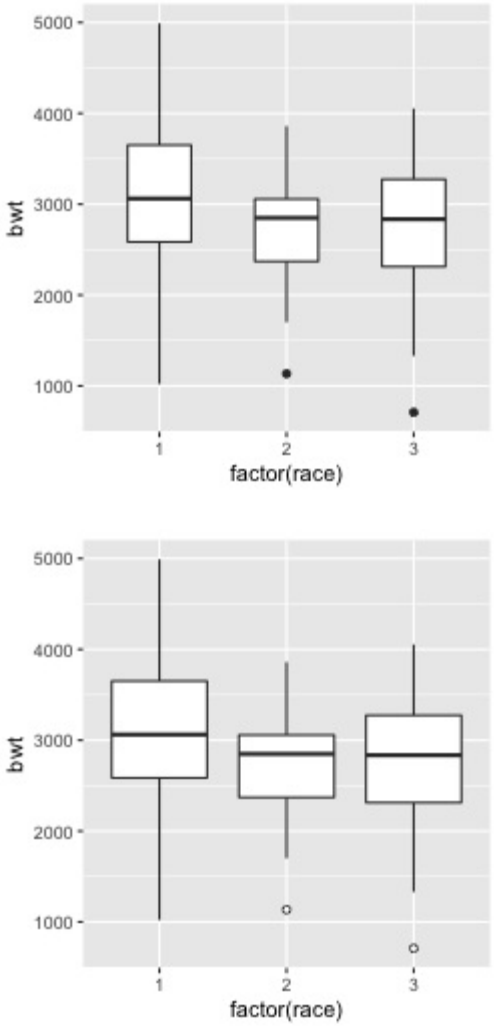
To change the width of the boxes, you can set width (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-WIDTH-POINT), left):

```
ggplot(birthwt, aes(x = factor(race), y = bwt)) + geom_boxplot(width = .5)
```

If there are many outliers and there is overplotting, you can change the size and shape of the outlier points with outlier.size and outlier.shape. The default size is 2 and the default shape is 16.

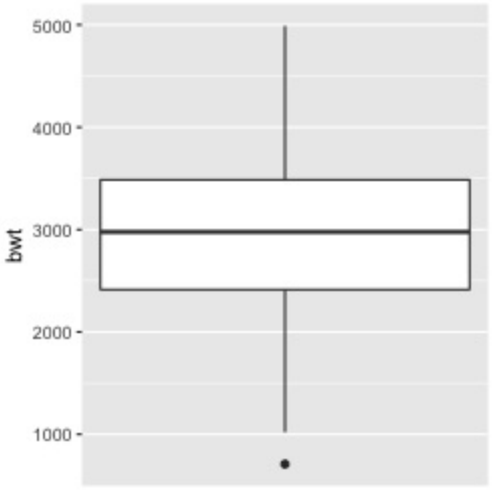
This will use smaller points, and hollow circles (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-WIDTH-POINT), right):

```
ggplot(birthwt, aes(x = factor(race), y = bwt)) +
    geom_boxplot(outlier.size = 1.5, outlier.shape = 21)
```



To make a box plot of just a single group, we have to provide some arbitrary value for x; otherwise, ggplot won't know what *x* coordinate to use for the box plot. In this case, we'll set it to 1 and remove the x-axis tick markers and label (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-SINGLE)):

```
ggplot(birthwt, aes(x = 1, y = bwt)) + geom_boxplot() +
    scale_x_continuous(breaks = NULL) +
    theme(axis.title.x = element_blank())
```



Note

The calculation of quantiles works slightly differently from the boxplot() function in base R. This can sometimes be noticeable for small sample sizes. See ?geom_boxplot for detailed information about how the calculations differ.

3.7 Adding Notches to a Box Plot

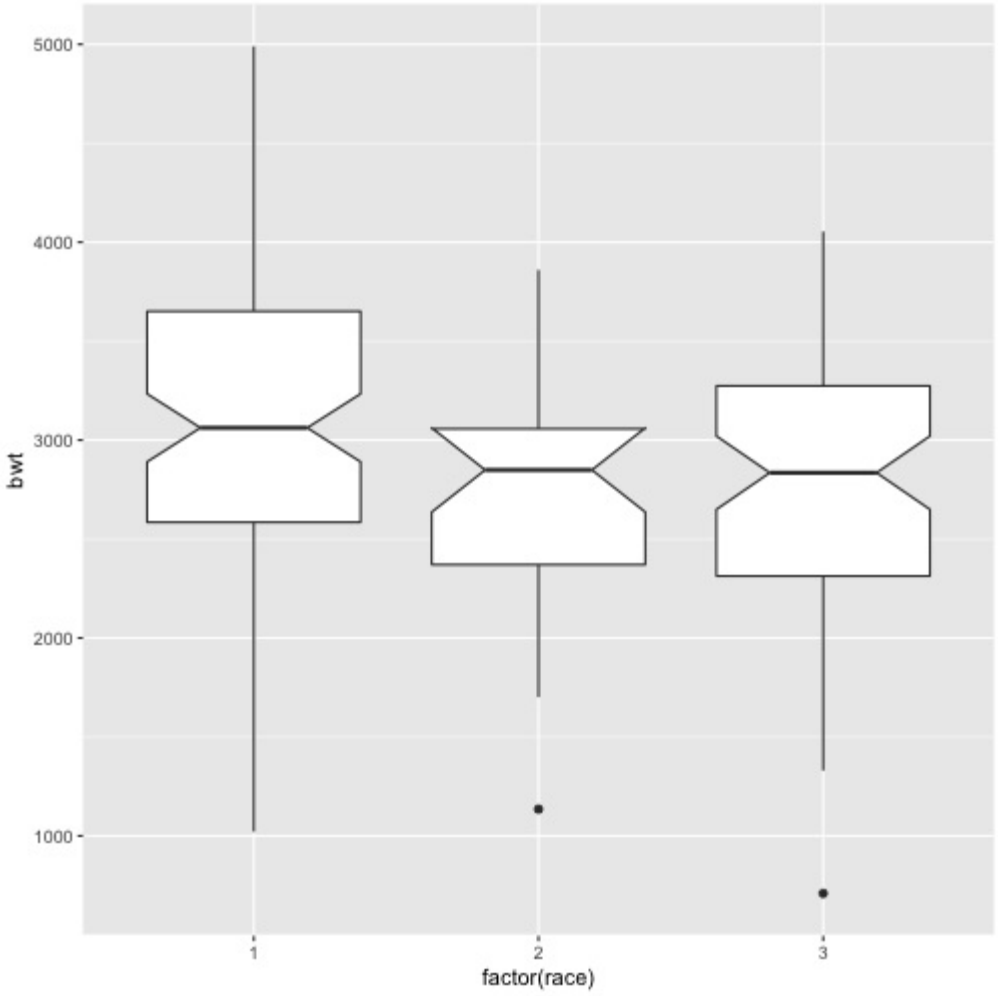
Problem

You want to add notches to a box plot to assess whether the medians are different.

Solution

Use `geom_boxplot()` and set `notch=TRUE` (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-NOTCH)):

```
library(MASS) # For the data set

ggplot(birthwt, aes(x = factor(race), y = bwt)) + geom_boxplot(notch = TRUE)
#> notch went outside hinges. Try setting notch=FALSE.
```



Discussion

Notches are used in box plots to help visually assess whether the medians of distributions differ. If the notches do not overlap, this is evidence that the medians are different.

With this particular data set, you'll see the following message:

```
Notch went outside hinges. Try setting notch=FALSE.
```

This means that the confidence region (the notch) went past the bounds (or hinges) of one of the boxes. In this case, the upper part of the notch in the middle box goes just barely outside the box body, but it's by such a small amount that you can't see it in the final output. There's nothing inherently wrong with a notch going outside the hinges, but it can look strange in more extreme cases.

3.8 Adding Means to a Box Plot

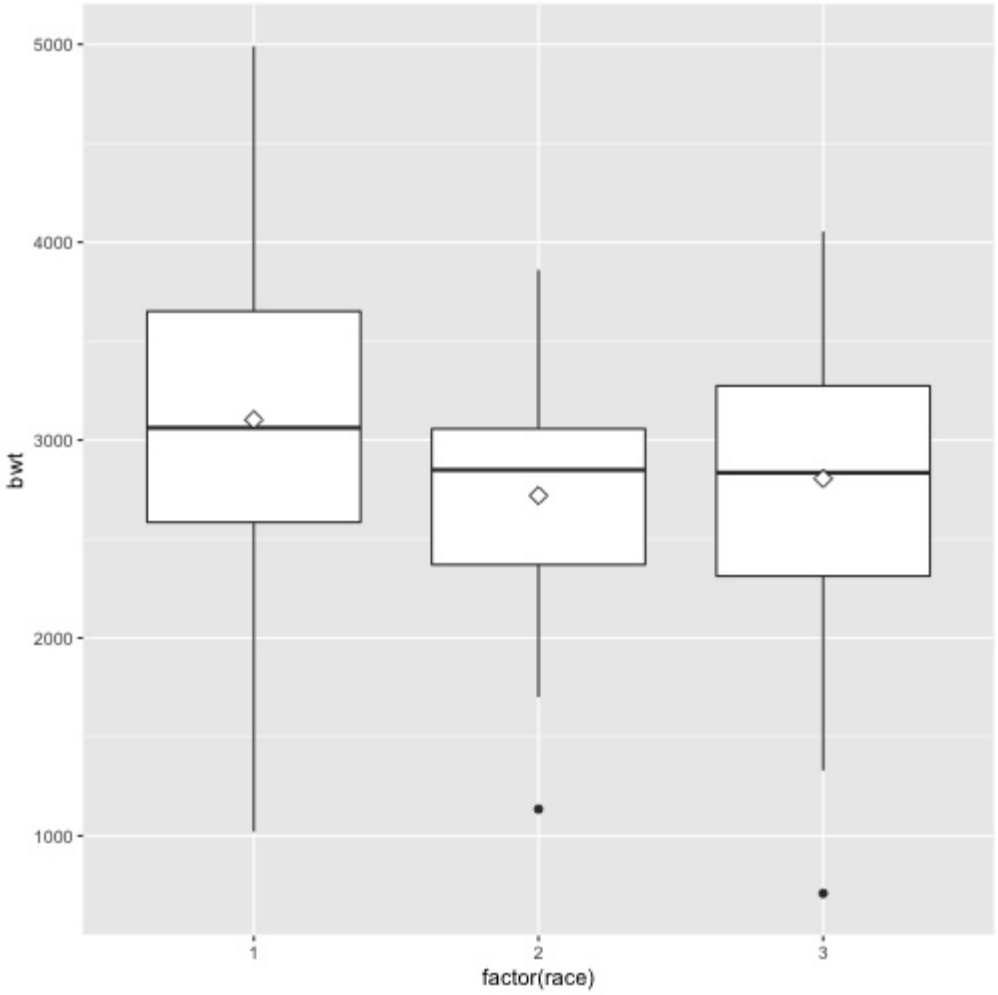
Problem

You want to add markers for the mean to a box plot.

Solution

Use `stat_summary()`. The mean is often shown with a diamond, so we'll use shape 23 with a white fill. We'll also make the diamond slightly larger by setting `size=3` (Figure \@ref(fig:FIG-DISTRIBUTION-BOXPLOT-MEAN)):

```
library(MASS) # For the data set

ggplot(birthwt, aes(x = factor(race), y = bwt)) + geom_boxplot() +
    stat_summary(fun.y = "mean", geom = "point", shape = 23, size = 3, fill = "white")
```



Discussion

The horizontal line in the middle of a box plot displays the median, not the mean. For data that is normally distributed, the median and mean will be about the same, but for skewed data these values will differ.

3.9 Making a Violin Plot

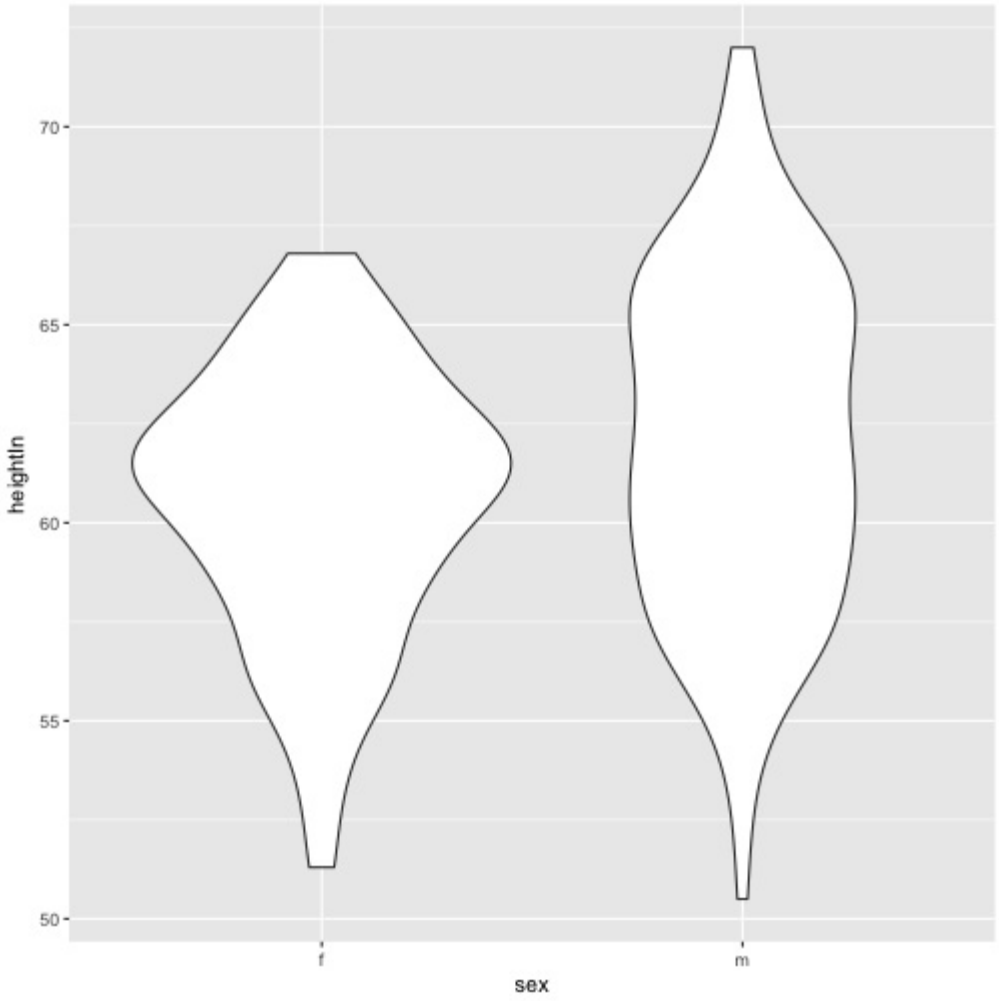
Problem

You want to make a violin plot to compare density estimates of different groups.

Solution

Use `geom_violin()` (Figure \@ref(fig:FIG-DISTRIBUTION-VIOLIN-BASIC)):

```
library(gcookbook) # For the data set

# Base plot
p <- ggplot(heightweight, aes(x=sex, y=heightIn))

p + geom_violin()
```

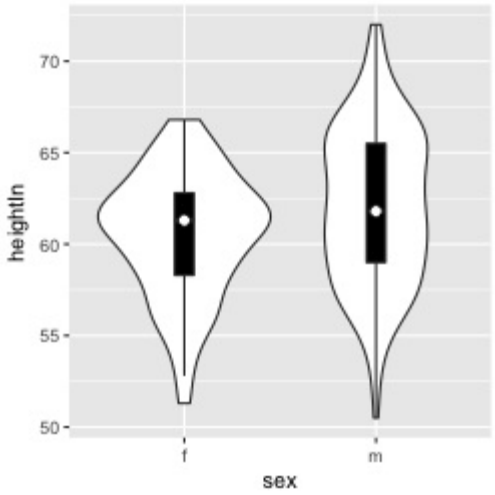


Discussion

Violin plots are a way of comparing multiple data distributions. With ordinary density curves, it is difficult to compare more than just a few distributions because the lines visually interfere with each other. With a violin plot, it's easier to compare several distributions since they're placed side by side.

A violin plot is a kernel density estimate, mirrored so that it forms a symmetrical shape. Traditionally, they also have narrow box plots overlaid, with a white dot at the median, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-VIOLIN-BOXPLOT). Additionally, the box plot outliers are not displayed, which we do by setting `outlier.colour=NA`:

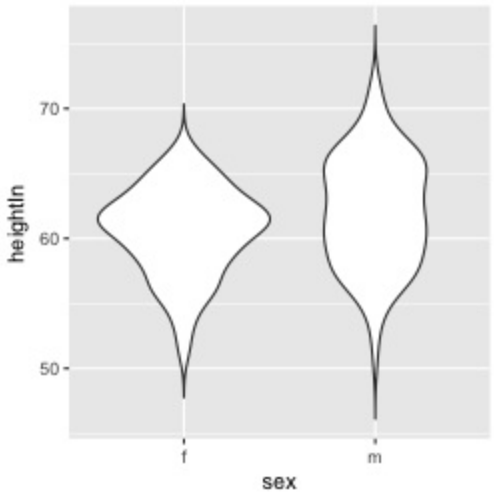
```
p + geom_violin() + geom_boxplot(width = .1, fill = "black", outlier.colour = NA) +
    stat_summary(fun.y = median, geom = "point", fill = "white", shape = 21, size = 2.5)
```



In this example we layered the objects from the bottom up, starting with the violin, then the box plot, then the white dot at the median, which is calculated using `stat_summary()`.

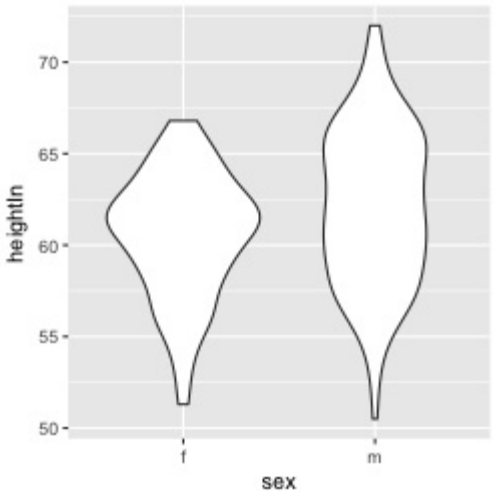
The default range goes from the minimum to maximum data values; the flat ends of the violins are at the extremes of the data. It's possible to keep the tails, by setting `trim=FALSE` (Figure \@ref(fig:FIG-DISTRIBUTION-VIOLIN-TAIL)):

```
p + geom_violin(trim = FALSE)
```



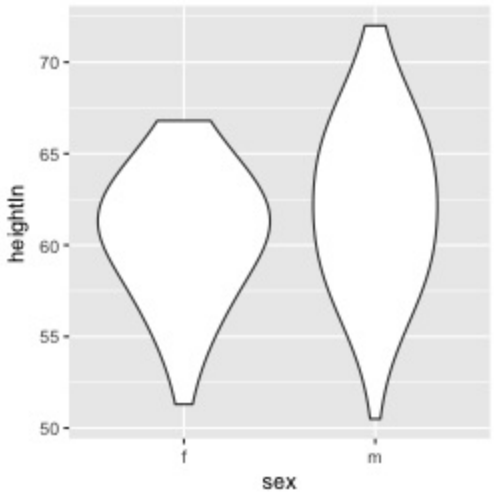
By default, the violins are scaled so that the total area of each one is the same (if `trim=TRUE`, then it scales what the area *would be* including the tails). Instead of equal areas, you can use `scale="count"` to scale the areas proportionally to the number of observations in each group (Figure \@ref(fig:FIG-DISTRIBUTION-VIOLIN-SCALECOUNT)). In this example, there are slightly fewer females than males, so the f violin is slightly narrower:

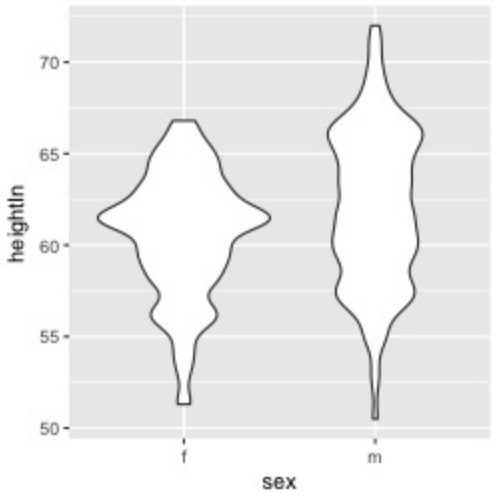
```
# Scaled area proportional to number of observations
p + geom_violin(scale = "count")
```



To change the amount of smoothing, use the adjust parameter, as described in Recipe \@ref(RECIPE-DISTRIBUTION-BASIC-DENSITY). The default value is 1; use larger values for more smoothing and smaller values for less smoothing (Figure \@ref(fig:FIG-DISTRIBUTION-VIOLIN-ADJUST)):

```
# More smoothing
p + geom_violin(adjust = 2)

# Less smoothing
p + geom_violin(adjust = .5)
```


70
65
60
55
50
heightIn
f
m
sex


70
65
60
55
50
heightIn
f
m
sex

See Also

To create a traditional density curve, see Recipe \@ref(RECIPE-DISTRIBUTION-BASIC-DENSITY).

To use different point shapes, see Recipe \@ref(RECIPE-LINE-GRAPH-POINT-APPEARANCE).

3.10 Making a Dot Plot

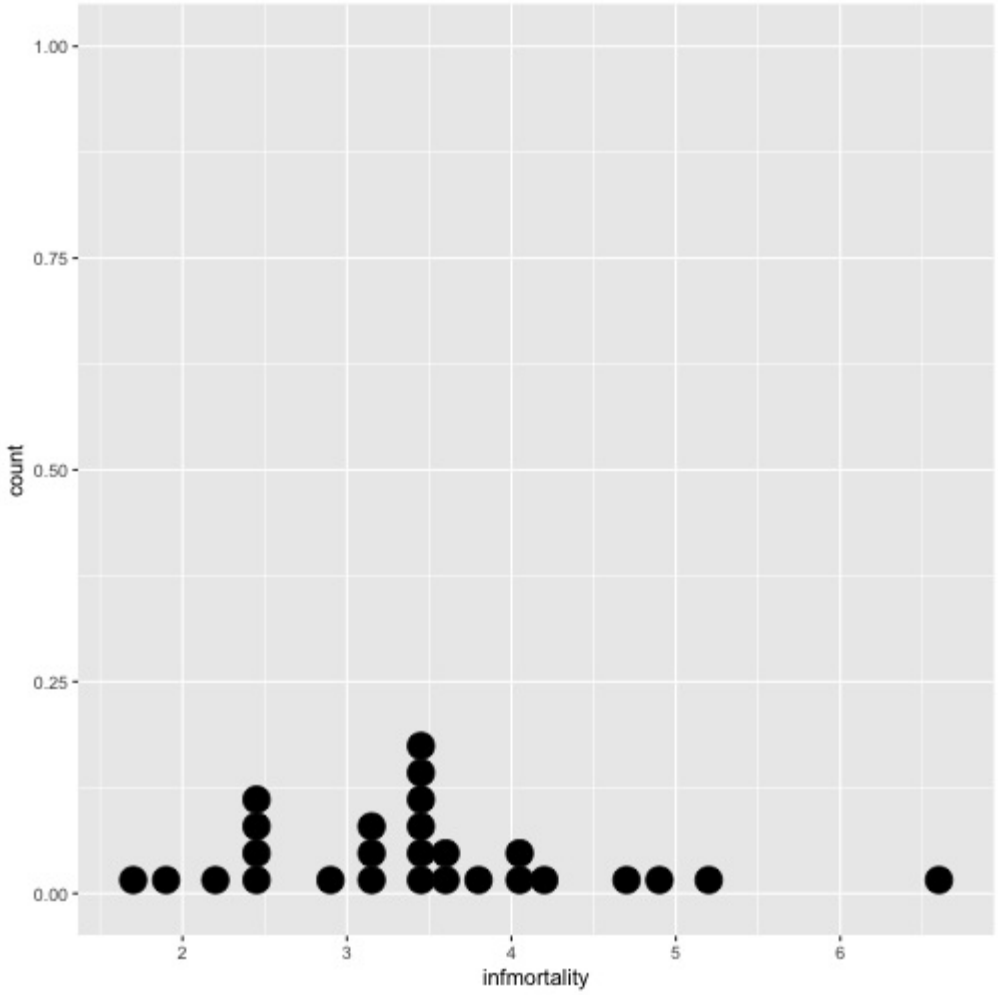
Problem

You want to make a Wilkinson dot plot, which shows each data point.

Solution

Use `geom_dotplot()`. For this example (Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-BASIC)), we'll use a subset of the `countries` data set:

```
library(gcookbook) # For the data set
countries2009 <- subset(countries, Year == 2009 & healthexp > 2000)

p <- ggplot(countries2009, aes(x = infmortality))

p + geom_dotplot()
#> `stat_bindot()` using `bins = 30`. Pick better value with `binwidth`.
```

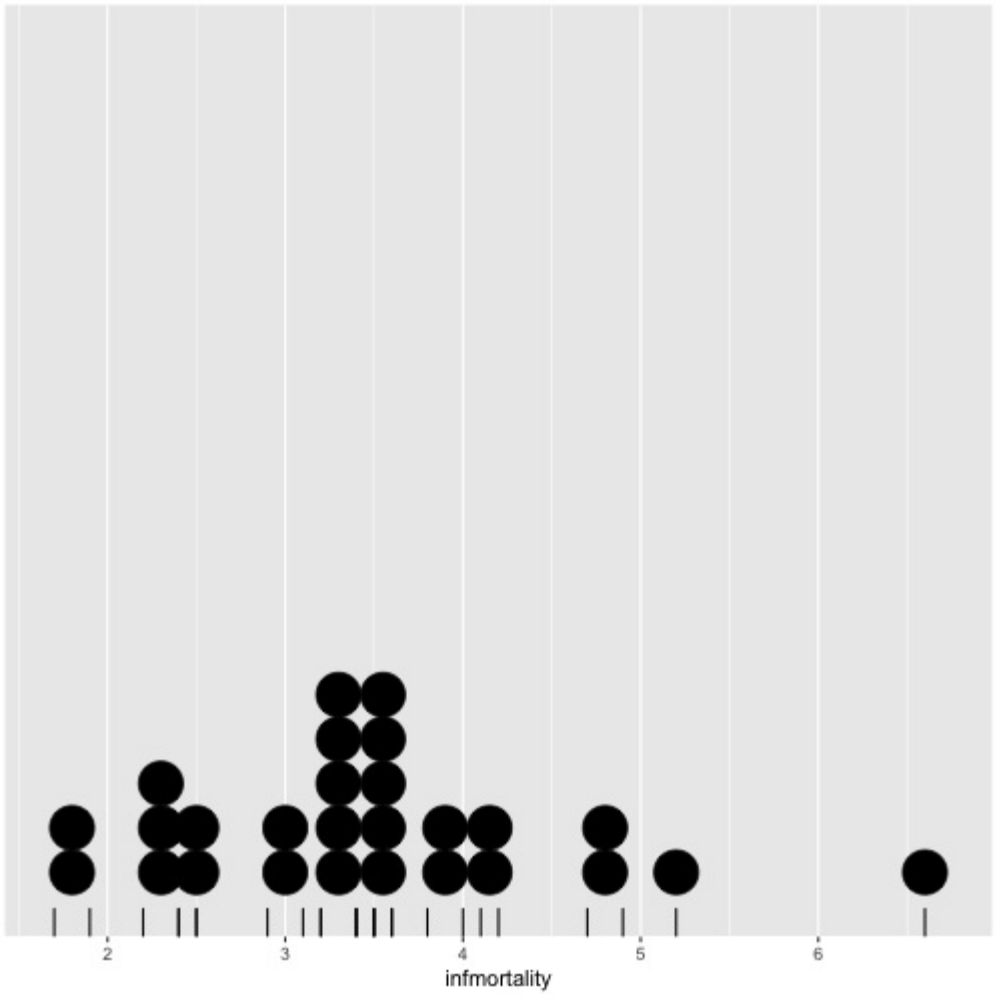


Discussion

This kind of dot plot is sometimes called a *Wilkinson* dot plot. It's different from the Cleveland dot plots shown in Recipe \@ref(RECIPE-BAR-GRAPH-DOT-PLOT). In these dot plots, the placement of the bins depends on the data, and the width of each dot corresponds to the maximum width of each bin. The maximum bin size defaults to 1/30 of the range of the data, but it can be changed with binwidth.

By default, `geom_dotplot()` bins the data along the x-axis and stacks on the y-axis. The dots are stacked visually, and for due to technical limitations of ggplot2, the resulting graph has y-axis tick marks that aren't meaningful. The y-axis labels can be removed by using `scale_y_continuous()`. In this example, we'll also use `geom_rug()` to show exactly where each data point is (Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-NO-Y-RUG)):

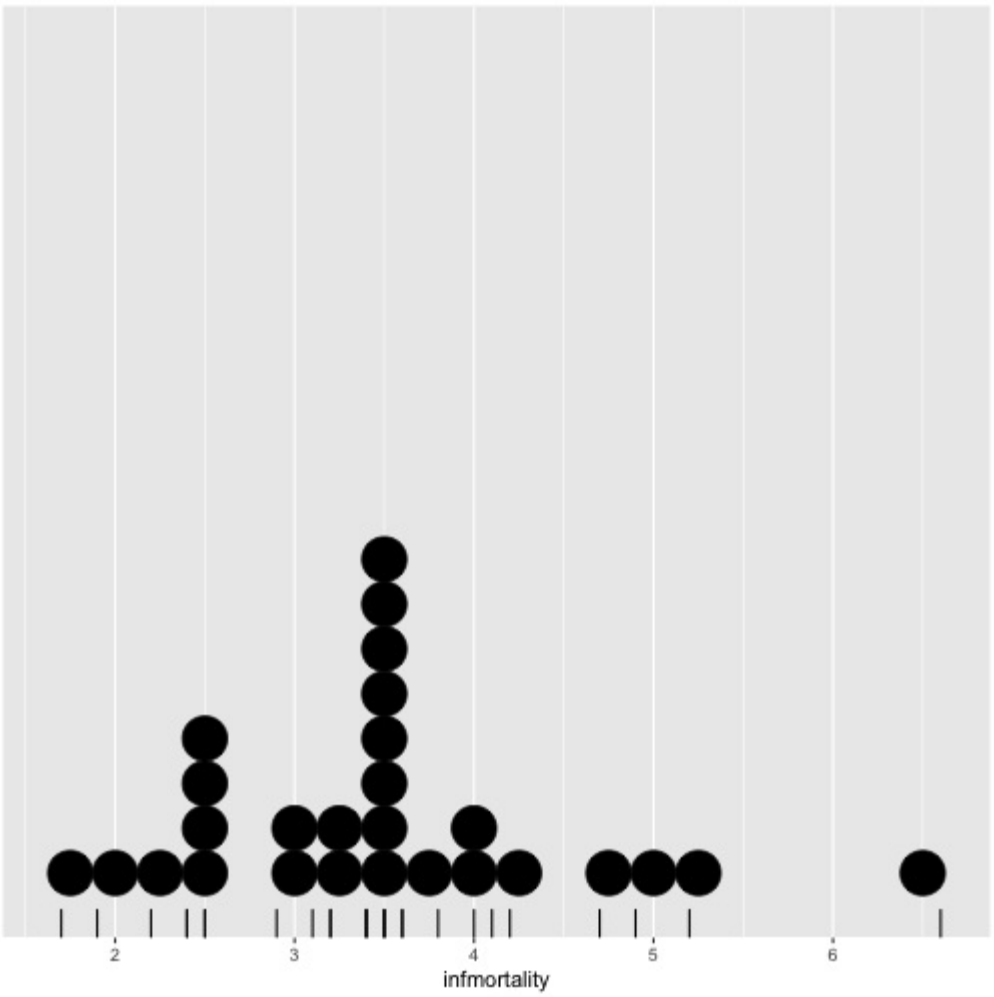
```
p + geom_dotplot(binwidth = .25) +
    geom_rug() +
    scale_y_continuous(breaks = NULL) +   # Remove tick markers
    theme(axis.title.y = element_blank()) # Remove axis label
```



You may notice that the stacks aren't regularly spaced in the horizontal direction. With the default dotdensity binning algorithm, the position of each stack is centered above the set of data

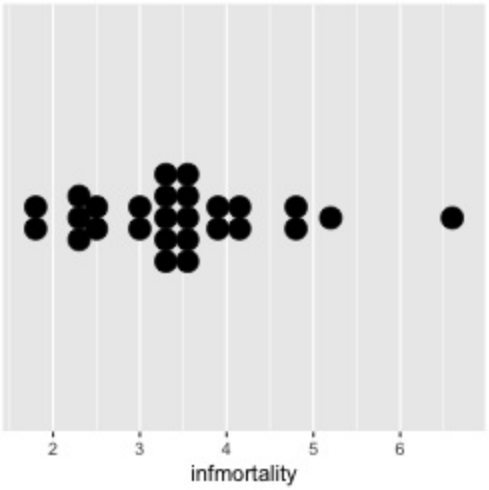
points that it represents. To use bins that are arranged with a fixed, regular spacing, like a histogram, use `method="histodot"`. In Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-HISTODOT), you'll notice that the stacks *aren't* centered above the data:

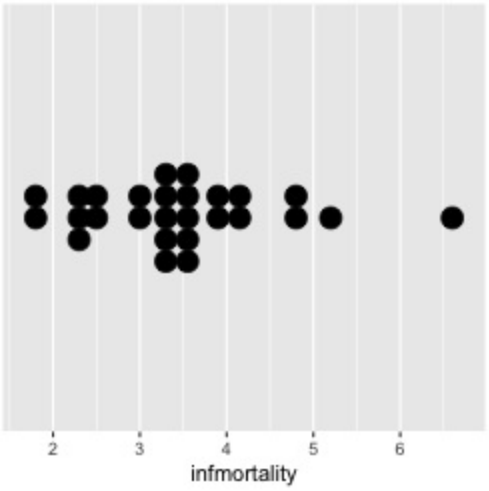
```
p + geom_dotplot(method = "histodot", binwidth = .25) +
    geom_rug() +
    scale_y_continuous(breaks = NULL) +
    theme(axis.title.y = element_blank())
```



The dots can also be stacked centered, or centered in such a way that stacks with even and odd quantities stay aligned. This can by done by setting `stackdir="center"` or `stackdir="centerwhole"`, as illustrated in Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-CENTER):

```
p + geom_dotplot(binwidth = .25, stackdir = "center") +
    scale_y_continuous(breaks = NULL) +
    theme(axis.title.y = element_blank())

p + geom_dotplot(binwidth = .25, stackdir = "centerwhole") +
    scale_y_continuous(breaks = NULL) +
    theme(axis.title.y = element_blank())
```


2
3
4
5
6
infmortality


2
3
4
5
6
infmortality

See Also

Leland Wilkinson, “Dot Plots,” *The American Statistician* 53 (1999): 276–281, http://www.cs.uic.edu/~wilkinson/Publications/dots.pdf.

3.11 Making Multiple Dot Plots for Grouped Data

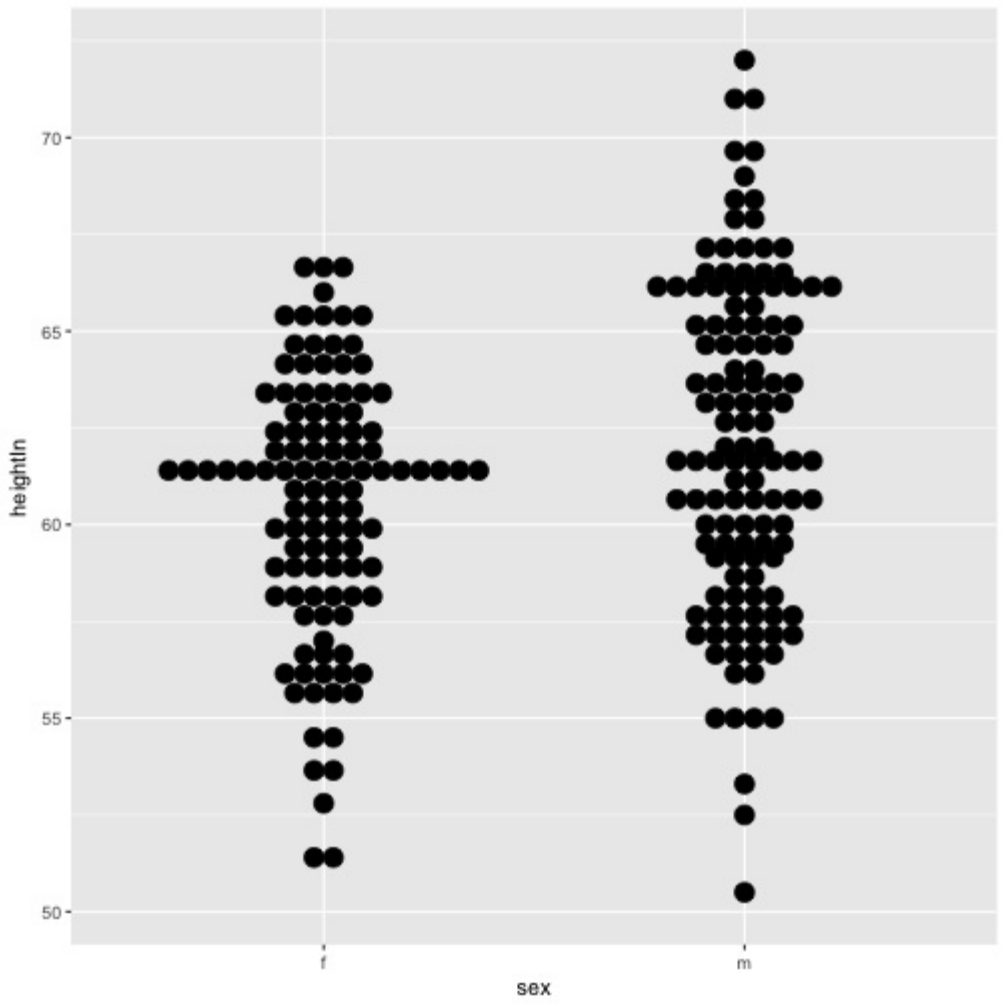
Problem

You want to make multiple dot plots from grouped data.

Solution

To compare multiple groups, it’s possible to stack the dots along the y-axis, and group them along the x-axis, by setting `binaxis="y"`. For this example, we’ll use the heightweight data set (Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-MULTI)):

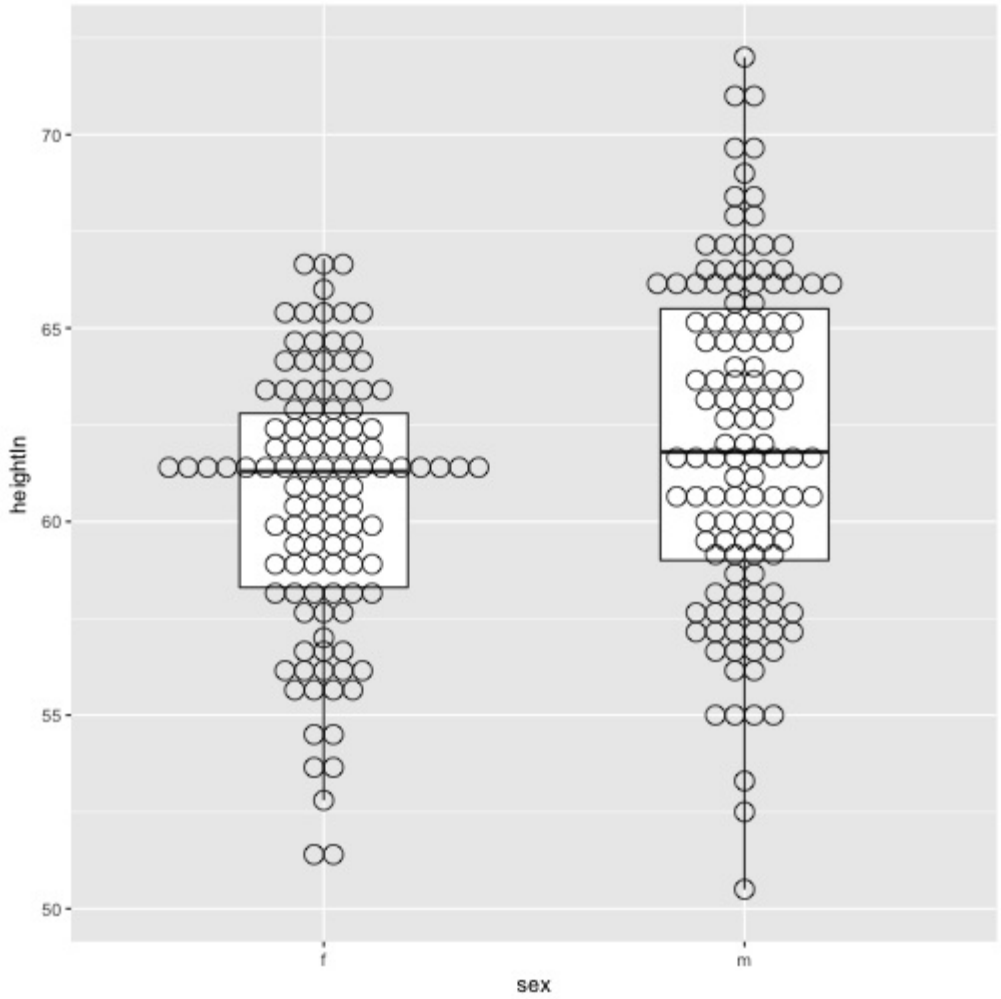
```
library(gcookbook) # For the data set

ggplot(heightweight, aes(x = sex, y = heightIn)) +
    geom_dotplot(binaxis = "y", binwidth = .5, stackdir = "center")
```



Discussion

Dot plots are sometimes overlaid on box plots. In these cases, it may be helpful to make the dots hollow and have the box plots *not* show outliers, since the outlier points will be shown as part of the dot plot (Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-MULTI-BOXPLOT)):

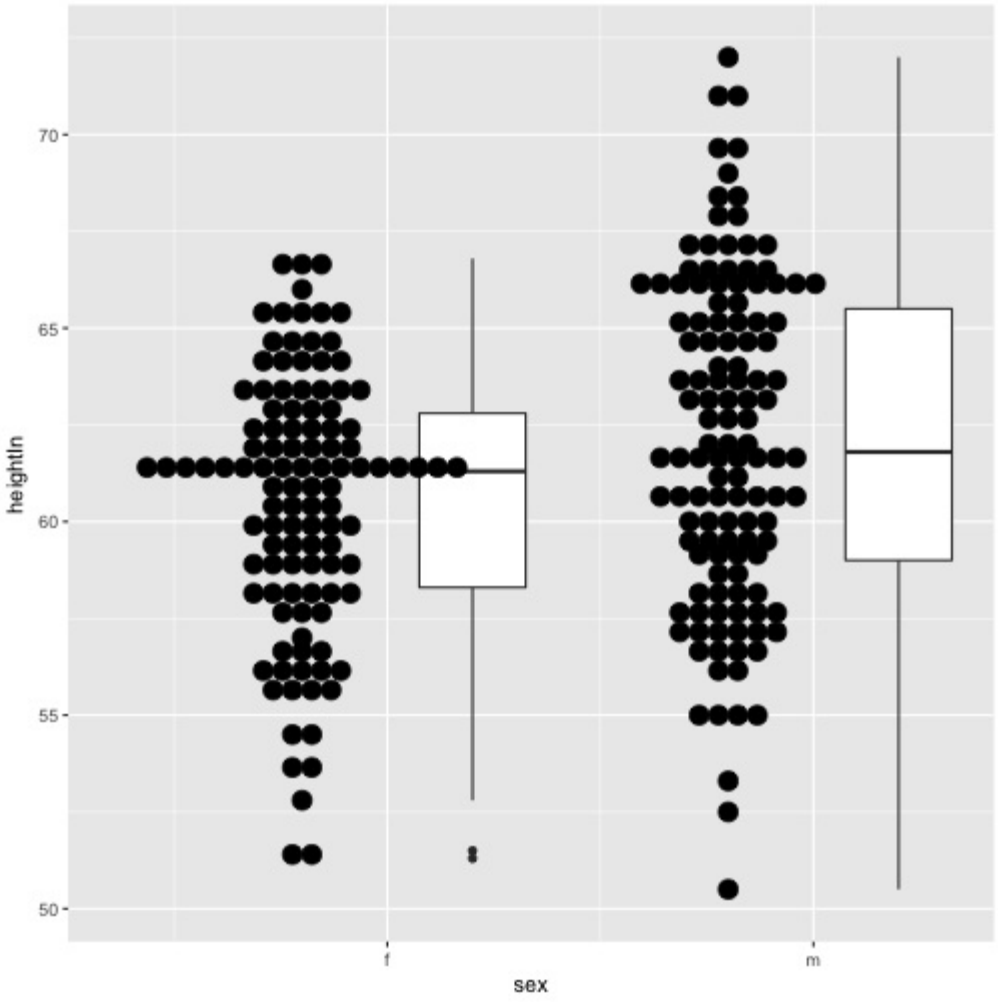
```
ggplot(heightweight, aes(x = sex, y = heightIn)) +
    geom_boxplot(outlier.colour = NA, width = .4) +
    geom_dotplot(binaxis = "y", binwidth = .5, stackdir = "center", fill = NA)
```



It’s also possible to show the dot plots next to the box plots, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-DOTPLOT-MULTI-SIDE). This requires using a bit of a hack, by treating the *x* variable as a numeric variable and subtracting or adding a small quantity to shift the box plots and dot plots left and right. When the *x* variable is treated as numeric you must also specify the group, or else the data will be treated as a single group, with just one box plot and dot plot. Finally, since the x-axis is treated as numeric, it will by default show numbers for the x-axis tick labels; they must be modified with `scale_x_continuous()` to show *x* tick labels as text corresponding to the factor levels:

```
ggplot(heightweight, aes(x = sex, y = heightIn)) +
    geom_boxplot(aes(x = as.numeric(sex) + .2, group = sex), width = .25) +
    geom_dotplot(aes(x = as.numeric(sex) - .2, group = sex), binaxis = "y",
```

```
              binwidth = .5, stackdir = "center") +
scale_x_continuous(breaks = 1:nlevels(heightweight$sex),
                   labels = levels(heightweight$sex))
```



3.12 Making a Density Plot of Two-Dimensional Data

Problem

You want to plot the density of two-dimensional data.

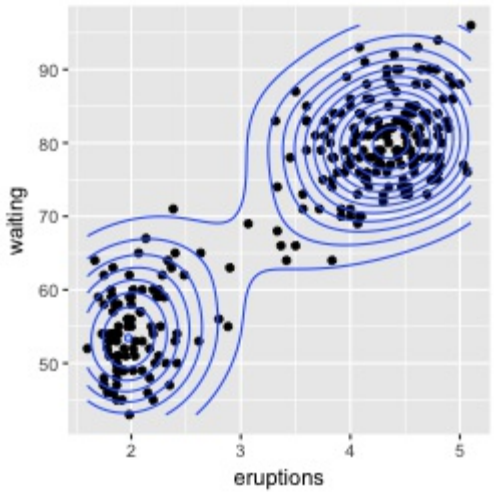
Solution

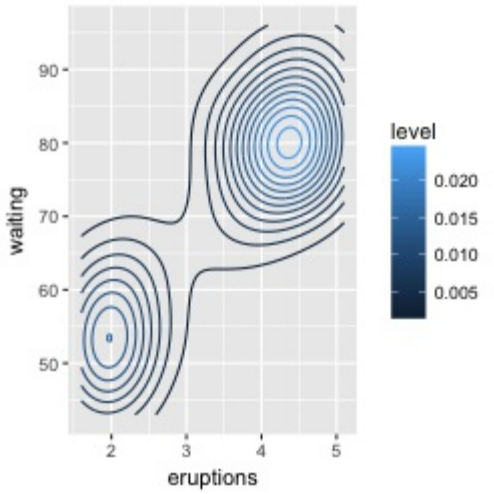
Use `stat_density2d()`. This makes a 2D kernel density estimate from the data. First we'll plot the density contour along with the data points (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY2D), left):

```
# The base plot
p <- ggplot(faithful, aes(x=eruptions, y=waiting))

p + geom_point() + stat_density2d()
```

It's also possible to map the *height* of the density curve to the color of the contour lines, by using ..level.. (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY2D), right):

```
# Contour lines, with "height" mapped to color
p + stat_density2d(aes(colour=..level..))
```

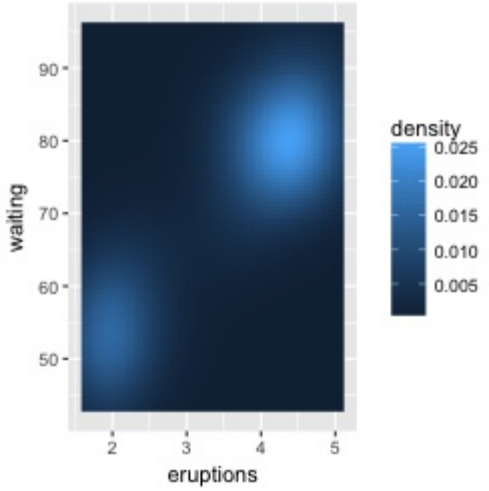


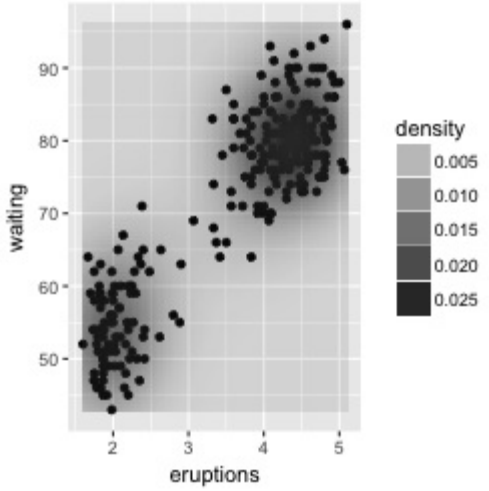


Discussion

The two-dimensional kernel density estimate is analogous to the one-dimensional density estimate generated by `stat_density()`, but of course, it needs to be viewed in a different way. The default is to use contour lines, but it's also possible to use tiles and map the density estimate to the fill color, or to the transparency of the tiles, as shown in Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY2D-TILE):

```
# Map density estimate to fill color
p + stat_density2d(aes(fill = ..density..), geom = "raster", contour = FALSE)

# With points, and map density estimate to alpha
p + geom_point() +
    stat_density2d(aes(alpha = ..density..), geom = "tile", contour = FALSE)
```





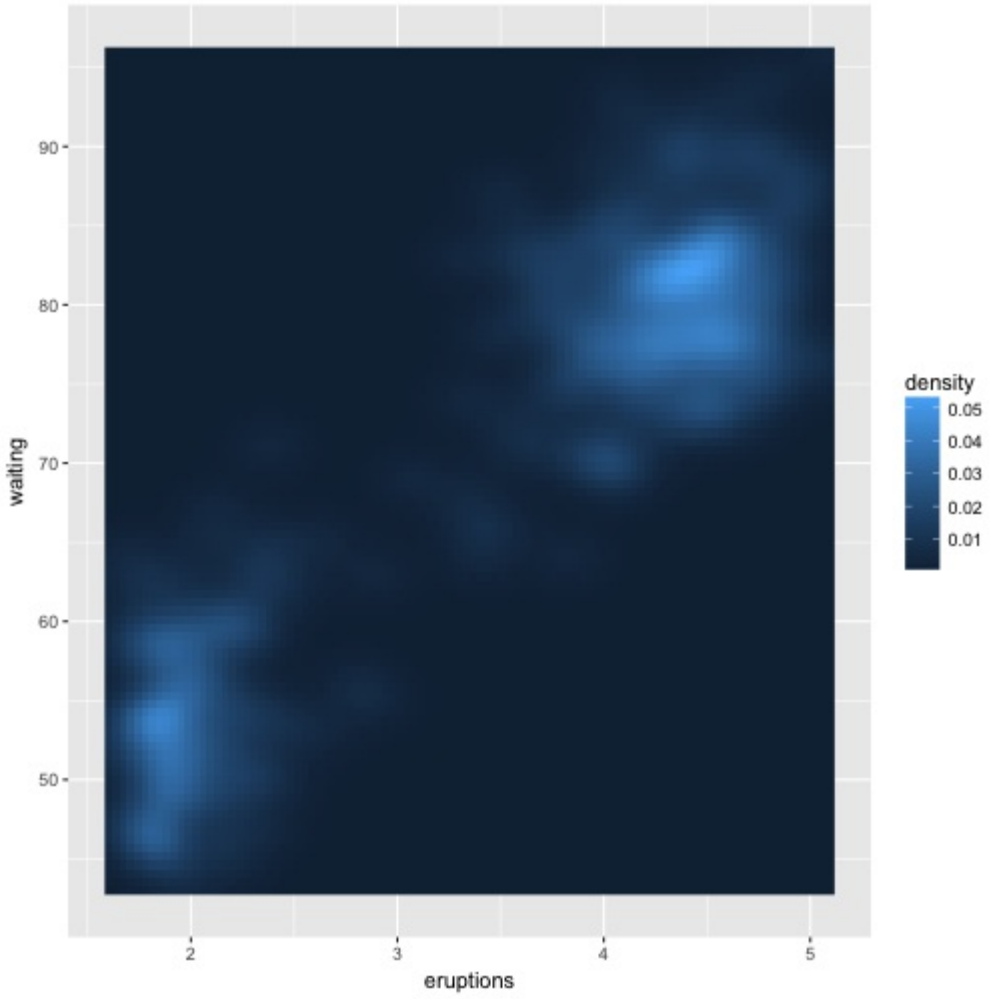
Note

We used `geom="raster"` in the first of the preceding examples and `geom="tile"` in the second. The main difference is that the raster geom renders more efficiently than the tile geom. In theory they *should* appear the same, but in practice they often do not. If you are writing to a PDF file, the appearance depends on the PDF viewer. On some viewers, when tile is used

there may be faint lines between the tiles, and when raster is used the edges of the tiles may appear blurry (although it doesn't matter in this particular case).

As with the one-dimensional density estimate, you can control the bandwidth of the estimate. To do this, pass a vector for the *x* and *y* bandwidths to h. This argument gets passed on to the function that actually generates the density estimate, `kde2d()`. In this example (Figure \@ref(fig:FIG-DISTRIBUTION-DENSITY2D-BANDWIDTH)), we'll use a smaller bandwidth in the *x* and *y* directions, so that the density estimate is more closely fitted (perhaps overfitted) to the data:

```
p + stat_density2d(aes(fill = ..density..), geom = "raster",
                   contour = FALSE, h = c(.5,5))
```



See Also

The relationship between `stat_density2d()` and `stat_bin2d()` is the same as the relationship between their one-dimensional counterparts, the density curve and the histogram. The density curve is an *estimate* of the distribution under certain assumptions, while the binned visualization represents the observed data directly. See Recipe \@ref(RECIPE-SCATTER-OVERPLOT) for more about binning data.

If you want to use a different color palette, see Recipe \@ref(RECIPE-COLORS-PALETTE-CONTINUOUS).

`stat_density2d()` passes options to `kde2d()`; see `?kde2d` for information on the available options.

About the Author

Winston Chang is a software engineer at RStudio, where he works on data visualization and software development tools for R. He has a Ph.D. in Psychology from Northwestern University, and created the Cookbook for R website, which contains recipes for common tasks in R.

Colophon

The animal on the cover of *R Graphics Cookbook* is a reindeer (*Rangifer tarandus*), also known as caribou in North America, which is a species of deer native to Arctic and Subarctic regions. Reindeer are ideally designed for life in hostile, cold environments, as their fur, antlers, noses, hooves, and vision have adapted to the low temperatures.

Their fur coat consists of an outer layer of straight, hollow, tubular hairs, which provide insulation from the cold and buoyancy in water, and a woolly undercoat. The coat is such an efficient insulator that when they lay on the snow, the snow does not melt. Reindeer are the only species of deer in which both male and female (and even calves) have antlers, and they have the largest antlers relative to body size among living deer species. Their antlers are shed annually and new antler growth occurs in the spring and summer.

Reindeer hooves adapt to the season: in the summer, when the tundra is soft and wet, the footpads become sponge-like and provide extra traction. In the winter, the pads shrink and tighten, exposing the rim of the hoof, which cuts into the ice and crusted snow to keep the deer from slipping. This also enables them to dig down (an activity known as cratering) through the snow to their favorite food, a lichen known as reindeer moss.

In 2012, researchers at University College London discovered reindeer are the only mammals that can see ultraviolet light. While human vision cuts off at wavelengths around 400 nm, reindeer can see up to 320 nm. This range only covers the part of the spectrum we can see with the help of a black light, but it is still enough to help reindeer see things in the glowing white of the Arctic that they would otherwise miss.

In the Santa Claus tale, Santa Claus's sleigh is pulled by flying reindeer. These were first named in the 1823 poem "A Visit from St. Nicholas," where they are called Dasher, Dancer, Prancer, Vixen, Comet, Cupid, Dunder, and Blixem.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. To learn more about how you can help, go to *animals.oreilly.com*.

The cover image is from Shaw's *Zoology*. The cover fonts are URW Typewriter and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.